ACPC Notes — Contents

1 Contest Playbook 1
1.1 ACPC, Kattis, and Team Strategy 1
1.1.1 What ACPC looks like (from all sets 2016–2024 on Kattis) 1
1.1.2 Kattis specifics 1
1.1.3 Team strategy (2–3 people, one machine) 1
1.1.4 Precision and number sizes (memorize) 2
1.2 Template, Debugging, Stress Testing 2
1.2.1 Main template 2
1.2.2 Useful snippets 2
1.2.3 Stress testing (do this after the first unexplained WA) 3
1.3 When You're Stuck 3
1.3.1 Constraints → target complexity 3
1.3.2 Keyword → technique 4
1.3.3 Stuck-on-an-idea checklist 5
1.3.4 WA / RE / TLE checklist 6
1.4 ACPC 2016–2024: Every Problem → Technique 6
1.5 Worked Patterns from Past ACPC Problems 8
1.5.1 Binary search on the answer 8
1.5.2 Shortest paths in disguise 9
1.5.3 DP 9
1.5.4 Flows and matching 9
1.5.5 Number theory / interactive 9
1.5.6 Strings 9
1.5.7 Offline / data structures 10
2 Core Techniques 11
2.1 Binary Search, Ternary Search, Parametric Search 11
2.1.1 Integer binary search on the answer 11
2.1.2 Real-valued binary search 11
2.1.3 Ternary search (unimodal functions) 11
2.1.4 Parametric search / fractional programming 12
2.1.5 Binary lifting search ("walk down powers of two") 12
2.1.6 Parallel binary search (offline, many queries) 12
2.2 Prefix Sums, Difference Arrays, Sweep Line, Contribution 12
2.2.1 Prefix sums (1D/2D) and difference arrays 12
2.2.2 Coordinate compression 12
2.2.3 Sweep line 13
2.2.4 Contribution technique (sum over all subsets/subarrays/pairs) 13
2.2.5 Offline tricks summary 13
2.3 Two Pointers, Monotonic Stack, Monotonic Deque 13
2.3.1 Two pointers / sliding window 13
2.3.2 Monotonic stack: nearest greater / smaller 14
2.3.3 Monotonic deque: sliding window min/max 14
2.3.4 Other amortised patterns 14
2.4 Greedy and Exchange Arguments 14
2.4.1 Finding the sort key by exchange 14
2.4.2 Interval greedies 15
2.4.3 Deadline scheduling 15
2.4.4 Heap greedies 15
2.4.5 Other common greedy facts 15
2.5 Complete Search, Meet in the Middle, Square Root, Randomization 15
2.5.1 Brute force done right 15
2.5.2 Meet in the middle 16
2.5.3 Square root decomposition 16
2.5.4 Randomization 16
2.5.5 Iterative deepening / search with memo 16
2.6 Bit Manipulation and Bitset 17
2.6.1 Tricks 17
2.6.2 XOR facts 17
2.6.3 Bitset 17
2.6.4 Gray code 17
2.7 Constructive and Ad-hoc Problems 18
2.7.1 Invariants and monovariants 18
2.7.2 Construction toolbox 18
2.7.3 Permutations and arrays 18
2.7.4 Ad-hoc reading discipline 18
3 Data Structures 19
3.1 Disjoint Set Union 19
3.1.1 Uses 19
3.1.2 DSU with parity / potential (weighted DSU) 19
3.1.3 DSU with rollback (for offline dynamic connectivity / divide & conquer) 19
3.2 Fenwick Tree (BIT) 20
3.2.1 Range update, point query / range update, range query 20
3.2.2 2D BIT 20
3.3 Segment Tree 21
3.3.1 Iterative (point update, range query) — fastest to type 21
3.3.2 Lazy propagation: range assign + range add, query sum/min/max 21
3.3.3 Useful node designs 22
3.3.4 Persistent segment tree: k-th smallest in a range 22
3.3.5 Variations (when you need them) 23
3.4 Sparse Table, Ordered Set, Treap, Mo's, Small-to-Large 23
3.4.1 Sparse table (static RMQ in O(1)) 23
3.4.2 Ordered set (PBDS) 23
3.4.3 Implicit treap (sequence with split/merge, reverse, range sum) 23
3.4.4 Mo's algorithm (offline range queries, O((n + q)√n)) 24
3.4.5 Small-to-large merging 24
3.4.6 Other standard-library power tools 25
4 Dynamic Programming 26
4.1 DP Design and Classic Patterns 26
4.1.1 Designing a DP 26
4.1.2 Knapsack family 26
4.1.3 Sequences 26
4.1.4 Interval DP 27
4.1.5 Grid, DAG, partition DPs 27
4.1.6 Counting DPs (mod p) 27
4.2 Bitmask DP, Digit DP, SOS, Broken Profile 27
4.2.1 Bitmask DP 27
4.2.2 SOS DP (sum over subsets) 28
4.2.3 Digit DP 28
4.2.4 Broken profile DP (tilings) 28
4.3 Tree DP and Rerooting 29
4.3.1 Standard subtree DPs 29
4.3.2 Tree knapsack (merge children with size bounds) — $O(n^2)$ total 29
4.3.3 Rerooting (answer for every root) 30
4.3.4 Other tree techniques (pointers) 30
4.4 DP Optimisations 30
4.4.1 Convex hull trick (any insertion/query order) 30
4.4.2 Li Chao tree (min, over a known set of query x's) 31
4.4.3 Divide & conquer optimisation 31
4.4.4 Knuth optimisation (interval DP) 31
4.4.5 Aliens trick (Lagrangian relaxation) 32
4.4.6 Slope trick 32
4.5 Probability, Expected Value, Games 32
4.5.1 Core facts 32
4.5.2 Expected value DP 32
4.5.3 Combinatorial games 33
5 Graphs and Trees 34
5.1 Shortest Paths and Graph Search 34
5.1.1 Basics: bipartite check, cycle detection 34
5.1.2 BFS on grids (template) 34
5.1.3 0-1 BFS 35
5.1.4 Dijkstra 35
5.1.5 Bellman–Ford and negative cycles 36
5.1.6 Floyd–Warshall 36
5.2 MST, DAGs, Functional Graphs, Permutations 36
5.2.1 Minimum spanning tree 36
5.2.2 Topological sort and DAG DP 37
5.2.3 Functional graphs (each node has exactly one outgoing edge) 37
5.2.4 Permutations 37
5.3 SCC, 2-SAT, Bridges, Articulation Points, Euler Paths 38
5.3.1 Strongly connected components (Tarjan) 38
5.3.2 2-SAT 38
5.3.3 Bridges and articulation points 38
5.3.4 Euler path / circuit (Hierholzer) 39

5.4 Trees: LCA, Euler Tour, HLD, Centroid Decomposition 39
5.4.1 LCA with binary lifting (+ distances, k-th ancestor, path aggregates) 39
5.4.2 Heavy-light decomposition 40
5.4.3 Centroid decomposition 40

6 Flows, Matching, Assignment 42
6.1 Max Flow, Min Cut, and Modelling 42
6.1.1 Dinic 42
6.1.2 Modelling cookbook 42
6.1.3 Flow theorems worth knowing 43
6.2 Bipartite Matching and Covers 43
6.2.1 Hopcroft–Karp (O(E √V)) 43
6.2.2 Facts (bipartite graphs, V = L ∪ R) 44
6.2.3 Stable marriage (Gale–Shapley) 44
6.3 Min-Cost Flow and the Assignment Problem 45
6.3.1 Hungarian algorithm (n ≤ m, O(n²m)) 45
6.3.2 Min-cost max-flow (successive shortest paths with SPFA) 45

7 Math 47
7.1 Modular Arithmetic, GCD, CRT, Binomials 47
7.1.1 Basics 47
7.1.2 Factorials and binomials 47
7.1.3 Linear Diophantine equations and CRT 47
7.1.4 Discrete logarithm (baby-step giant-step) 48
7.1.5 GCD / LCM facts 48
7.1.6 Useful closed forms 48
7.2 Primes, Factorisation, Divisors, Multiplicative Functions 48
7.2.1 Sieves 48
7.2.2 Trial division (n ≤ 10^12–10^14) and factoring big numbers 49
7.2.3 Divisors 49
7.2.4 Euler's totient and Möbius 50
7.2.5 Other facts 50
7.3 Counting 50
7.3.1 The standard toolbox 50
7.3.2 Identities 51
7.3.3 Inclusion–exclusion 51
7.3.4 Burnside / Pólya (counting up to symmetry) 51
7.3.5 Probability-flavoured counting 51
7.4 Matrices, Linear Recurrences, Gaussian Elimination, XOR Basis 51
7.4.1 Matrix exponentiation 51
7.4.2 Berlekamp–Massey: guess the recurrence from terms 52
7.4.3 Gaussian elimination (reals) 52
7.4.4 Polynomial interpolation (Lagrange) 53
7.4.5 XOR basis (Gaussian elimination over GF(2)) 53
7.5 FFT / NTT (Convolution) 54
7.5.1 FFT with doubles (exact for integer results up to ~10^14) 54
7.5.2 NTT modulo 998244353 (exact) 54
7.5.3 Applications 55

8 Strings 56
8.1 Hashing, Prefix Function (KMP), Z-Function 56
8.1.1 Polynomial hashing (mod 2^61 − 1, random base) 56
8.1.2 Prefix function and KMP 56
8.1.3 Z-function 57
8.2 Palindromes, Rotations, Tries, Aho–Corasick, Suffix Structures 57
8.2.1 Manacher (all maximal palindromes in O(n)) 57
8.2.2 Minimal rotation / canonical cyclic form 57
8.2.3 Trie 58
8.2.4 Aho–Corasick (many patterns in one text) 58
8.2.5 Suffix array + LCP 58
8.2.6 Suffix automaton (when you need counts of occurrences of every substring) 59

9 Geometry 60
9.1 Geometry Primitives 60
9.1.1 Point / vector 60
9.1.2 Segments and lines 60
9.1.3 Angles and sorting by angle 61
9.1.4 Polygons 61
9.1.5 Precision checklist 61
9.2 Convex Hull, Calipers, Closest Pair, Circles, Integration 62
9.2.1 Convex hull (monotone chain) 62
9.2.2 Rotating calipers 62
9.2.3 Closest pair of points 62
9.2.4 Circles 63
9.2.5 Sphere / 3-D quick facts 63
9.2.6 Numerical integration and root finding 63

10 Implementation, Interactive, Misc 64
10.1 Interactive Problems 64
10.1.1 Protocol checklist 64
10.1.2 Strategy patterns 64
10.1.3 Testing locally 64
10.2 Parsing, Simulation, Dates, Big Numbers 65
10.2.1 Reading input safely 65
10.2.2 Expression parsing (recursive descent) 65
10.2.3 Simulation tips 65
10.2.4 Dates and times 66
10.2.5 Big numbers without Python 66
10.2.6 Base conversion and misc 66
10.3 Python as a Backup Language 66

1. Contest Playbook

1.1 ACPC, Kattis, and Team Strategy

How to use these notes. Stuck on a problem → go to *When You're Stuck* (§1.3): constraint → complexity table, then keyword → technique table, which points to the section with code. Every C++ block compiles as-is after the template (§1.2) with `#include <bits/stdc++.h>`, `using namespace std;`, `using ll = long long;`. Blocks marked **[stress-tested]** were checked against brute force; the rest compile and were hand-verified, so still test on samples.

1.1.1 What ACPC looks like (from all sets 2016–2024 on Kattis)

- 8–15 problems per set. Kattis difficulty (1–10 scale) spread is very consistent:
- **~35% easy (1.2–2.9):** implementation, careful reading, simulation. Solve these *fast and without WA* — penalty time decides ranks among teams with equal solves.
- **~35% medium (3.0–5.0):** one standard idea: binary search on answer + greedy, monotonic stack, Dijkstra/BFS with a twist, tree DP, counting mod p, interval DP, grid flood fill, offline DSU.
- **~30% hard (5.1–8.3):** probability/expected-value DP, flows with modelling (min flow with lower bounds, assignment), FFT, number theory (factoring, gcd tricks), interactive problems with tiny query budgets, geometry + search, string canonical forms, nasty interval DP.
- Expert/CM level: easy+medium ≈ CF Div2 A–C; hard ≈ Div2 D–F / Div1 C–D. Hard ACPC problems are usually *one known technique plus modelling* rather than research-level tricks — recognizing the model is the whole game. See §1.4 for every past problem mapped to its technique.
- Themes recur: Kattis "Medium/Hard" problems from ACPC very often involve **probability**, **flows/matching**, **Dijkstra on augmented state**, **binary search on the answer**, and **number theory**.

1.1.2 Kattis specifics

- Language: C++ compiled with GCC, `-O2`, gnu++17 or newer, 64-bit. `__int128`, `__builtin_*`, `<bits/stdc++.h>`, `__gnu_pbds` all work.
- Time limit is **per test file**; there may be dozens of files. Memory is usually 1024 MB (check the problem page).
- Verdicts stop at the first failing test file — you don't learn which case failed. A WA on test 1 means the sample itself fails (or format is wrong).
- Output: trailing whitespace/newline differences are ignored. Floating answers are accepted within the stated absolute/relative error; print with `fixed << setprecision(10)`.
- **Interactive problems** (ACPC 2024 had two): flush after every query (`cout << ... << endl` or `cout.flush()`), never read past what the judge sends, terminate immediately after the final answer. See §10.1.
- Multiple test cases inside one file (`T` given): **reset all global state**.
- Python: big integers are trivial in Python; consider it for one-off big-number problems (e.g. ACPC 2017 *Divide by 100* has numbers with 10^6 digits — just string manipulation though).
- Recursion: Kattis has a generous stack, but DFS depth 10^6 in C++ is still risky; prefer iterative for path graphs with $n \geq 10^6$.

1.1.3 Team strategy (2–3 people, one machine)

1. **First 15 min:** split the problem set; everyone reads, labels each problem `easy / medium / hard / unknown` and writes one-line summary + constraints on paper. Read *all* problems — ACPC difficulty is not ordered by letter.
2. **Keyboard discipline:** only one person types; others think on paper. Never debug at the keyboard while someone with a ready solution waits — print/write the code and debug on paper.
3. Watch the scoreboard: problems solved by many teams are easier than they look. If 10 teams solved X and you think it's hard, you're missing something simple.
4. **Before submitting:** run all samples, a tiny edge case ($n = 1$, all equal, max values), and check output format. One WA costs 20 minutes of penalty.
5. **After WA:** reread the statement *first* (most WAs are misreads), then check §1.3 bug list, then write a brute force and compare on random small inputs.
6. **Last hour:** stop starting new hard problems unless the idea is complete; finish/debug what's closest. Freeze-time submissions still count.
7. Two people can pair on the hardest problem: one explains the problem from scratch to the other — this exposes wrong assumptions.

1.1.4 Precision and number sizes (memorize)

type	max	notes
`int`	$2.1 \cdot 10^9$	`a*b` overflows when both $\sim 5 \cdot 10^4$
`long long`	$9.2 \cdot 10^{18}$	product of two 10^9 values fits; two 10^{10} doesn't
`__int128`	$1.7 \cdot 10^{38}$	multiply two `ll` safely; no I/O, write a printer
`double`	15–16 digits	integers exact up to $2^{53} \approx 9 \cdot 10^{15}$
`long double`	18–19 digits	80-bit on x86 GCC; slower but helps precision

- Operations/second: $\sim 3 \cdot 10^8$ simple ops in 1 s with `-O2`; `set/map` $\sim 10^7$ ops/s; `unordered_map` $\sim 3 \cdot 10^7$ with reserve; `cin` with sync off $\sim 10^7$ ints/s.
- Memory: `int` array of 10^8 = 400 MB. `vector<vector<int>>` with 10^6 small vectors has big overhead — use CSR / flat arrays.

1.2 Template, Debugging, Stress Testing

1.2.1 Main template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "] = ", _dbg(__VA_ARGS__)
template <class T> void _dbg(const T& x) { cerr << x << endl; }
template <class T, class... R> void _dbg(const T& x, const R&... r) {
    cerr << x << ", "; _dbg(r...);
}
#else
#define dbg(...)
#endif
const ll INF = 4e18;  // careful: INF + INF overflows. Use 1e18 if you add two.
const int MOD = 1e9 + 7;

void solve() {
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int T = 1;
    // cin >> T;
    while (T--) solve();
}
```

Compile locally: `g++ -std=gnu++17 -O2 -Wall -Wextra -DLOCAL -fsanitize=address,undefined -g a.cpp`. Sanitizers catch out-of-bounds and signed overflow immediately — use them whenever you get RE/WA.

1.2.2 Useful snippets

```cpp
// print __int128
void print128(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar('0' + (int)(x % 10));
}
// floor/ceil division that work for negatives
ll fdiv(ll a, ll b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
ll cdiv(ll a, ll b) { return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); }
// exact integer sqrt (floor)
ll isqrt(ll n) { ll r = sqrtl((ld)n); while (r * r > n) --r; while ((r + 1) * (r + 1) <= n) ++r; return r; }
// random
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(rng); }
// anti-hack hash for unordered_map<ll, V, chash>
```

```cpp
struct chash {
    static ull sm(ull x) { x += 0x9e3779b97f4a7c15; x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb; return x ^ (x >> 31); }
    size_t operator()(ull x) const { static const ull R = rng(); return sm(x + R); }
};
// hash a pair: use key = (ll)a << 32 | (unsigned)b
```

- Read until EOF: `while (cin >> x)`. Read a line: `getline(cin, s)`; after `cin >> n` call `cin.ignore()` (or `getline` once) before the first `getline`.
- Output many lines: use `'\n'`, never `endl` (except interactive).
- Doubles: `cout << fixed << setprecision(10) << x;` — avoid printing `-0.000000` (add `+ 0.0` or check `abs(x) < eps`).
- Sorting with comparator must be a *strict weak ordering* (`<`, never `<=`), otherwise RE/UB.
- `lower_bound(all(v), x)` → first `>= x`; `upper_bound` → first `> x`. Count of `x` in sorted v: `upper - lower`.
- `set.lower_bound(x)` is O(log n); `lower_bound(all(set), x)` is **O(n)**. Same for multiset/map.
- `multiset.erase(x)` removes **all** copies of x; use `ms.erase(ms.find(x))`.
- `priority_queue` is a max-heap; min-heap: `priority_queue<T, vector<T>, greater<T>>`.
- `accumulate(all(v), 0LL)` — the `0LL` matters.
- `(int)v.size() - 1` — `v.size() - 1` is unsigned and wraps when empty.
- `1 << k` is `int`; use `1LL << k` for k ≥ 31. `__builtin_popcountll` for `ll`.
- `memset` only for 0 / -1 / `0x3f` (int ≈ 1.06e9, sum of two still fits) .

1.2.3 Stress testing (do this after the first unexplained WA)

```bash
# gen.py prints a random small test; brute.cpp is the obviously-correct slow solution
g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp
for i in $(seq 1 1000); do
  python3 gen.py $i > in.txt
  ./sol < in.txt > o1.txt; ./brute < in.txt > o2.txt
  if ! cmp -s o1.txt o2.txt; then echo "DIFF on seed $i"; cat in.txt; break; fi
done
```

```python
# gen.py
import random, sys
random.seed(int(sys.argv[1]))
n = random.randint(1, 6)
print(n)
print(*[random.randint(1, 10) for _ in range(n)])
```

Keep tests *tiny* (n ≤ 6, values ≤ 10): the smallest failing case is the easiest to debug by hand.

1.3 When You're Stuck

1.3.1 Constraints → target complexity

Budget ≈ $3 \cdot 10^8$ simple operations per second (less with maps/sets, recursion, cache misses).

n up to	target	typical techniques
8–11	O(n!·n)	all permutations (`next_permutation`), brute force
15–20	$O(2^n \cdot n)$, $O(3^n)$ at 15	bitmask DP, subset enumeration, submask DP
35–45	$O(2^{n/2})$	meet in the middle
50–100	$O(n^4)$, $O(n^3 \log)$	Floyd, flows, small Hungarian, DP with 3–4 dims
300–500	$O(n^3)$	interval DP, Floyd, Gaussian elimination, Hungarian
2000–5000	$O(n^2)$	2D DP, all pairs, $O(n^2)$ with bitset /32 for 10^4–10^5
10^5–$2 \cdot 10^5$	O(n log n), $O(n \log^2 n)$, O(n√n)	sort, BIT/segtree, binary search, Dijkstra, Mo's
10^6	O(n), O(n log n) small const	linear scans, sieve, prefix sums, two pointers, KMP/Z
10^9–10^{18}	O(log n), O(√n), O(cube-root)	math formula, binary search, matrix power, digit DP, factorization

- **Sum of n over tests ≤ X** → per-test complexity can use X. Otherwise multiply by T!
- Two small parameters (e.g. n ≤ 10^5, k ≤ 20) → O(n·k) or $O(n \cdot 2^k)$ DP is intended (ACPC 2022 *City Numbers*: k ≤ 20 colours on a tree).

- A weirdly specific bound is a hint: `n ≤ 15` (bitmask/interval brute), `values ≤ 200` (enumerate the value — ACPC 2018 *Dog Trouble*), `prime factors < 10^6` (trial division works), `p < 10^10, s ≤ 10^5` (s^2 can exceed p).
- Output modulo a prime → counting, combinatorics, DP; answer would be huge otherwise.
- Output a real number with 1e-6 → binary/ternary search on reals, geometry, probability.

1.3.2 Keyword → technique

if the problem says / looks like ...	try	where
minimize the maximum / maximize the minimum	binary search on answer + greedy/BFS check	§2.1
"find the smallest X such that feasible" and feasibility is monotone	binary search on X	§2.1
maximize ratio Σa/Σb	binary search λ: is max Σ(a − λb) ≥ 0 (Dinkelbach)	§2.1
unimodal function of a real variable	ternary search / golden section	§2.1
k-th smallest of something implicit	binary search on value, count ≤ mid	§2.1
subarray sums/xors, count subarrays with property	prefix sums + hashmap / two pointers	§2.2, §2.3
range add many times, query once	difference array	§2.2
sum over all pairs/subarrays of f	contribution technique: count how often each element contributes	§2.2
nearest greater/smaller, "visible", histogram	monotonic stack	§2.3
sliding window min/max	monotonic deque	§2.3
schedule jobs with deadlines/durations	sort by deadline + heap; exchange argument	§2.4
choose k things to maximize something, n large	greedy + heap; check with exchange argument	§2.4
n ≤ 40, subset sums	meet in the middle	§2.5
connectivity under edge deletions, offline	reverse time → DSU additions	§3.1
point update + range query	Fenwick / segment tree	§3.2–3.3
range update + range query	lazy segment tree	§3.3
static range min/gcd	sparse table	§3.4
k-th element in dynamic set, order of key	PBDS ordered set / BIT walk	§3.4
many range queries, offline, no updates, hard to merge	Mo's algorithm	§3.4
count / optimize over sequences, choices at each step	DP; define state = what you need to know about the past	§4.1
subsequence counting (e.g. palindromic subsequences)	interval DP with inclusion–exclusion	§4.1
interleaving two strings, edit distance, LCS	2D DP	§4.1
merge adjacent / parenthesize / split interval	interval DP $O(n^3)$; keep max AND min if signs	§4.1
n ≤ 20 items, choose order/assignment	bitmask DP	§4.2
count numbers in [L, R] with digit property	digit DP	§4.2
tree, choose labels/colours per node with neighbour constraint	tree DP (keep best & second best)	§4.3
answer for every root	rerooting DP	§4.3
dp[i] = min_j (dp[j] + b_j·a_i)	convex hull trick / Li Chao	§4.4
dp[k][i] = min_j dp[k−1][j] + cost(j,i), cost "Monge"	divide & conquer opt / Knuth	§4.4
expected value, probability of winning, optimal stopping	linearity of expectation; DP over states backward	§4.5
turns repeat in a cycle with probabilities	geometric series / solve linear equations	§4.5
two players, optimal play, who wins	win/lose DP, Sprague–Grundy, find pattern by brute force	§4.5
shortest path, unweighted / weights 0–1 / nonneg	BFS / 0-1 BFS / Dijkstra	§5.1
shortest path with extra condition (fuel, horse, key, parity)	Dijkstra on (node, state)	§5.1
distance to nearest of many sources	multi-source BFS/Dijkstra	§5.1
path maximizing the minimum edge/vertex value	binary search + BFS, or max spanning tree, or modified Dijkstra	§5.1
which sums are representable by coins (unbounded)	Dijkstra on residues mod smallest coin	§5.1
minimum ratio spanning tree	binary search λ + MST	§5.2
dependencies / ordering	topological sort, DAG DP	§5.2
each node has one outgoing edge	functional graph: cycles + trees, binary lifting	§5.2
boolean constraints "a or b"	2-SAT	§5.3

if the problem says / looks like ...	try	where
edges whose removal disconnects	bridges / articulation points	§5.3
use every edge exactly once	Euler path (Hierholzer)	§5.3
tree path queries, ancestors, distances	LCA / binary lifting / Euler tour + BIT / HLD	§5.4
count paths in tree with property	centroid decomposition / small-to-large	§5.4
assign workers to jobs one-to-one, min cost	Hungarian / min-cost flow	§6.3
max pairs with compatibility	bipartite matching	§6.2
min #paths covering all vertices of a DAG	n − max matching (split graph)	§6.2
min #paths covering all **edges** of a DAG	min flow with lower bounds = Dilworth on edges	§6.1
cut with costs, "choose set with dependencies, max profit"	min cut / project selection	§6.1
grid with dominoes / chessboard parity	bipartite matching on black/white cells	§6.2
gcd, lcm, divisibility, a·x + b·y = c	ext. gcd, CRT	§7.1–7.2
sum of divisors, factorization of big n	sieve / trial division up to √n / Pollard rho	§7.2
count arrangements, multisets, paths	stars & bars, Catalan, inclusion–exclusion, Burnside	§7.3
linear recurrence with huge n	matrix exponentiation	§7.4
system of linear (mod 2) equations, max xor subset	Gaussian elimination / xor basis	§7.4
count pairs with sum = k for all k, pattern matching counts	FFT / NTT convolution	§7.5
string equality of substrings fast	polynomial hashing	§8.1
pattern occurrences / borders / periods	KMP prefix function, Z-function	§8.1
rotations / cyclic shifts equal	minimal rotation (Booth) as canonical form	§8.2
longest palindrome substring	Manacher	§8.2
many patterns in a text	Aho–Corasick	§8.2
distinct substrings, LCP of suffixes	suffix array + LCP	§8.2
polygon area, orientation, lattice points	shoelace, cross product, Pick	§9.1–9.2
smallest enclosing / outer boundary of points	convex hull	§9.2
interactive: judge answers queries	binary search / information-theoretic planning / gcd tricks	§10.1
evaluate expressions, mini-language	recursive descent parser	§10.2

1.3.3 Stuck-on-an-idea checklist

1. **Re-read the statement.** Underline every constraint. Is something guaranteed (distinct values, connected, sorted, a < b)? That guarantee is usually *used*.
2. **Solve tiny cases by hand** (n = 1, 2, 3) and **write a brute force**; print answers for n = 1..10 and look for a pattern (then search OEIS-style in your head: powers of 2, Catalan, Fibonacci, n^2, C(2n,n)).
3. **What is the answer monotone in?** If "answer ≥ x is feasible" is monotone → binary search. Fixing the answer often turns optimization into a yes/no greedy.
4. **Fix one thing and iterate over it:** fix the maximum element, the left endpoint, the last chosen item, the root, the value of one variable (e.g. enumerate the max time t ≤ 200 then solve the rest).
5. **Sort** the input (by start, end, value, ratio). Many greedy/DP problems become easy after sorting by the right key; find the key with an exchange argument.
6. **Reverse** something: process time backward (deletions → additions), reverse the graph, think about the complement ("at least one" = total − "none").
7. **Change what you iterate over:** over values instead of indices, over divisors instead of multiples (harmonic sum n log n), over the answer instead of the input.
8. **Model as a graph:** states = nodes, moves = edges. Grid + extra state (direction, keys, used power-up) → BFS/Dijkstra on product graph.
9. **Model as a flow/matching** when there are "pairings", "assign each X to one Y", "each used at most once", "minimum number of chains/paths".
10. **Linearity of expectation:** E[sum] = Σ E[indicator]. Never compute the full distribution if only an expectation is asked.
11. **Bound the search:** the answer is small (≤ 60 when doubling, ≤ log n), the number of distinct values is small (≤ √(2n) distinct lengths if Σ = n), only O(√n) distinct `n / i` values.
12. **Look for invariants** (parity, sum mod k, xor) in games and constructive problems.

13. Think about the **special structure:** tree (unique paths, DP), DAG (topo order), bipartite, planar grid, permutation (cycles), convex polygon ($O(n^3)$ DP over vertices).
14. Still stuck after 20–30 min with no progress → switch problems, come back later.

1.3.4 WA / RE / TLE checklist

- **Overflow**: `int * int`, sums of 10^5 values of 10^9, `1 << 40`, `INF + x`. When in doubt, `ll` everywhere.
- **Multi-test state** not reset (globals, adjacency lists, visited arrays, DSU).
- **Off-by-one**: inclusive vs exclusive ranges; 0- vs 1-indexed input; array size `n` vs `n + 1`.
- **Edge cases**: n = 1, empty set, all equal, all negative, answer 0, disconnected graph, self-loops, multi-edges, duplicate points, collinear points.
- **Reading**: multiple values per line, strings with spaces (`getline`), trailing spaces, Windows `\r`.
- **Output**: exact format/capitalisation ("Yes" vs "YES"), precision, one answer per line, `-0.0`.
- **Negative modulo**: `((a % m) + m) % m`. Subtraction in modular arithmetic: `(a - b + MOD) % MOD`.
- **Doubles compared with `==`**; use eps or exact integer arithmetic (cross products in `ll`).
- **Uninitialised** local variables / arrays (globals are zeroed, locals aren't).
- **Recursion depth** (10^6 deep DFS), **stack arrays** too big (`int a[10^7]` inside `main` → use global/vector).
- **TLE**: `endl` flushing, `map` where an array works, passing `vector` by value, O(n) `lower_bound` on `set`, `string +=` in a loop creating copies (`s = s + c` is O(n)), recomputing in loops, too many `long double`.
- **Dijkstra**: skip stale heap entries (`if (d > dist[u]) continue;`), else worst case is quadratic.
- **Comparator** not strict weak ordering → crash in `sort`.
- **Wrong algorithm**: greedy without proof → stress test against brute force.

1.4 ACPC 2016–2024: Every Problem → Technique

Kattis difficulty in brackets. Technique = the intended core idea (my analysis of the statements; `?` = likely but unverified). Use this to calibrate: what does a 4.0 vs 6.5 ACPC problem look like?

year	problem [diff]	core technique
2016	Mirror Images [1.6], Soylent [1.7], Bits [2.2]	implementation
	Wood Cutting [2.5]	sort ascending (shortest processing time first), prefix sums
	Dacey the Dice [3.3]	BFS on (cell, die orientation) — 24 orientations
	Flower Garden [3.6]	prefix of Manhattan distances + sieve for primes
	Palindromic Naming [4.1]	count palindromic subsequences, interval DP $O(n^2)$ mod p (§4.1)
	City Destruction [5.6]	linear DP over buildings with state "was neighbour destroyed before me"
	Modelling Problems [6.8]	parse a mini-language, brute force over unknown 0/1 values
2017	Musical Scales [1.6], Tetration [1.7]	implementation / math ($x = a^{1/a}$)
	Quantum Superposition [2.2]	DAG DP: set of reachable path lengths (bitset), combine two sets
	Anthony and Diablo [2.3]	geometry: max area for perimeter is a circle
	Divide by 100 [3.2]	big-number string manipulation
	Concentration [4.8]	probability / game simulation reasoning ?
	Lane Switching [5.5]	maximize the minimum (bottleneck path) over intervals: Dijkstra/BFS ?
	Maximal Sequences [6.2]	offline by start index; ordered set of next occurrences per value; check first m+1
	Matchings [7.5]	2-D pattern matching counts via FFT convolution, one FFT per colour (§7.5)
2018	Early Winter [2.0], Eating Out [2.5]	implementation / pigeonhole counting
	Arachnophobia [4.0]	multi-source Dijkstra from spiders + binary search D + Dijkstra s→t
	Dog Trouble [5.8]	enumerate max time t (≤ 200), assignment (Hungarian) maximizing Σt_i (§6.3)
	PUBNite [5.8]	geometry + binary/ternary search on time
	Exploding Kittens [6.4]	simulation, skipping irrelevant cards ?
	Homework [6.5]	string interleaving DP O(len1·len2) (bitset if tight)

year	problem [diff]	core technique
	Trimming Convex Polygons [6.9]	DP over convex polygon vertices $O(n^3)$ (fix first kept vertex, triangle fan)
2020	Test Drive [1.9], Laptop Stickers [1.9], Warring Scoring [2.8]	implementation
	Path Crossings [2.9]	hashing (x,y,t) / sort events
	Straights [2.9]	greedy on value counts
	Listen To Your Boss [3.9]	tree ancestor queries: Euler tour tin/tout (§5.4)
	Wormhole Extreme!!! [4.4]	kinematics; minimum initial velocity via v^2 recursion / binary search
	Hired Help [5.5]	binary search #employees + greedy deadline check
	Monochromatic Minesweeper [6.2]	constructive / search with pruning ?
	Password Rotation [6.3]	canonical form = min rotation of s and reverse(s) (Booth), hash set (§8.2)
	Wrapping Trees [7.3]	fractal → convex hull of the self-similar set; geometry
	Best Investing [7.8]	math / greedy over deposit–withdraw schedule ?
2021	Finding An A [1.2], GCVWR [1.4], Square Peg [1.5], Simone [1.8], Colour Wars [2.3]	implementation
	Snakes and Masters [2.9]	Fibonacci DP mod 10^6
	Bitmask [3.1]	string/bit matching with DP ?
	Menu Updates [3.3]	maintain smallest available number: set/heap
	Associative Exponents [3.4]	compare $a^{b\hat{}c}$ vs $(a^b)^c \Leftrightarrow b^c$ vs b·c, careful with overflow
	Ghostbusters 3 [3.8]	non-crossing assignment: sort + greedy/DP ?
	Wonky Pizza [4.2]	polar area $\int r^2/2\, d\theta$ closed form + binary search the angle
	Beep Code [4.6]	bit decomposition of a sum of square waves; reason level by level
	Balls and Bins [5.1]	optimal-stopping expected-value DP over (#bins with 1, #bins with 2) (§4.5)
	Swapping Places [8.1]	constructive: permutation routing on spanning tree, process leaves first
	Compression [8.2]	DP over block boundaries with careful cost precomputation
2022	Undead or Alive [1.3], Ordinals [1.8], Count Doubles [1.8]	implementation
	Contingency Planning [2.1]	$\Sigma_k\ n!/(n-k)!$ with cap
	Simple Factoring [2.4]	discriminant $b^2 - 4ac$ is a perfect square
	Maximum Fix [3.4]	each element votes for the one shift that fixes it; count array
	Supply Routes [3.5]	offline: reverse edge deletions → DSU unions (§3.1)
	Fight to Survive [3.8]	greedy / simulation ?
	City Numbers [4.4]	tree DP dp[v][colour], keep best & second best child value (§4.3)
	Long Walk [5.5]	implicit graph $i \leftrightarrow 2i \bmod 2^n$: bit manipulation, LCA-like on binary strings
	Braaaains [5.8]	Frobenius/coin reachability: Dijkstra on residues mod smallest size (§5.1)
	Tricky Factoring [6.7]	$b = u + ac/u$ over divisors u of ac ⇒ answer $2\cdot\lceil\tau(ac)/2\rceil$; factor via sieve
	Maximal Parentheticals [8.0]	interval DP tracking max and min; parentheses can create unary minus + implicit multiplication
	Escape Routes [8.3]	min #paths covering all DAG edges = min flow with lower bounds (§6.1)
2023	Buildin' Fences [1.5]	implementation
	Flying Straight [3.0]	trigonometry (arc vs tangent)
	Buildin' Roads [3.3]	minimum ratio spanning tree: binary search λ + MST (§2.1, §5.2)
	OBD Codes [3.9]	pattern matching on event streams ?
	Counting Cacti [4.0]	grid parsing + flood fill components
	Outlaw Numbers [4.0]	aliquot sequence: sum of proper divisors via factorization, ≤ 32 steps

year	problem [diff]	core technique
	Wrangler Rating [4.5]	careful simulation of rules
	Circuit Game [5.2]	graph connectivity under switch removal + budget ?
	Dice Tables [5.2]	counting multisets (stars and bars) over overlapping face ranges, mod 10^9+9
	Desert Strike [5.4]	lattice counting of a rasterized rotated square (math)
	High Noon Language Showdown [5.5]	brute force over n with exact integer ceil(log2)
	Temporal Turmoil [6.7]	clock-hand alignment math with exact fractions
	Horse Snatchers [6.8]	binary search horse speed + Dijkstra on (node, has horse)
	Gunslingers [7.8]	probability DP over survivor sets with cyclic turns (geometric series)
2024	Snowfall [1.6], Melting Snow [1.7], Handheld Fan [2.2]	implementation
	Wrapping Snowballs [3.5]	?
	Alpine Agility [3.8]	grid DP (moves S/E/SE)
	Slippery Floor [3.8]	BFS where a move slides until hitting an obstacle
	Topographic Isolation [4.0]	monotonic stack: nearest ≥ on each side (§2.3)
	A Little to the Right [4.2]	count distinct sorted orders among properties with all-distinct values
	Hockey Fans [4.4]	binary search on D + greedy left-to-right placement
	Uniform Variation [4.7]	brute force: groups of all-different are ≤ 4; enumerate small subsets
	Snow Way Out [6.7]	interactive, 6 comparisons on 10×10 grid: offline minimax search for a strategy
	Four Questions [7.1]	interactive: $s^2 \bmod p \Rightarrow p$ divides $s^2 - r$; gcd of such values (§10.1)
	Speedy Slopes [7.2]	shortest path with speed state on grid ?

Patterns to take away

- Binary search on the answer appears almost every year (Arachnophobia, Hired Help, Hockey Fans, Horse Snatchers, Buildin' Roads).
- Dijkstra variants: multi-source, state-augmented, residue graph (Braaaains).
- Probability/expected value DP appears most years at 5–8 difficulty.
- One flow/matching problem every year or two (Dog Trouble, Escape Routes).
- Interactive problems appeared in 2024 (two of them) — read §10.1 before the contest.
- Number theory with "all prime factors < 10^6" or big-number twists.

1.5 Worked Patterns from Past ACPC Problems

Short solution sketches of past ACPC problems, grouped by the *recognition trigger*. The point is to train "this smells like X". Read the trigger, guess the technique, then read the sketch.

1.5.1 Binary search on the answer

Hockey Fans (2024, 4.4). *Largest D such that M disjoint windows of length S all have values ≥ D.* Trigger: "largest value such that it is possible". Feasibility is monotone in D. Check(D): scan left to right with a counter of consecutive values ≥ D; whenever it reaches S, take a window and reset the counter. Count ≥ M ⇒ feasible. Binary search D over the sorted distinct values. O(N log N).

Hired Help (2020, 5.5). *Max employees, each must repair exactly K shoes (1 minute each) before the shoes' deadlines.* Trigger: "maximum number such that …". Check(k): take the k·K shoes with the latest deadlines (exchange argument: a later deadline is never worse); sort them ascending; shoe number i (0-indexed) is repaired in minute $\lfloor i/k \rfloor + 1$ at the earliest, so feasible iff `d_i ≥ ⌊i/k⌋ + 1` for all i. Binary search k.

Arachnophobia (2018, 4.0). *Maximise the minimum spider distance along a path that must take ≤ T seconds.* Trigger: maximin + another constraint. Multi-source Dijkstra from all spiders gives `sd[v]`. Binary search D: Dijkstra from s using only vertices with `sd ≥ D`; feasible iff dist(t) ≤ T. O((n + m) log n · log V).

Horse Snatchers (2023, 6.8). *Slowest horse that still reaches town n by time T; entering a snatcher town loses the horse, then you walk.* Faster horse never hurts ⇒ binary search the speed. Check: Dijkstra over states (town, still riding) where riding edges cost d/speed and walking edges cost d/w; entering a snatcher town switches the state to walking.

Buildin' Roads (2023, 3.3). *Spanning tree minimising Σpitch / Σgravel.* Ratio objective ⇒ binary search λ, MST with weights `pitch − λ·gravel`, feasible iff total ≤ 0.

1.5.2 Shortest paths in disguise

Braaaains (2022, 5.8). *Container sizes (unbounded); which order sizes are exactly packable?* Classic coin reachability for huge targets: Dijkstra on residues modulo the smallest size a (§5.1); X is packable iff `d[X mod a] ≤ X`. O(a·k log a).

Dacey the Dice (2016) / Slippery Floor (2024). BFS where a node is (cell, extra state): the die orientation (24 states) or "where you stop after sliding".

1.5.3 DP

Balls and Bins (2021, 5.1). *Optimal stopping; throwing into a bin with 3 balls loses everything.* Because every successful throw earns p_i and failure loses everything, the money earned so far is just the prefix sum of p up to t = #balls thrown. State = (#bins with 1, #bins with 2, #bins with 3) determines t. `V(s) = max(P_t, Σ_{outcomes} prob · V(next))` with the "full bin" outcome contributing 0. O(N³) states, O(1) transitions, compute in decreasing t.

City Numbers (2022, 4.4). *Colour a tree with 1..k, adjacent different, minimise Σ colours.* Tree DP `dp[v][c]`, merging a child needs `min_{c' ≠ c} dp[child][c']` = best or second best (§4.3). O(nk).

Palindromic Naming (2016, 4.1). Count palindromic subsequences: interval DP with inclusion–exclusion (§4.1).

Homework (2018, 6.5). *Is s an interleaving of s1 and s2?* 2D boolean DP over (i, j) — $25 \cdot 10^6$ states, tight but fine with a `vector<char>` and a tight loop.

Trimming Convex Polygons (2018, 6.9). *Keep a subset of convex polygon vertices: gain 2·area of kept polygon + values of discarded vertices.* Kept vertices form a subsequence in cyclic order; area = fan of triangles from the first kept vertex. dp over (first kept, last kept) adding one triangle per step. O(n³) for n = 200.

Maximal Parentheticals (2022, 8.0). Parenthesise `d op d op …` (N ≤ 15) to maximise the value. Twist: writing `1 (- 2)` turns the binary minus into a unary minus and creates an *implicit multiplication*, so sub-expressions can be multiplied. Interval DP over token ranges storing the max and min achievable value (multiplication and subtraction of negatives flip extremes), with separate states for "interval starts with a unary sign". Read such statements for every way the given rules can be combined — the hard part is the case analysis, not the DP.

1.5.4 Flows and matching

Dog Trouble (2018, 5.8). *Assign N dogs to distinct bowls minimising Σ(t − t_i) where t is the max.* Mixed max/sum objective ⇒ enumerate t (≤ 200 values), Hungarian maximising Σt_ij over allowed pairs (§6.3). 200 × 50³.

Escape Routes (2022, 8.3). *Minimum number of 0 → V−1 paths in a DAG covering every edge.* Flow with lower bound 1 on every edge and infinite capacity, minimise the flow value ⇒ min flow with lower bounds (§6.1). (Equivalently: Dilworth on edges.)

1.5.5 Number theory / interactive

Four Questions (2024, 7.1). *Find a prime $p < 10^{10}$ from 4 replies of $s^2 \bmod p$, $s \le 10^5$.* With s near 10^5, $s^2 \ge p$ (usually), so the reply r is informative: $p \mid s^2 - r$. g = gcd of the values $s_i^2 - r_i$ is a multiple of p; candidates are prime factors q of g with q > max(r_i) and `s_i² mod q == r_i` for all i (factor g by trial division up to 10^5). Subtlety: for **small p** several primes can explain the same replies ($s^2 \bmod p$ is tiny) — use the remaining queries adaptively, picking an s whose predicted replies differ between the surviving candidates. Lesson: count candidates after each query and spend queries to split them.

Tricky Factoring (2022, 6.7). *Count b with $ax^2 + bx + c$ factorable over integers (a, c ≤ 10^{16}, prime factors < 10^6).* (dx + e)(fx + g): a = df, c = eg, b = dg + ef. Enumerating divisor pairs is too slow (τ up to ~$4 \cdot 10^4$ each). Key observation: u = dg and v = ef satisfy u·v = ac, and *every* divisor u of ac splits as d·g with d | a, g | c. So |b| ranges over {u + ac/u : u | ac}, which is injective for u ≤ √(ac). Answer = $2 \cdot \lceil \tau(ac)/2 \rceil$ (sign ±), τ from the merged factorisations of a and c (matches all samples: 2, 18, 128, 486). Lesson: **look for an invariant product/sum that collapses the double enumeration.**

Outlaw Numbers (2023, 4.0). Iterate n → σ(n) − n at most 32 times; each step needs factorisation by trial division / Pollard.

1.5.6 Strings

Password Rotation (2020, 6.3). *Do any two passwords match up to rotation or reversed rotation?* Canonical form = min(minRotation(s), minRotation(reverse s)) (§8.2); insert into a hash set; any repeat ⇒ "Yes". Total length 10^6 ⇒ O(total).

Matchings (2017, 7.5). *For every placement of a small image inside a big one, count equal pixels; report the best placements.* For each colour c: correlation of the big image's indicator with the small image's indicator — FFT after flattening with the big image's width (§7.5).

1.5.7 Offline / data structures

Supply Routes (2022, 3.5). Edges get destroyed; answer connectivity queries. Reverse time: start from the final graph, add edges back with DSU, answer queries in reverse.

Topographic Isolation (2024, 4.0). Nearest index with height $\geq$ h on each side: monotonic stack from both directions; peak's isolation = min distance (-1 if none).

Maximal Sequences (2017, 6.2). *Longest run starting at i using only values in B ($\Sigma|B| \leq 10^5$).* Offline by i descending; keep for each value its next occurrence $\geq$ i in an ordered set; the answer is the first next-occurrence of a value not in B — check the earliest $|B| + 1$ entries of the set.

2. Core Techniques

2.1 Binary Search, Ternary Search, Parametric Search

2.1.1 Integer binary search on the answer

Use when `ok(x)` is monotone: `false false ... false true true ...`. Write the invariant, not a formula you half-remember: **`lo` is always bad, `hi` is always good.**

```
// smallest x in (lo, hi] with ok(x) true; requires ok(hi) true, ok(lo) false (lo may be a sentinel)
template <class F> ll firstTrue(ll lo, ll hi, F ok) {
    while (hi - lo > 1) {
        ll mid = lo + (hi - lo) / 2;
        (ok(mid) ? hi : lo) = mid;
    }
    return hi;
}
// largest good x for "true ... true false ... false": lastTrue = firstTrue of !ok, minus 1
```

- Bounds: pick `lo` = something surely infeasible (often `-1` or `0`), `hi` = surely feasible (`1e18`, `2e9`). Check that `ok` never overflows at `hi` (e.g. `mid * cnt` with `mid = 1e18` → use `__int128` or early exit).
- **Maximize the minimum** (e.g. Hockey Fans, ACPC 2024: largest D such that M non-overlapping windows of length S all have values ≥ D): `ok(D)` = greedy scan left to right taking a window as soon as the last S values are all ≥ D. Monotone because a smaller D is easier.
- **Minimize the maximum** (split array into k parts minimizing max sum; minimize max edge on a path): `ok(X)` = greedy "how many parts do I need with cap X" ≤ k.
- **k-th smallest of an implicit set** (k-th smallest pairwise sum/difference, k-th fraction): binary search value v, count elements ≤ v (two pointers on sorted arrays is usually O(n)).
- **Binary search + graph** (Arachnophobia, Horse Snatchers): fix the parameter, then run BFS/Dijkstra on the allowed subgraph and check if the target is reachable within the time limit. Complexity × log(range).
- Searching over sorted *candidate* values (e.g. distinct edge weights) instead of the whole range keeps the answer exact and iterations ≤ log n.

2.1.2 Real-valued binary search

```
// sketch: ok(mid) = your feasibility test
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; ++it) {   // 100 halvings: 2^-100 relative; never loop on hi-lo>eps
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid; else lo = mid;
}
```

Fixed iteration count avoids infinite loops caused by precision. 60 iterations are enough for 1e-9 relative on [0, 1e9]; cost is usually negligible.

2.1.3 Ternary search (unimodal functions)

```
// maximum of strictly unimodal f on [lo, hi] (reals)
template <class F> double ternaryMax(double lo, double hi, F f) {
    for (int it = 0; it < 200; ++it) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; else hi = m2;
    }
    return (lo + hi) / 2;
}
// integers: shrink while hi - lo > 2, then check lo..hi directly
template <class F> ll ternaryMaxInt(ll lo, ll hi, F f) {
    while (hi - lo > 2) {
        ll m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1 + 1; else hi = m2;
    }
    ll best = lo;
    for (ll x = lo; x <= hi; ++x) if (f(x) > f(best)) best = x;
    return best;
}
```

- Integer version fails with plateaus (f(m1) == f(m2) not at the max). If f can be flat, binary search on the sign of `f(x+1) - f(x)` instead (valid when f is concave: differences are non-increasing).

- Convex functions: sums of convex functions are convex (e.g. $\Sigma|x - a_i|$, Σ dist(P(t), Q_i) for linear motion). Distance between two points moving linearly over time is convex in t → ternary search on time.
- Nested ternary search works for functions convex in 2 variables (e.g. Fermat point, smallest enclosing circle centre) but is slow: 100×100 evaluations.

2.1.4 Parametric search / fractional programming

Maximize $\Sigma a_i / \Sigma b_i$ over feasible sets S ($b > 0$). The answer $\geq \lambda$ iff $\max_S \Sigma(a_i - \lambda b_i) \geq 0$. So binary search λ and solve the *unweighted* problem with weights $a_i - \lambda b_i$:

- Choose exactly k items: sort by $a_i - \lambda b_i$, take top k.
- Spanning tree minimizing Σcost/Σgain (ACPC 2023 *Buildin' Roads*): min ratio ⇔ binary search λ, MST with weights cost $- \lambda \cdot$gain, check total ≤ 0.
- Cycle with maximum mean (Σw/len): binary search λ, look for positive cycle with weights $w - \lambda$ (Bellman–Ford).

Dinkelbach converges faster: start $\lambda = 0$; repeat: $S = \text{argmax } \Sigma(a - \lambda b)$; $\lambda = \Sigma_S a / \Sigma_S b$; until no improvement (usually < 10 iterations).

2.1.5 Binary lifting search ("walk down powers of two")

When you can test "is the answer $\geq$ pos + 2^k" incrementally (Fenwick tree prefix walk, binary lifting on ancestors, sparse table), descend over k from high to low instead of a separate binary search — saves a log factor. See BIT `lower_bound` (§3.2) and LCA (§5.4).

2.1.6 Parallel binary search (offline, many queries)

Q queries each need "first time t when condition holds" where the structure evolves over time (e.g. first time u,v connected after edge additions; first time a sum reaches a threshold). Do log T rounds: in each round sort queries by their current `mid`, replay the events once from the start, and answer all `ok(mid)` checks along the way. Cost: O((T + Q) log T × cost of one event).

2.2 Prefix Sums, Difference Arrays, Sweep Line, Contribution

2.2.1 Prefix sums (1D/2D) and difference arrays

```
// 1D: p[i] = a[0] + ... + a[i-1]; sum of a[l..r] = p[r+1] - p[l]
// 2D: P[i][j] = sum of a[0..i-1][0..j-1]
struct Pref2D {
    vector<vector<ll>> P;
    Pref2D(const vector<vector<ll>>& a) {
        int n = a.size(), m = a[0].size();
        P.assign(n + 1, vector<ll>(m + 1, 0));
        rep(i, 0, n) rep(j, 0, m) P[i + 1][j + 1] = a[i][j] + P[i][j + 1] + P[i + 1][j] - P[i][j];
    }
    ll query(int r1, int c1, int r2, int c2) {  // inclusive rectangle
        return P[r2 + 1][c2 + 1] - P[r1][c2 + 1] - P[r2 + 1][c1] + P[r1][c1];
    }
};
```

- **Difference array:** add v to a[l..r] ⇒ `d[l] += v; d[r+1] -= v;` then prefix-sum d once. 2D: `d[r1][c1] += v; d[r1][c2+1] -= v; d[r2+1][c1] -= v; d[r2+1][c2+1] += v`.
- Adding an arithmetic progression to ranges: apply difference twice (second-order difference).
- **Count subarrays with sum = k:** iterate prefix p, `ans += cnt[p - k]; cnt[p]++` (hashmap; start with `cnt[0] = 1`). Same for xor (`p ^ k`), and for "sum divisible by m" (`cnt[p mod m]`).
- **Longest subarray with sum ≤ k (non-negative values)**: two pointers. With negatives: prefix + monotonic stack / sorted prefix minima + binary search.
- **Prefix of a function over a cycle** (rotations, circular arrays): double the array, take windows of length n.

2.2.2 Coordinate compression

```
// sketch: vals = every coordinate that matters
vector<ll> xs = vals;                       // all coordinates that matter
sort(all(xs)); xs.erase(unique(all(xs)), xs.end());
auto id = [&](ll x) { return int(lower_bound(all(xs), x) - xs.begin()); };
```

For intervals on a compressed axis, remember that compressed index i represents the *point* xs[i]; the gap between xs[i] and xs[i+1] has length `xs[i+1] - xs[i]` (needed for union-of-intervals length / area).

2.2.3 Sweep line

Turn geometry/intervals into **events sorted by coordinate**, process with a data structure.

- **Max overlapping intervals:** events (l, +1), (r, −1); sort; for closed intervals put +1 before −1 at equal coordinate.
- **Union length of intervals:** sort by l, merge greedily.
- **Area of union of rectangles:** sweep x; segment tree over compressed y storing "count covered > 0 length" (classic: node stores `cnt` and `covered len`, no push-down needed).
- **Pairs of intersecting segments / closest pair:** sweep with `set` ordered by current y (rarely needed at ACPC).
- **Offline queries:** sort queries together with updates by coordinate/time; answer each query when the sweep reaches it (e.g. "count points with x ≤ X and y ≤ Y" → sort by x, BIT on y).
- **Events at same coordinate:** decide the order explicitly (insertions before queries? removals first?) — this is the #1 sweep bug.

```
// sketch: max number of simultaneously active closed intervals [l, r]
vector<pair<ll,int>> ev;
for (auto [l, r] : iv) { ev.push_back({l, -1}); ev.push_back({r, +1}); } // -1 = open sorts first
sort(all(ev));
int cur = 0, best = 0;
for (auto [x, t] : ev) { cur -= t; best = max(best, cur); }
```

2.2.4 Contribution technique (sum over all subsets/subarrays/pairs)

Instead of enumerating structures, ask: **for each element, in how many structures does it contribute, and how much?**

- Σ over all subarrays of sum = Σ a_i · (i+1)·(n−i) (0-indexed).
- Σ over all subarrays of max: a_i is the max of subarrays with l ∈ (L_i, i], r ∈ [i, R_i) where L_i = previous index with value ≥ a_i (strict on one side, non-strict on the other to avoid double counting ties), R_i = next index with value > a_i. Contribution a_i·(i − L_i)·(R_i − i). Use monotonic stack (§2.3).
- Σ over pairs |a_i − a_j|: sort; a_i contributes `a_i·i − (sum of previous)`.
- Σ over all subsets of max: sort; a_i is the max in 2^i subsets.
- Σ of path lengths over all pairs in a tree: edge (parent, v) is used by sz[v]·(n − sz[v]) pairs.
- Expected value versions: same with probabilities (linearity of expectation, §4.5).

2.2.5 Offline tricks summary

- Answer queries in a convenient order (sorted by right endpoint, by threshold, by time) and restore original order at the end — store the query index.
- Reverse time for deletions (DSU can't delete, but can add).
- Divide & conquer over time / CDQ: split operations, handle "left half updates → right half queries" with a sort + BIT.

2.3 Two Pointers, Monotonic Stack, Monotonic Deque

2.3.1 Two pointers / sliding window

Valid when, for each left end `l`, the best/maximal right end `r(l)` is non-decreasing in `l` (window property is monotone: shrinking a valid window keeps it valid).

```
// longest window with at most k distinct values
int longestKDistinct(const vector<int>& a, int k) {
    unordered_map<int, int> cnt;
    int best = 0;
    for (int l = 0, r = 0; r < sz(a); ++r) {
        cnt[a[r]]++;
        while (sz(cnt) > k) { if (--cnt[a[l]] == 0) cnt.erase(a[l]); ++l; }
        best = max(best, r - l + 1);
    }
    return best;
}
```

- Count windows satisfying a monotone property: for each r, all l' ≥ l(r) work → `ans += r - l + 1`.
- "Exactly k" = atMost(k) − atMost(k−1).
- Two sorted arrays: pairs with a_i + b_j ≤ X — one pointer forward on a, one backward on b.
- **Does not work with negative numbers for sums** (window sum isn't monotone) — use prefix sums + sorted structure instead.

2.3.2 Monotonic stack: nearest greater / smaller

```
// prv[i] = largest j < i with a[j] >= a[i] (or -1); nxt[i] = smallest j > i with a[j] > a[i] (or n)
// (>= on one side, > on the other: ties are counted exactly once in "subarrays where a[i] is max")
void nearestGreater(const vector<ll>& a, vector<int>& prv, vector<int>& nxt) {
    int n = sz(a); prv.assign(n, -1); nxt.assign(n, n);
    vector<int> st;
    rep(i, 0, n) {
        while (!st.empty() && a[st.back()] < a[i]) { nxt[st.back()] = i; st.pop_back(); }
        prv[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
}
```

Uses: largest rectangle in histogram (nearest smaller on both sides, area $h_i \cdot (R - L - 1)$); sum of subarray maxima/minima (contribution, §2.2); ACPC 2024 *Topographic Isolation* (for each peak, nearest index on the left and right with height ≥ peak: answer = min distance); visibility ("how many people can i see"); next greater element in a circular array (iterate 2n).

Maximal all-ones rectangle in a 0/1 grid: for each row keep `h[j]` = consecutive ones above; run histogram per row. O(nm).

2.3.3 Monotonic deque: sliding window min/max

```
// mins[i] = min(a[i-k+1..i]) for i >= k-1
vector<ll> slidingMin(const vector<ll>& a, int k) {
    deque<int> dq; vector<ll> res;
    rep(i, 0, sz(a)) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

- **DP optimisation:** `dp[i] = min_{i-k ≤ j < i} dp[j] + c_i` → keep deque of j by dp value. Also `dp[i] = max_{j} (dp[j] - f(j)) + g(i)` with a sliding range of j.
- 2D sliding window min: apply 1D on rows, then on columns of the result.
- Shortest subarray with sum ≥ K with negative numbers: deque over prefix sums (increasing), pop front while `p[i] - p[front] ≥ K`.

2.3.4 Other amortised patterns

- **Stack simulation**: bracket matching, "remove adjacent equal pairs", evaluating expressions (shunting-yard).
- **Next pointer / "jump" with path compression**: "find next unvisited index ≥ i" — DSU where `nxt[i] = i+1` after visiting i; O(α) per query. Great for "paint ranges, each cell painted once" problems.
- **Small-to-large merging**: merge smaller set into larger → O(n log n) total insertions (§3.4).

2.4 Greedy and Exchange Arguments

A greedy is only as good as its proof. Standard proof: take an optimal solution that differs from greedy at the first position; **swap** to agree with greedy without making it worse. If you can't prove it, stress test against a brute force (§1.2) before submitting.

2.4.1 Finding the sort key by exchange

Two adjacent items i, j in an order; compare cost(…i j…) vs cost(…j i…). The difference only involves i and j ⇒ sort by that comparison.

problem	key
minimise Σ completion times (1 machine)	ascending processing time (SPT) — ACPC 2016 *Wood Cutting*
minimise $\Sigma w_i \cdot C_i$	ascending t_i / w_i, i.e. compare `t_i*w_j < t_j*w_i` (integers!)
minimise max lateness	ascending deadline (EDF)
concatenate strings to get smallest string	`a + b < b + a`
tasks with requirement r_i and cost c_i (need ≥ r to start, lose c)	descending $r_i - c_i$
two machines, each job on A then B (Johnson)	jobs with a<b first by a ascending, then others by b descending

Always compare with cross-multiplication in integers instead of dividing.

2.4.2 Interval greedies

- **Max number of disjoint intervals:** sort by right end, take if start > last end.
- **Min points to stab all intervals:** sort by right end; if interval not stabbed, put a point at its right end.
- **Min intervals to cover [0, L]:** sort by left; repeatedly among intervals with left ≤ covered, take the one with max right.
- **Partition intervals into min number of "rooms":** sort by start, min-heap of end times; answer = max overlap.
- **Assign intervals to k machines maximizing count:** sort by end, multiset of machine free times; assign to the machine with the latest free time ≤ start.

2.4.3 Deadline scheduling

Unit jobs with deadlines, maximise #jobs (or profit): sort by profit desc, place each job in the latest free slot ≤ its deadline (DSU "next free slot" trick, §3.1). Or: sort by deadline, push profits into a min-heap, pop the smallest whenever heap size > deadline.

Jobs with durations and deadlines, maximise count (Moore–Hodgson):

```
// jobs (deadline, duration); maximum number completed on time
int maxOnTime(vector<pair<ll,ll>> jobs) {
    sort(all(jobs));
    priority_queue<ll> pq; ll t = 0;
    for (auto [d, p] : jobs) {
        pq.push(p); t += p;
        if (t > d) { t -= pq.top(); pq.pop(); }   // drop the longest job so far
    }
    return sz(pq);
}
```

Feasibility check with deadlines (ACPC 2020 *Hired Help*: can we form k groups of K unit jobs?): a set of unit jobs is schedulable on c machines iff for every t, #jobs with deadline ≤ t is ≤ c·t. Combine with binary search on k and pick the jobs with the *latest* deadlines.

2.4.4 Heap greedies

- **Merge costs (Huffman):** always merge two smallest; total cost Σ.
- **Regret / "undo" greedy:** buy low sell high with one transaction per day: min-heap; for each price p: if `top < p` then `profit += p - top`, pop, and push p (this copy lets the sale be "undone" later); then always push p. Same idea: choose k non-adjacent elements with max sum — take max, replace it with (left + right − it) node, linked list.
- **k largest sums / k best combinations:** heap of states with "next" transitions (e.g. pairs (i, j) of two sorted arrays, push (i+1, j) and (i, j+1) with a visited set).

2.4.5 Other common greedy facts

- To maximise Σ a_i·b_{π(i)}: sort both same direction (rearrangement inequality); to minimise: opposite directions.
- Median minimises Σ|x − a_i|; mean minimises Σ(x − a_i)².
- Making an array non-decreasing with ±1 ops (min cost) → slope trick / median heap.
- Lexicographically smallest answer: build character by character, at each step try the smallest choice and check feasibility of completing (feasibility check is often a simple counting argument).
- Coin change greedy only works for canonical systems (1,5,10,25…); for arbitrary coins use DP.
- Pair the largest with the smallest (boats/teams with capacity): sort + two pointers.

2.5 Complete Search, Meet in the Middle, Square Root, Randomization

2.5.1 Brute force done right

- Enumerate subsets: `for (int m = 0; m < (1 << n); ++m)`. Permutations: sort, then `do { } while (next_permutation(all(p)));` (n ≤ 10: 3.6M perms).
- **Backtracking with pruning:** order choices (most constrained first), prune with an optimistic bound (current + best possible remaining ≤ best found ⇒ stop), memoize repeated states.
- **Iterate over the small dimension:** if one of the parameters is ≤ 20–200 (number of colours, max value, number of properties), loop over it explicitly.
- **Search the answer space instead of the input space** when the answer is small (e.g. answer ≤ 60 because values double).
- Before brute forcing, count states × transitions and compare with $3 \cdot 10^8$.

2.5.2 Meet in the middle

Split n ≤ 40 items into halves, enumerate $2^{n/2}$ each, combine with sort + two pointers/binary search/hash map.

```
// number of subsets with sum <= S (n <= 40)
ll countSubsets(const vector<ll>& a, ll S) {
    int n = sz(a), h = n / 2;
    auto gen = [&](int l, int r) {
        vector<ll> s = {0};
        for (int i = l; i < r; ++i) {
            int k = sz(s);
            rep(j, 0, k) s.push_back(s[j] + a[i]);
        }
        sort(all(s)); return s;
    };
    vector<ll> L = gen(0, h), R = gen(h, n);
    ll cnt = 0; int j = sz(R) - 1;
    for (ll x : L) {                        // L ascending -> pointer in R only moves down
        while (j >= 0 && x + R[j] > S) --j;
        cnt += j + 1;
    }
    return cnt;
}
```

Also: 4-sum = pair sums of two halves; bidirectional BFS on huge implicit graphs (puzzle states) — expand from both ends, stop when frontiers meet.

2.5.3 Square root decomposition

- **Blocks of size B ≈ √n:** each block stores an aggregate (+ lazy tag). Update/query touch O(n/B) full blocks + O(B) loose elements. Handy when the operation is weird (e.g. "add x to range, count elements < y in range": keep each block sorted → O(√n log n)).
- **Heavy/light by frequency:** values occurring > √n times are "heavy" (at most √n of them) → precompute for heavy, brute force for light.
- **Σ of sizes = n ⇒ at most O(√n) distinct sizes.** (Knapsack with Σ w = n has O(√n) distinct weights → bounded knapsack per distinct weight.)
- **Divisor/quotient blocks:** `n / i` takes O(√n) distinct values; iterate blocks:

```
// sum_{i=1}^{n} f(n / i) grouped: for each block [l, r], n / i == q
ll sumQuotients(ll n) {
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll q = n / l; r = n / q;
        res += (r - l + 1) * q;     // replace q by f(q), (r-l+1) by the weight of [l, r]
    }
    return res;
}
```

- **Mo's algorithm** for offline range queries: §3.4.
- **Rebuild periodically:** buffer updates; every √q updates rebuild the static structure; queries check the static structure + the buffer.

2.5.4 Randomization

- **Random shuffle** before inserting into structures that degrade on sorted input (treap keys, BST, hull insertion).
- **Random sampling** to find a majority/frequent element: an element occupying ≥ 1/k of the array is hit with probability ≥ 1/k per sample; 30·k samples → failure ≈ e^{-30}.
- **Hashing with random base/mod** (§8.1); **xor-hashing of sets/multisets**: assign each value a random 64-bit number; set equality ⇔ xor (or sum) of hashes equal. Subarray is a permutation of 1..k ⇔ sum of hashes of subarray = prefix-hash of {1..k}.
- **Random restarts / simulated annealing** for optimization problems with no clear structure (last resort; tune by time limit with `clock()`).
- `mt19937_64 rng(seed)`; never `rand()` (max 32767 on some platforms).

2.5.5 Iterative deepening / search with memo

- Game/puzzle with small depth: IDDFS or BFS over states stored in hash map (encode state into a 64-bit integer).
- **Memoized recursion over states** = DP. If the state space is unclear, write the recursive brute force, then add a `map<State, Answer>` and see if it's fast enough.

2.6 Bit Manipulation and Bitset

2.6.1 Tricks

expression	meaning
`x & -x`	lowest set bit
`x & (x - 1)`	clear lowest set bit; `== 0` ⇔ x is a power of two (x > 0)
`__builtin_popcountll(x)`	number of set bits
`__builtin_ctzll(x)` / `__builtin_clzll(x)`	trailing / leading zeros (undefined for 0!)
`63 - __builtin_clzll(x)`	floor(log2 x) (`__lg(x)` also works)
`x >> k & 1`	k-th bit
`x ^ (1LL << k)`	toggle k-th bit
`(1LL << n) - 1`	mask of n ones
`a + b == (a ^ b) + 2 * (a & b)`	sum decomposes into xor + carries
`a ^ b ^ b == a`	xor cancels; prefix xor gives range xor

```cpp
// iterate all submasks of mask (descending), including 0: total over all masks is 3^n
void subs(int mask) {
    for (int s = mask; ; s = (s - 1) & mask) {
        // use s
        if (s == 0) break;
    }
}
// iterate all supersets of mask within n bits
void supers(int mask, int n) {
    for (int s = mask; s < (1 << n); s = (s + 1) | mask) { /* use s */ }
}
// next mask with the same popcount (Gosper's hack)
ll nextSamePop(ll x) { ll c = x & -x, r = x + c; return (((r ^ x) >> 2) / c) | r; }
```

2.6.2 XOR facts

- XOR of 0..n: `n % 4 == 0 ? n : n % 4 == 1 ? 1 : n % 4 == 2 ? n + 1 : 0`.
- Maximum xor of two numbers in a set: binary trie, greedily go to the opposite bit (O(30) per query).
- Maximum xor of any subset / k-th xor / "is x representable": **linear basis** (§7.4).
- Per-bit independence: many problems on sums of xors/ands/ors split into 30 independent problems, one per bit (count pairs with bit k set in a_i ^ a_j = cnt0·cnt1).
- `a | b` and `a & b`: `a + b = (a | b) + (a & b)`.

2.6.3 Bitset

`bitset<N>` operations are N/64 word ops — 64× speedup for boolean DP.

```cpp
// subset-sum reachability up to MAXS: O(n * MAXS / 64)
const int MAXS = 100001;
bitset<MAXS> reachable(const vector<int>& w) {
    bitset<MAXS> b; b[0] = 1;
    for (int x : w) b |= b << x;
    return b;
}
```

- Transitive closure / reachability in a DAG: `reach[v] |= reach[u]` in reverse topo order: O(n·m/64).
- Counting triangles: for each edge (u,v): `(adj[u] & adj[v]).count()`.
- Sets of reachable path lengths in a DAG (ACPC 2017 *Quantum Superposition*): `len[v] |= len[u] << 1`.
- O(n^2) DP where the transition is an OR/AND over a row (LCS-like boolean, interleaving check like ACPC 2018 *Homework*) can often be done row-by-row with shifts.
- `_Find_first()`, `_Find_next(i)` iterate set bits fast. Size must be a compile-time constant; for dynamic sizes pick the max or template over sizes.

2.6.4 Gray code

`g = i ^ (i >> 1)`: consecutive values differ in one bit — enumerate subsets toggling one element at a time (useful when add/remove of an item is O(1) but recomputation is expensive).

2.7 Constructive and Ad-hoc Problems

Div2 C – Div1 C problems are very often "observation + simple implementation". Process:

1. **Brute force small cases** ($n \le 8$) and *print* the answers / an optimal construction. Most patterns are visible immediately (answer depends on parity, n mod 3, whether n is a power of 2, …).
2. **Find a lower bound** (each operation fixes at most one inversion / changes the sum by at most 2 / reduces the number of components by one) and then **construct a solution meeting it**.
3. **Necessary conditions first** (sum parity, total divisible by k, counts of each value), then check with the brute force whether they are also sufficient.

2.7.1 Invariants and monovariants

- Quantities that never change: sum mod k, parity of the number of inversions (each swap flips it), xor of all elements, colour counts on a chessboard (dominoes cover one black + one white), Σ x_i·i mod something.
- Quantities that strictly decrease: sum of values, number of inversions, potential functions $\Rightarrow$ the process terminates and the number of steps is bounded.
- Permutation parity: a sequence of transpositions realises p only with the same parity as p; 3-cycles are always even.

2.7.2 Construction toolbox

- **Extreme first:** place the largest/smallest element, solve the rest recursively ($n \to n-1$ or $n \to n-2$ induction). Writing the solution as a recursion on n often directly gives the code.
- **Pairing:** pair i with $n+1-i$, or adjacent pairs (1,2), (3,4), …; handle the odd leftover separately.
- **Alternating patterns:** 1, n, 2, $n-1$, … (maximises adjacent differences); zig-zag for "no three monotone".
- **Sorting-based:** sort and then interleave halves (`a[0], a[n/2], a[1], a[n/2+1], …`) to avoid equal neighbours or to make every element differ from its position (derangement: rotate by one).
- **Graph constructions:** star (diameter 2), path (diameter $n-1$), complete bipartite, binary tree (depth log n), cycles for "every vertex degree 2".
- **Bits:** build numbers bit by bit from the top; xor-based constructions (a, b, a ^ b); powers of two as "independent" choices (any number up to $2^k - 1$ is a subset sum of 1, 2, 4, …).
- **Two operations suffice:** many "minimum operations" problems have answer $\in \{0, 1, 2\}$; check 0, then 1 by brute force over all single operations, else 2.
- **Answer by symmetry / mirroring** in games (second player copies the first's move on the mirror).

2.7.3 Permutations and arrays

- Count/compute things via **cycles** of the permutation (swaps needed = n − cycles).
- Prefix maxima / suffix minima for "can we split the array into sorted parts".
- "Make all equal with operations on adjacent pairs": look at the **difference array** — operations become point updates.
- "Operation on a subarray": look at prefix sums — subarray operations become two point updates.
- Lexicographically smallest result: greedy left to right, choose the smallest feasible value at each position with a fast feasibility test.

2.7.4 Ad-hoc reading discipline

- Many easy ACPC problems hide one special case: $n = 1$, all values equal, zero, negative numbers, answer exactly at a boundary, ties broken in a specific way. Before submitting, list the edge cases explicitly.
- Rephrase the problem in one mathematical sentence; if you can't, you haven't understood it yet.
- Simulation problems: follow the statement literally; don't "optimise" the rules away until a brute version works.

3. Data Structures

3.1 Disjoint Set Union

```cpp
// [stress-tested]
struct DSU {
    vector<int> p;                      // p[x] < 0: x is root, size = -p[x]
    DSU(int n) : p(n, -1) {}
    int find(int x) { return p[x] < 0 ? x : p[x] = find(p[x]); }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -p[find(x)]; }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (p[a] > p[b]) swap(a, b);     // union by size
        p[a] += p[b]; p[b] = a;
        return true;
    }
};
```

Recursion depth is O(log n) thanks to union by size, so `find` is safe even for $n = 10^6$.

3.1.1 Uses

- Connected components online under edge additions; Kruskal MST (§5.2); cycle detection in undirected graphs (`unite` returns false).
- **Offline deletions** (ACPC 2022 *Supply Routes*): read all operations, start from the final graph (all never-deleted edges), process operations in reverse: a deletion becomes an addition, queries are answered in reverse and output reversed.
- "**Next free slot**" **pointer jumping:** `nxt = DSU` where after using slot i you link $i \to i+1$; `find(i)` gives the first free slot $\geq i$. Paint-the-range problems become $O(n\,\alpha)$.
- Track per-component aggregates (sum, min, count of something) in arrays indexed by root; merge them in `unite`.
- Bipartiteness / parity constraints online: DSU with parity (below).
- Process edges in sorted order of weight to answer "max edge on path / min bottleneck" queries offline (Kruskal reconstruction tree idea: when a union happens at weight w, all pairs between the two components have bottleneck w).

3.1.2 DSU with parity / potential (weighted DSU)

Maintains `d[x]` = value(x) − value(root) for relations like "x and y differ by w" or parity "x and y have different colours".

```cpp
// [stress-tested]
struct WDSU {
    vector<int> p; vector<ll> d;        // d[x] = val[x] - val[parent chain root]
    WDSU(int n) : p(n, -1), d(n, 0) {}
    int find(int x) {
        if (p[x] < 0) return x;
        int r = find(p[x]);
        d[x] += d[p[x]]; p[x] = r;      // d[p[x]] is already relative to r
        return r;
    }
    // add constraint val[b] - val[a] = w; returns false on contradiction
    bool unite(int a, int b, ll w) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return d[b] - d[a] == w;
        if (p[ra] > p[rb]) { swap(ra, rb); swap(a, b); w = -w; }
        // attach rb under ra: val[rb] - val[ra] = d[a] + w - d[b]
        p[ra] += p[rb]; p[rb] = ra; d[rb] = d[a] + w - d[b];
        return true;
    }
    ll diff(int a, int b) { find(a); find(b); return d[b] - d[a]; }  // assumes same set
};
```

For parity, use w ∈ {0,1} and compare `(d[b] - d[a]) & 1` (or replace + by xor).

3.1.3 DSU with rollback (for offline dynamic connectivity / divide & conquer)

No path compression; `undo()` pops the last union. Each `find` is O(log n).

```cpp
// [stress-tested]
struct RollbackDSU {
    vector<int> p; vector<pair<int,int>> st;   // (attached root, old parent size)
```

```
    int comps;
    RollbackDSU(int n) : p(n, -1), comps(n) {}
    int find(int x) { while (p[x] >= 0) x = p[x]; return x; }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) { st.push_back({-1, -1}); return false; }
        if (p[a] > p[b]) swap(a, b);
        st.push_back({b, p[b]});
        p[a] += p[b]; p[b] = a; --comps;
        return true;
    }
    void undo() {
        auto [b, sb] = st.back(); st.pop_back();
        if (b < 0) return;
        int a = p[b]; p[b] = sb; p[a] -= sb; ++comps;
    }
};
```

Offline dynamic connectivity: each edge is alive during a time interval; insert the interval into a segment tree over time (O(log T) nodes); DFS the segment tree, `unite` on entry, `undo` on exit, answer queries at leaves. O(m log T log n).

3.2 Fenwick Tree (BIT)

Point update, prefix query, O(log n), tiny constant. Works for any **invertible** operation (sum, xor) for range queries; for max/min only prefix queries with non-decreasing updates.

```
// [stress-tested] 0-indexed interface, internally 1-indexed
struct BIT {
    int n; vector<ll> t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v) { for (++i; i <= n; i += i & -i) t[i] += v; }
    ll pref(int i) { ll s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }  // sum [0, i)
    ll sum(int l, int r) { return pref(r) - pref(l); }                            // sum [l, r)
    // smallest idx such that pref(idx + 1) >= k (all values must be >= 0); returns n if none
    int lower_bound(ll k) {
        int pos = 0;
        for (int pw = 1 << __lg(max(n, 1)); pw; pw >>= 1)
            if (pos + pw <= n && t[pos + pw] < k) { pos += pw; k -= t[pos]; }
        return pos;
    }
};
```

- **Order statistics:** BIT over compressed values with counts; `lower_bound(k)` = k-th smallest (1-based k). Insert/erase = `add(v, ±1)`.
- **Count inversions:** iterate from left, `ans += i - pref(a[i] + 1)` (number of previous elements > a[i]), then `add(a[i], 1)`. Values compressed to 0..n−1.
- **Count of distinct values in ranges (offline):** sort queries by r; when processing position i, `add(i, +1)` and `add(last[a[i]], -1)`; answer = sum(l, r+1).
- **2D points dominance count:** sort by x, BIT on y (sweep, §2.2).

3.2.1 Range update, point query / range update, range query

```
// [stress-tested] range add, point query: store differences
struct BITRangeAdd {
    BIT b; BITRangeAdd(int n) : b(n + 1) {}
    void add(int l, int r, ll v) { b.add(l, v); b.add(r + 1, -v); }   // a[l..r] += v (inclusive)
    ll get(int i) { return b.pref(i + 1); }
};
// range add, range sum: two BITs
struct BITRangeRange {
    BIT b1, b2; BITRangeRange(int n) : b1(n + 1), b2(n + 1) {}
    void add(int l, int r, ll v) {                                    // inclusive
        b1.add(l, v); b1.add(r + 1, -v);
        b2.add(l, v * l); b2.add(r + 1, -v * (r + 1));
    }
    ll pref(int i) { return b1.pref(i) * i - b2.pref(i); }            // sum [0, i)
    ll sum(int l, int r) { return pref(r + 1) - pref(l); }            // inclusive
};
```

3.2.2 2D BIT

```
struct BIT2D {   // point add, rectangle-prefix sum; n*m memory
    int n, m; vector<vector<ll>> t;
```

```
    BIT2D(int n, int m) : n(n), m(m), t(n + 1, vector<ll>(m + 1)) {}
    void add(int x, int y, ll v) {
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j) t[i][j] += v;
    }
    ll pref(int x, int y) {  // sum over [0,x) x [0,y)
        ll s = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) s += t[i][j];
        return s;
    }
};
```

If coordinates are large and points known in advance: compress, or do the offline sweep with 1D BIT instead.

3.3 Segment Tree

3.3.1 Iterative (point update, range query) — fastest to type

```
// [stress-tested] change T / ID / op; op must be associative (need not be commutative)
struct Seg {
    using T = ll;
    static constexpr T ID = LLONG_MIN;
    static T op(T a, T b) { return max(a, b); }
    int n; vector<T> t;
    Seg(int n) : n(n), t(2 * n, ID) {}
    void set(int i, T v) {
        for (t[i += n] = v; i > 1; i >>= 1) t[i >> 1] = op(t[i & ~1], t[i | 1]);
    }
    T query(int l, int r) {                // [l, r)
        T ra = ID, rb = ID;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ra = op(ra, t[l++]);
            if (r & 1) rb = op(t[--r], rb);
        }
        return op(ra, rb);
    }
};
```

3.3.2 Lazy propagation: range assign + range add, query sum/min/max

```
// [stress-tested] 0-indexed, inclusive [l, r]. Delete what you don't need.
struct LazySeg {
    struct Node { ll sum = 0, mn = INF, mx = -INF; };
    static Node merge(const Node& a, const Node& b) {
        return {a.sum + b.sum, min(a.mn, b.mn), max(a.mx, b.mx)};
    }
    int n; vector<Node> t; vector<ll> add, setv; vector<char> hasSet;
    LazySeg(const vector<ll>& a) : n(sz(a)), t(4 * n), add(4 * n), setv(4 * n), hasSet(4 * n) {
        build(1, 0, n - 1, a);
    }
    void build(int x, int l, int r, const vector<ll>& a) {
        if (l == r) { t[x] = {a[l], a[l], a[l]}; return; }
        int m = (l + r) / 2;
        build(2 * x, l, m, a); build(2 * x + 1, m + 1, r, a);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    void applySet(int x, int l, int r, ll v) {
        t[x] = {v * (r - l + 1), v, v}; hasSet[x] = 1; setv[x] = v; add[x] = 0;
    }
    void applyAdd(int x, int l, int r, ll v) {
        t[x].sum += v * (r - l + 1); t[x].mn += v; t[x].mx += v;
        if (hasSet[x]) setv[x] += v; else add[x] += v;
    }
    void push(int x, int l, int r) {
        int m = (l + r) / 2;
        if (hasSet[x]) {
            applySet(2 * x, l, m, setv[x]); applySet(2 * x + 1, m + 1, r, setv[x]); hasSet[x] = 0;
        }
        if (add[x]) {
            applyAdd(2 * x, l, m, add[x]); applyAdd(2 * x + 1, m + 1, r, add[x]); add[x] = 0;
        }
    }
    // type 0 = add v, type 1 = assign v
    void update(int ql, int qr, ll v, int type) { update(1, 0, n - 1, ql, qr, v, type); }
    void update(int x, int l, int r, int ql, int qr, ll v, int type) {
```

```
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { type ? applySet(x, l, r, v) : applyAdd(x, l, r, v); return; }
        push(x, l, r);
        int m = (l + r) / 2;
        update(2 * x, l, m, ql, qr, v, type); update(2 * x + 1, m + 1, r, ql, qr, v, type);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    Node query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }
    Node query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return Node();
        if (ql <= l && r <= qr) return t[x];
        push(x, l, r);
        int m = (l + r) / 2;
        return merge(query(2 * x, l, m, ql, qr), query(2 * x + 1, m + 1, r, ql, qr));
    }
    // first index i >= ql with a[i] >= v, or -1  (O(log^2 n) worst, usually ~log n)
    int firstAtLeast(int ql, ll v) { return firstAtLeast(1, 0, n - 1, ql, v); }
    int firstAtLeast(int x, int l, int r, int ql, ll v) {
        if (r < ql || t[x].mx < v) return -1;
        if (l == r) return l;
        push(x, l, r);
        int m = (l + r) / 2;
        int res = firstAtLeast(2 * x, l, m, ql, v);
        return res != -1 ? res : firstAtLeast(2 * x + 1, m + 1, r, ql, v);
    }
};
```

How to design a new lazy segment tree: decide (1) the node summary, (2) the tag, (3) how a tag changes a node summary *using only the summary and the segment length*, (4) how two tags compose (new tag applied after old). If (3) is impossible, the problem needs a different summary (e.g. store sum of squares to support range add with sum-of-squares query: $\Sigma(x+v)^2 = \Sigma x^2 + 2v\Sigma x + \text{len}\cdot v^2$).

3.3.3 Useful node designs

query	node stores	merge
max subarray sum	`sum, pref, suf, best`	`best = max(a.best, b.best, a.suf + b.pref)`, `pref = max(a.pref, a.sum + b.pref)`
count of minimum	`mn, cnt`	take smaller, add counts on tie
longest run of 1s	`len, pref1, suf1, best`	like max subarray
bracket sequence validity	`open, close` unmatched	`m = min(a.open, b.close)`; `open = a.open + b.open - m`, `close = a.close + b.close - m`
polynomial hash of range	`hash, len`	`a.hash · B^len_b + b.hash`
matrix product (DP transitions along array)	2×2 / 3×3 matrix	matrix multiply (order matters)
area of union of rectangles (sweep)	`cnt, coveredLen`	no push: `covered = cnt > 0 ? len : left + right`

3.3.4 Persistent segment tree: k-th smallest in a range

```
// [stress-tested] versions: root[0] = 0 (empty), root[i+1] = insert a[i] into root[i].
// kth(root[l], root[r+1], k) = k-th smallest (1-based) value index among a[l..r].
struct PST {
    struct N { int l, r, c; };
    vector<N> t; int V;                         // values are 0..V-1 (compress first)
    PST(int V) : t(1, {0, 0, 0}), V(V) {}
    int insert(int prev, int pos) { return ins(prev, 0, V - 1, pos); }
    int ins(int prev, int lo, int hi, int pos) {
        int cur = sz(t); t.push_back(t[prev]); t[cur].c++;
        if (lo < hi) {
            int mid = (lo + hi) / 2;
            if (pos <= mid) { int x = ins(t[prev].l, lo, mid, pos); t[cur].l = x; }
            else { int x = ins(t[prev].r, mid + 1, hi, pos); t[cur].r = x; }
        }
        return cur;
    }
    int kth(int a, int b, int k) {
        int lo = 0, hi = V - 1;
        while (lo < hi) {
            int mid = (lo + hi) / 2, cnt = t[t[b].l].c - t[t[a].l].c;
            if (k <= cnt) { a = t[a].l; b = t[b].l; hi = mid; }
```

```
            else { k -= cnt; a = t[a].r; b = t[b].r; lo = mid + 1; }
        }
        return lo;
    }
};
```

Memory: n·(log V + 1) nodes. The same "difference of two versions" trick counts values ≤ x in a range, online.

3.3.5 Variations (when you need them)

- **Segment tree on values** (counts per value, compressed): k-th smallest by descending (go left if left count ≥ k).
- **Merge-sort tree**: each node stores the sorted subarray → count elements ≤ x in [l, r] in $O(\log^2 n)$. Memory n log n.
- **Dynamic / implicit segment tree**: create children on demand (`vector<array<int,2>> ch`) when coordinates are up to 10^9 and online. Offline → compress instead.
- **Persistent segment tree**: every update copies O(log n) nodes; version i = prefix i of array → k-th smallest in range via `root[r] - root[l-1]`.
- **Segment tree over time** for offline dynamic connectivity (§3.1).
- Recursion is fine for n ≤ 10^6; size arrays `4n`.

3.4 Sparse Table, Ordered Set, Treap, Mo's, Small-to-Large

3.4.1 Sparse table (static RMQ in O(1))

Works for idempotent operations (min, max, gcd, and, or). O(n log n) build.

```
// [stress-tested]
struct Sparse {
    vector<vector<ll>> t;
    static ll op(ll a, ll b) { return min(a, b); }
    Sparse(const vector<ll>& a) {
        int n = sz(a), K = __lg(max(n, 1)) + 1;
        t.assign(K, a);
        rep(k, 1, K) rep(i, 0, n - (1 << k) + 1)
            t[k][i] = op(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    }
    ll query(int l, int r) {              // inclusive, l <= r
        int k = __lg(r - l + 1);
        return op(t[k][l], t[k][r - (1 << k) + 1]);
    }
};
```

For non-idempotent ops (sum, product) use prefix sums / disjoint sparse table / segment tree. Range GCD with sparse table is O(log) per query due to gcd cost.

3.4.2 Ordered set (PBDS)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
// os.find_by_order(k) -> iterator to k-th smallest (0-based); os.order_of_key(x) -> #elements < x
// multiset: store pair<value, unique_id>; order_of_key({x, -1}) = #elements < x
```

3.4.3 Implicit treap (sequence with split/merge, reverse, range sum)

Use for: cut a block and paste elsewhere, reverse ranges, insert/delete at position — all O(log n).

```
// [stress-tested]
mt19937 trng(91);
struct Treap {
    struct N { int l = 0, r = 0, sz = 0; unsigned pr = 0; ll val = 0, sum = 0; bool rev = false; };
    vector<N> t;
    Treap() { t.push_back(N()); }        // node 0 = null
    int make(ll v) { N x; x.sz = 1; x.pr = trng(); x.val = x.sum = v; t.push_back(x); return sz(t) - 1; }
    void pull(int x) {
        t[x].sz = 1 + t[t[x].l].sz + t[t[x].r].sz;
        t[x].sum = t[x].val + t[t[x].l].sum + t[t[x].r].sum;
    }
    void push(int x) {
        if (x && t[x].rev) {
            swap(t[x].l, t[x].r);
            if (t[x].l) t[t[x].l].rev ^= 1;
```

```
            if (t[x].r) t[t[x].r].rev ^= 1;
            t[x].rev = false;
        }
    }
    // first k elements -> a, rest -> b
    void split(int x, int k, int& a, int& b) {
        if (!x) { a = b = 0; return; }
        push(x);
        if (t[t[x].l].sz < k) { split(t[x].r, k - t[t[x].l].sz - 1, t[x].r, b); a = x; }
        else { split(t[x].l, k, a, t[x].l); b = x; }
        pull(x);
    }
    int merge(int a, int b) {
        if (!a || !b) return a ^ b;
        if (t[a].pr > t[b].pr) { push(a); t[a].r = merge(t[a].r, b); pull(a); return a; }
        push(b); t[b].l = merge(a, t[b].l); pull(b); return b;
    }
    // helpers on root (0-indexed, inclusive ranges)
    void insert(int& root, int pos, ll v) {
        int a, b; split(root, pos, a, b); root = merge(merge(a, make(v)), b);
    }
    void erase(int& root, int pos) {
        int a, b, c; split(root, pos, a, b); split(b, 1, b, c); root = merge(a, c);
    }
    void reverse(int& root, int l, int r) {
        int a, b, c; split(root, l, a, b); split(b, r - l + 1, b, c);
        t[b].rev ^= 1; root = merge(merge(a, b), c);
    }
    ll rangeSum(int& root, int l, int r) {
        int a, b, c; split(root, l, a, b); split(b, r - l + 1, b, c);
        ll s = t[b].sum; root = merge(merge(a, b), c); return s;
    }
    void collect(int x, vector<ll>& out) {   // in-order
        if (!x) return; push(x); collect(t[x].l, out); out.push_back(t[x].val); collect(t[x].r, out);
    }
};
```

Keyed treap (ordered by value) = same with split by key `t[x].val < k`. Add other lazy tags (range add) exactly like the segment tree: apply to node + mark children in `push`.

3.4.4 Mo's algorithm (offline range queries, O((n + q)√n))

Needs O(1) (or O(log)) add/remove of one element at either end. Example: number of distinct values in [l, r].

```
// [stress-tested]
vector<int> mo(const vector<int>& a, const vector<pair<int,int>>& qs) {  // inclusive queries
    int n = sz(a), q = sz(qs), B = max(1, (int)(n / sqrt(max(q, 1))));
    vector<int> ord(q); iota(all(ord), 0);
    sort(all(ord), [&](int i, int j) {
        int bi = qs[i].first / B, bj = qs[j].first / B;
        if (bi != bj) return bi < bj;
        return (bi & 1) ? qs[i].second > qs[j].second : qs[i].second < qs[j].second;
    });
    vector<int> cnt(*max_element(all(a)) + 1), res(q);   // compress a first if values are big
    int cur = 0, L = 0, R = -1;
    auto add = [&](int i) { if (cnt[a[i]]++ == 0) ++cur; };
    auto rem = [&](int i) { if (--cnt[a[i]] == 0) --cur; };
    for (int id : ord) {
        auto [l, r] = qs[id];
        while (R < r) add(++R);
        while (L > l) add(--L);
        while (R > r) rem(R--);
        while (L < l) rem(L++);
        res[id] = cur;
    }
    return res;
}
```

Expand before shrinking (as above) so the window is never "negative". Mo on trees: run on the Euler tour (ET of size 2n, LCA handled separately).

3.4.5 Small-to-large merging

Merging sets of children into the parent: always insert the smaller into the larger (`if (sz(A) < sz(B)) swap(A, B);`). Each element moves O(log n) times → O(n log² n) with `set/map`, O(n log n) with hash/vectors. Classic use: for every subtree, number of distinct colours / most frequent colour / answer queries about subtree.

```cpp
// distinct colours in every subtree; g = children lists, col = colours
void dfsMerge(int v, const vector<vector<int>>& g, const vector<int>& col,
              vector<set<int>>& S, vector<int>& ans) {
    S[v].insert(col[v]);
    for (int c : g[v]) {
        dfsMerge(c, g, col, S, ans);
        if (sz(S[c]) > sz(S[v])) swap(S[c], S[v]);     // O(1) swap of std::set
        for (int x : S[c]) S[v].insert(x);
        S[c].clear();
    }
    ans[v] = sz(S[v]);
}
```

3.4.6 Other standard-library power tools

- `multiset` as a priority queue with deletion; `prev(ms.end())` = max.
- `map<ll, ll>` as an "interval map" (Chtholly / ODT): store maximal constant segments `[l → value]`; range assign = split at l and r+1, erase between, insert one — amortised O(log n) with random assignments.
- Set of disjoint intervals (merge on insert): `map<l, r>`, find the first interval with `prev(upper_bound(l))`, absorb overlaps.
- `deque` supports O(1) push/pop both ends + random access.
- `bitset` §2.6; `__int128` for 128-bit keys.

4. Dynamic Programming

4.1 DP Design and Classic Patterns

4.1.1 Designing a DP

1. **State** = the minimum information about the past that determines the future. Write it as a sentence: "dp[i][j] = max score using the first i items with j capacity used".
2. **Transition**: what is the last decision? (last item taken or not, where the last segment starts, which colour the last node got.)
3. **Order**: compute states after everything they depend on (increasing i; increasing interval length; post-order on a tree; topological order on a DAG). If there are cycles in the dependency, it's a shortest path / system of equations, not a DP.
4. **Count states × transitions** — reduce with prefix sums/min, monotonic deque, data structures, or a smarter state.
5. Base cases, answer location, modulo, overflow, ±INF for impossible states (use `INF/2` so additions don't overflow).

Tricks: **memoized recursion** is the fastest way to a correct DP when the order is unclear. **Rolling arrays** (`dp[i & 1]`) cut memory. **Sort first** if items can be processed in any order (by deadline, by weight) — then a knapsack-like DP over the sorted order is valid. To **reconstruct** the solution, store the argmax choice per state or re-derive by checking which transition achieves the optimum.

4.1.2 Knapsack family

```cpp
// [stress-tested] 0/1 knapsack: max value with capacity W  (iterate capacity DOWN)
ll knap01(const vector<int>& w, const vector<ll>& v, int W) {
    vector<ll> dp(W + 1, 0);
    rep(i, 0, sz(w)) for (int c = W; c >= w[i]; --c) dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
// unbounded: iterate capacity UP.  count ways to make sum (order-independent): items outer loop
ll countWays(const vector<int>& coins, int S, ll mod) {
    vector<ll> dp(S + 1, 0); dp[0] = 1;
    for (int c : coins) rep(s, c, S + 1) dp[s] = (dp[s] + dp[s - c]) % mod;
    return dp[S];   // swap loops (sum outer, coins inner) -> counts ordered sequences
}
```

- **Bounded** (item i available k_i times): split k into 1, 2, 4, …, remainder → O(W Σ log k_i). Or monotonic deque per residue class mod w → O(nW).
- **Values small, weights huge:** dp[value] = min weight.
- **Subset sum reachability**: bitset (§2.6). Σ w = n ⇒ ≤ √(2n) distinct weights.
- Min items to reach exact sum: dp[s] = min(dp[s − c] + 1).
- **Huge sums with few coin types (Frobenius / reachability for all large X):** Dijkstra on residues modulo the smallest coin (§5.1, ACPC 2022 *Braaaains*).

4.1.3 Sequences

```cpp
// [stress-tested] LIS length, O(n log n). For non-decreasing use upper_bound.
int lis(const vector<ll>& a) {
    vector<ll> d;                        // d[k] = min tail of an increasing subsequence of length k+1
    for (ll x : a) {
        auto it = lower_bound(all(d), x);
        if (it == d.end()) d.push_back(x); else *it = x;
    }
    return sz(d);
}
```

To reconstruct LIS: store for each i the position `pos[i]` it took in d and `prev[i]` = last index placed at `pos[i]-1`. Count of LIS: BIT over values storing (best length, count).

- **LCS** O(nm): `dp[i][j] = s[i-1]==t[j-1] ? dp[i-1][j-1]+1 : max(dp[i-1][j], dp[i][j-1])`.
- **Edit distance**: `dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1] + (s[i-1]!=t[j-1]))`.
- **Interleaving** s = merge of s1, s2 keeping order (ACPC 2018 *Homework*): `ok[i][j]` = first i+j chars of s are an interleaving of s1[0..i), s2[0..j): `ok[i][j] = (ok[i-1][j] && s1[i-1]==s[i+j-1]) || (ok[i][j-1] && s2[j-1]==s[i+j-1])`. $25 \cdot 10^6$ booleans is fine; use `vector<char>` or bitset rows.
- **Count distinct subsequences**: `dp[i] = 2·dp[i-1] − dp[last[c]-1]` where last[c] = previous occurrence of the same char.
- Max subarray sum (Kadane): `cur = max(x, cur + x)`.

4.1.4 Interval DP

`dp[l][r]` from smaller intervals; iterate `len` from 1 to n. $O(n^3)$ with a split point, $O(n^2)$ without.

```
// [stress-tested] number of palindromic subsequences (as index sets, non-empty) mod p
// (ACPC 2016 Palindromic Naming)
ll countPalSubseq(const string& s, ll mod) {
    int n = sz(s);
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int l = n - 1; l >= 0; --l) {
        dp[l][l] = 1;
        rep(r, l + 1, n) {
            ll in = (l + 1 <= r - 1) ? dp[l + 1][r - 1] : 0;
            dp[l][r] = (dp[l + 1][r] + dp[l][r - 1] - in + mod) % mod;  // inclusion-exclusion
            // s[l], s[r] can wrap any inner palindrome, or nothing
            if (s[l] == s[r]) dp[l][r] = (dp[l][r] + in + 1) % mod;
        }
    }
    return dp[0][n - 1];
}
```

- **Split point**: `dp[l][r] = min over l ≤ k < r of dp[l][k] + dp[k+1][r] + cost(l, r)` (matrix chain, merging stones, optimal BST; Knuth optimisation §4.4 if cost is quadrangle-monotone).
- **Expressions with + and −, choose parenthesisation to maximise** (ACPC 2022 *Maximal Parentheticals*): keep **both max and min** of every sub-expression, since `a - (b)` is maximised by minimising b. With multiplication by negatives also keep both (that problem also lets parentheses create unary minus + implicit multiplication — enumerate every rule combination).
- **Remove/merge adjacent elements** ("zuma", "remove all with cost"): dp over [l, r] plus an extra dimension for "how many equal elements are glued to the right end".
- **Convex polygon DP** (triangulations, ACPC 2018 *Trimming Convex Polygons*): vertices 0..n−1 in order; choosing a sub-polygon = choosing a subsequence of vertices; its area is a fan of triangles from the first chosen vertex. dp over (first kept vertex fixed, last kept vertex) with transitions adding one triangle: $O(n^3)$.
- **Circular** intervals: double the array and take windows of length n, or fix the element at position 0.

4.1.5 Grid, DAG, partition DPs

- Grid paths with obstacles: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`. Count paths avoiding k special cells (n, m large): sort cells, inclusion–exclusion with C(dx+dy, dx) — $O(k^2)$.
- Grid with moves S/E/SE and accumulating state (ACPC 2024 *Alpine Agility*): row-major order works because all moves go forward.
- DAG longest path / count paths: topological order (§5.2).
- **Partition into segments**: `dp[i] = min_j dp[j] + cost(j, i)`; with k segments add a dimension → D&C / CHT optimisations (§4.4) or the λ-trick.
- **State "was my neighbour already processed"**: when the order of operations on a line matters only locally (destroying buildings that damage neighbours, ACPC 2016 *City Destruction*), dp over position i with a flag for whether i−1 is destroyed before or after i.

4.1.6 Counting DPs (mod p)

- dp over (position, last value) with prefix sums over last value → $O(n \cdot V)$.
- "Number of sequences with sum S": dp over sum; with bounded parts use prefix sums to make each step O(1).
- Avoid double counting: define a **canonical** construction (e.g. always extend from the leftmost undecided position, or count by "first time the event happens").
- When the DP counts objects up to symmetry, use Burnside (§7.3) instead.

4.2 Bitmask DP, Digit DP, SOS, Broken Profile

4.2.1 Bitmask DP

Assignment / ordering of n ≤ 20 items. `dp[mask]` = best for having used the set `mask`; the next item index is often `popcount(mask)`.

```
// [stress-tested] TSP / Hamiltonian path: dp[mask][v] = min cost path visiting mask, ending at v. O(2^n n^2)
ll tsp(const vector<vector<ll>>& d) {      // returns min Hamiltonian cycle from 0
    int n = sz(d);
    vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
    dp[1][0] = 0;
    rep(mask, 1, 1 << n) rep(v, 0, n) {
```

```
        if (!(mask >> v & 1) || dp[mask][v] >= INF) continue;
        rep(u, 0, n) if (!(mask >> u & 1))
            dp[mask | 1 << u][u] = min(dp[mask | 1 << u][u], dp[mask][v] + d[v][u]);
    }
    ll best = INF;
    rep(v, 1, n) best = min(best, dp[(1 << n) - 1][v] + d[v][0]);
    return n == 1 ? 0 : best;
}
```

- **Assignment** (n workers ≤ 20 to n jobs): `dp[mask]` over assigned jobs, worker = popcount(mask). $O(2^n \cdot n)$. For n > 20 use Hungarian (§6.3).
- **Partition into groups** (min #groups each with weight ≤ W): `dp[mask] = (groups, current fill)` pairs, compare lexicographically. $O(2^n \cdot n)$.
- **Over submasks**: `dp[mask] = min over sub ⊆ mask of dp[mask ^ sub] + cost[sub]` → $O(3^n)$; n ≤ 15–16.
- Memory: `2^20 × 20` ints = 80 MB; prefer `int` and drop dimensions.
- Bitmask over a **small dimension** of a grid (rows ≤ 10): dp[column][mask of current column].

4.2.2 SOS DP (sum over subsets)

`F[mask] = Σ_{sub ⊆ mask} A[sub]` in $O(n \cdot 2^n)$:

```
// [stress-tested]
void sos(vector<ll>& F, int n) {          // in place: F starts as A
    rep(i, 0, n) rep(mask, 0, 1 << n) if (mask >> i & 1) F[mask] += F[mask ^ (1 << i)];
}
// superset sums: condition !(mask >> i & 1), F[mask] += F[mask | 1 << i]
// inverse (Möbius): same loops with -=
```

Uses: count pairs with `a_i & a_j == 0` (count submasks of the complement), `a_i | a_j ⊆ x`, number of elements that are submasks of each x. AND/OR convolution = SOS transform, multiply pointwise, inverse transform.

4.2.3 Digit DP

Count integers in [0, N] with a property depending on digits. Answer for [L, R] = f(R) − f(L − 1).

```
// [stress-tested] count x in [0, N] whose digit sum is divisible by K (K <= 100)
ll countDigitSum(ll N, int K) {
    if (N < 0) return 0;
    string s = to_string(N); int n = sz(s);
    // memo[pos][rem] for non-tight states only (tight path is unique, recomputed)
    vector<vector<ll>> memo(n, vector<ll>(K, -1));
    function<ll(int, int, bool)> go = [&](int pos, int rem, bool tight) -> ll {
        if (pos == n) return rem == 0;
        if (!tight && memo[pos][rem] != -1) return memo[pos][rem];
        int lim = tight ? s[pos] - '0' : 9;
        ll res = 0;
        rep(d, 0, lim + 1) res += go(pos + 1, (rem + d) % K, tight && d == lim);
        if (!tight) memo[pos][rem] = res;
        return res;
    };
    return go(0, 0, true);
}
```

Extra state flags as needed: `started` (to handle leading zeros, e.g. "no two adjacent equal digits"), `last digit`, `count of nonzero digits`, `rem mod m`. For "k-th number with property": compute counts per prefix choice and descend digit by digit.

4.2.4 Broken profile DP (tilings)

Count domino tilings of an n × m grid (n ≤ 10): process cells in row-major order; mask = which of the next m cells are already covered.

```
// [stress-tested]
ll dominoTilings(int n, int m, ll mod) {          // m = width (small)
    vector<ll> dp(1 << m, 0); dp[0] = 1;
    rep(i, 0, n) rep(j, 0, m) {
        vector<ll> nd(1 << m, 0);
        rep(mask, 0, 1 << m) if (dp[mask]) {
            ll v = dp[mask];
            if (mask >> j & 1) {                                          // already covered
                nd[mask ^ (1 << j)] = (nd[mask ^ (1 << j)] + v) % mod; continue;
            }
            nd[mask | (1 << j)] = (nd[mask | (1 << j)] + v) % mod;                  // vertical domino down
            if (j + 1 < m && !(mask >> (j + 1) & 1))                                // horizontal domino right
```

```
                nd[mask | (1 << (j + 1))] = (nd[mask | (1 << (j + 1))] + v) % mod;
        }
        dp = nd;
    }
    return dp[0];
}
```

Bit j of the mask refers to cell (i, j) when processing row i (set = covered by a vertical domino from above or a horizontal domino from the left). The same skeleton handles "place objects with local constraints" problems (e.g. minesweeper-like boards: keep the last two rows in the state when constraints reach distance 1).

4.3 Tree DP and Rerooting

Root the tree (usually at 0), compute children before parents. For n up to 10^6 use an **iterative** order: BFS/DFS order array, then process it in reverse.

```
// [stress-tested] parent + pre-order. Bottom-up: for (int i = n-1; i >= 0; --i) { v = order[i]; ... }
void rootTree(const vector<vector<int>>& g, int root, vector<int>& par, vector<int>& order) {
    int n = sz(g); par.assign(n, -1); order.clear(); order.reserve(n);
    vector<int> st = {root}; par[root] = root;
    while (!st.empty()) {
        int v = st.back(); st.pop_back(); order.push_back(v);
        for (int u : g[v]) if (u != par[v]) { par[u] = v; st.push_back(u); }
    }
    par[root] = -1;
}
```

4.3.1 Standard subtree DPs

- **Max independent set** (weights w): `in[v] = w_v + Σ out[c]`, `out[v] = Σ max(in[c], out[c])`.
- **Min vertex cover** = n − max independent set (trees are bipartite); **max matching** greedy from leaves: match a leaf with its parent if both free.
- **Diameter**: for each v keep the two longest downward paths `d1 ≥ d2`; diameter = max(d1 + d2). (Or BFS twice for unweighted/positive weights.)
- **Subtree sizes / sums**, **counting pairs at distance k**: small-to-large or centroid (§5.4).

Colouring with k colours minimising cost (ACPC 2022 *City Numbers* pattern)

`dp[v][c]` = min cost of subtree of v when v gets colour c = cost(c) + Σ_children min_{c' ≠ c} dp[child][c']. The inner min over c' ≠ c is O(1) with the child's **best and second best**:

```
// [stress-tested] colours 1..k, cost of colour c = c. Min total cost (k >= 2 is always feasible)
ll minColouring(const vector<vector<int>>& g, int k) {
    int n = sz(g); vector<int> par, order; rootTree(g, 0, par, order);
    vector<vector<ll>> dp(n, vector<ll>(k + 1, 0));
    for (int i = n - 1; i >= 0; --i) {
        int v = order[i];
        rep(c, 1, k + 1) dp[v][c] += c;
        if (par[v] < 0) continue;
        ll b1 = INF, b2 = INF; int bc = -1;          // best, second best over colours of v
        rep(c, 1, k + 1) {
            if (dp[v][c] < b1) { b2 = b1; b1 = dp[v][c]; bc = c; }
            else if (dp[v][c] < b2) b2 = dp[v][c];
        }
        rep(c, 1, k + 1) dp[par[v]][c] += (c == bc ? b2 : b1);
    }
    return *min_element(dp[0].begin() + 1, dp[0].end());
}
```

(Fact: an optimal colouring never needs more than O(log n) colours, so `k = min(k, 20)` is safe; with k = 1 and n > 1 it's impossible.)

4.3.2 Tree knapsack (merge children with size bounds) — O(n²) total

`dp[v][j]` = best value choosing j nodes in subtree of v. Merging child c into v: loop `j ≤ size(v so far)`, `t ≤ size(c)`. Bounding loops by current sizes makes the total O(n²) (each pair of nodes is "merged" once at their LCA). With a cap K on j: O(nK).

```
// sketch of merge step: cur = dp of v so far (size sv), ch = dp of child (size sc)
vector<ll> nd(sv + sc + 1, -INF);
rep(a, 0, sv + 1) if (cur[a] > -INF) rep(b, 0, sc + 1) if (ch[b] > -INF)
```

```
    nd[a + b] = max(nd[a + b], cur[a] + ch[b]);
cur = nd; sv += sc;
```

4.3.3 Rerooting (answer for every root)

Two passes: `down[v]` (subtree answer) bottom-up, then `up[v]` (answer of the part outside v's subtree) top-down; combine. Requires: the contribution of a child to its parent can be computed from the child's value, and the combination is "removable" (sum) or handled with prefix/suffix aggregates (max).

```
// [stress-tested] sum of distances from every vertex to all others, O(n)
vector<ll> sumDistAll(const vector<vector<int>>& g) {
    int n = sz(g); vector<int> par, order; rootTree(g, 0, par, order);
    vector<ll> sub(n, 1), down(n, 0), ans(n, 0);
    for (int i = n - 1; i > 0; --i) {
        int v = order[i]; sub[par[v]] += sub[v]; down[par[v]] += down[v] + sub[v];
    }
    ans[0] = down[0];
    rep(i, 1, n) { int v = order[i]; ans[v] = ans[par[v]] - sub[v] + (n - sub[v]); }
    return ans;
}
```

Generic rerooting with max (e.g. farthest node from every vertex): for each node compute children contributions `f(child)`, prefix-max and suffix-max over the children list, so the value "excluding child c" is `max(prefix[i-1], suffix[i+1], up-contribution of v)`.

4.3.4 Other tree techniques (pointers)

- Euler tour (tin/tout) turns subtree into a contiguous range → BIT/segment tree on subtree queries (§5.4).
- "Is u an ancestor of v": `tin[u] ≤ tin[v] && tout[v] ≤ tout[u]` (ACPC 2020 *Listen To Your Boss*).
- DP on paths/LCA: binary lifting tables store aggregates (min/max edge) along 2^k jumps.
- Tree isomorphism: canonical hashing of sorted child hashes (AHU) from the centre(s).

4.4 DP Optimisations

recurrence shape	technique	complexity
`dp[i] = min_j dp[j] + f(j) + g(i)` with j in a sliding window	monotonic deque (§2.3)	O(n)
`dp[i] = min_j (a_j · x_i + b_j)` (+ terms of i)	convex hull trick / Li Chao	O(n log n)
`dp[k][i] = min_{j<i} dp[k-1][j] + C(j, i)`, opt(i) monotone	divide & conquer	O(k n log n)
`dp[i][j] = min_{i≤k<j} dp[i][k] + dp[k+1][j] + C(i, j)`	Knuth (opt[i][j−1] ≤ opt[i][j] ≤ opt[i+1][j])	O(n²)
"exactly k groups" and answer convex in k	Aliens trick (λ-penalty, binary search)	O(n log n log C)
`dp[x]` is convex piecewise linear in a value x	slope trick (heaps of breakpoints)	O(n log n)

Monge / quadrangle inequality `C(a, c) + C(b, d) ≤ C(a, d) + C(b, c)` for a ≤ b ≤ c ≤ d implies monotone opt (both D&C and Knuth). Typical Monge costs: (sum of segment)², sum of |x − median| over a segment, number of pairs inside a segment. When unsure, stress-test opt monotonicity with a brute force.

4.4.1 Convex hull trick (any insertion/query order)

```
// [stress-tested] max of k*x + m. For min: add(-k, -m) and negate query.
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }  // floor
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
    }
```

```
    ll query(ll x) {
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Watch overflow: $|k \cdot x|$ must fit in `ll`. Example: `dp[i] = min_j dp[j] + (p_i - p_j)²` = `p_i² + min_j (-2p_j·p_i + dp[j] + p_j²)` → line with slope `-2p_j`, intercept `dp[j] + p_j²`, query at `p_i`.

4.4.2 Li Chao tree (min, over a known set of query x's)

Also supports adding a line only on an x-range (descend to O(log n) nodes and insert there).

```
// [stress-tested]
struct LiChao {
    struct L { ll k, m; ll eval(ll x) const { return k * x + m; } };
    vector<ll> xs; vector<L> t; int n;
    LiChao(vector<ll> pts) : xs(pts) {
        sort(all(xs)); xs.erase(unique(all(xs)), xs.end());
        n = sz(xs); t.assign(4 * n, {0, INF});
    }
    void add(ll k, ll m) { add({k, m}, 1, 0, n - 1); }
    void add(L nw, int x, int l, int r) {
        int mid = (l + r) / 2;
        bool lef = nw.eval(xs[l]) < t[x].eval(xs[l]);
        bool md = nw.eval(xs[mid]) < t[x].eval(xs[mid]);
        if (md) swap(t[x], nw);
        if (l == r) return;
        if (lef != md) add(nw, 2 * x, l, mid); else add(nw, 2 * x + 1, mid + 1, r);
    }
    ll query(ll qx) {                    // qx must be one of pts
        int i = lower_bound(all(xs), qx) - xs.begin(), x = 1, l = 0, r = n - 1;
        ll res = INF;
        while (true) {
            res = min(res, t[x].eval(qx));
            if (l == r) return res;
            int mid = (l + r) / 2;
            if (i <= mid) { x = 2 * x; r = mid; } else { x = 2 * x + 1; l = mid + 1; }
        }
    }
};
```

The empty line `{0, INF}` evaluates to INF; don't add lines with huge slopes where `k*x` overflows.

4.4.3 Divide & conquer optimisation

```
// [stress-tested] split a[0..n) into exactly K contiguous groups minimising sum of cost(j, i) for
// group = a[j..i). cost must satisfy the quadrangle inequality.
ll partitionDC(int n, int K, const function<ll(int, int)>& cost) {
    vector<ll> prv(n + 1, INF), cur(n + 1, INF);
    prv[0] = 0;
    function<void(int, int, int, int)> solve = [&](int l, int r, int optl, int optr) {
        if (l > r) return;
        int mid = (l + r) / 2, opt = optl;
        ll best = INF;
        for (int j = optl; j <= min(mid - 1, optr); ++j)
            if (prv[j] < INF && prv[j] + cost(j, mid) < best) { best = prv[j] + cost(j, mid); opt = j; }
        cur[mid] = best;
        solve(l, mid - 1, optl, opt); solve(mid + 1, r, opt, optr);
    };
    rep(k, 0, K) { fill(all(cur), INF); solve(1, n, 0, n - 1); swap(prv, cur); }
    return prv[n];
}
```

4.4.4 Knuth optimisation (interval DP)

```
// sketch: dp[i][j] over closed intervals, C(i, j) Monge and monotone on inclusion
for (int i = 0; i < n; ++i) { dp[i][i] = base(i); opt[i][i] = i; }
for (int len = 2; len <= n; ++len)
    for (int i = 0; i + len - 1 < n; ++i) {
        int j = i + len - 1; dp[i][j] = INF;
        for (int k = opt[i][j - 1]; k <= min(j - 1, opt[i + 1][j]); ++k) {
            ll v = dp[i][k] + dp[k + 1][j] + C(i, j);
            if (v < dp[i][j]) { dp[i][j] = v; opt[i][j] = k; }
        }
    }
```

4.4.5 Aliens trick (Lagrangian relaxation)

If `f(k)` = optimal cost with exactly k groups/items is **convex** in k, then for a penalty λ solve the unconstrained problem where every group costs an extra λ: `g(λ) = min_k f(k) + λk`. Track the number of groups used (minimum count among ties). Binary search λ (integer) so that the count is ≤ k with smallest λ; answer `f(k) = g(λ) − λ·k`. Turns O(n·k) states into O(n) per λ.

4.4.6 Slope trick

Maintain a convex piecewise-linear function by the multiset of its slope-change points. Classic: minimum Σ|a_i − b_i| to make b non-decreasing:

```
// [stress-tested]
ll minCostNonDecreasing(const vector<ll>& a) {
    priority_queue<ll> pq; ll ans = 0;
    for (ll x : a) {
        pq.push(x);
        if (pq.top() > x) { ans += pq.top() - x; pq.pop(); pq.push(x); }
    }
    return ans;   // strictly increasing: apply to a_i - i
}
```

4.5 Probability, Expected Value, Games

ACPC has a probability/EV problem at difficulty 5–8 most years (Balls and Bins, Gunslingers, Concentration). They are almost always **DP over a small state space**, solved backward from terminal states.

4.5.1 Core facts

- **Linearity:** E[X + Y] = E[X] + E[Y] always (even dependent). Write the quantity as a sum of indicators: E[#fixed points of random permutation] = n·(1/n) = 1; E[#inversions] = n(n−1)/4.
- E[X] for X ≥ 0 integer: Σ_{k≥1} P(X ≥ k). E[max] via P(max ≤ k) = Π P(X_i ≤ k).
- **Geometric:** success prob p per trial ⇒ expected trials 1/p. **Coupon collector:** n·H_n = n(1 + 1/2 + … + 1/n).
- Independent events: P(A ∩ B) = P(A)P(B). Conditional: P(A | B) = P(A ∩ B)/P(B). Bayes: P(A | B) = P(B | A)P(A)/P(B).
- **Gambler's ruin** (fair): start at a, absorb at 0 or N ⇒ P(reach N) = a/N, expected steps a(N − a). Biased with ratio r = q/p: P(reach N) = $(1 - r^a)/(1 - r^N)$.
- Sum of probabilities over all outcomes must be 1 — assert it in your DP to catch bugs.
- Random permutation: element i ends before j with prob 1/2; i is the max of a random set of size k with prob 1/k.

4.5.2 Expected value DP

`E[s] = cost(s) + Σ_t P(s → t)·E[t]`, terminal states have E = 0. Compute in an order where t is done before s (e.g. "number of remaining items" decreases).

- **Self-loops** (state can stay the same with prob p): E = (c + Σ_{t ≠ s} P(t)E[t]) / (1 − p).
- **Optimal stopping / decisions** (ACPC 2021 *Balls and Bins*): `V[s] = max(value if stop now, Σ P(t)·(reward + V[t]))`. The state must capture everything that affects future probabilities — e.g. (#bins with 1 ball, #bins with 2 balls); the number of balls thrown is then determined.
- **Forward probability DP** when you need the distribution: `P[t] += P[s]·p(s → t)`; answer = Σ over final states.
- **Probability of winning with optimal play:** `W[s] = max over moves of Σ P(outcome)·W[next]` (or 1 − opponent's W when turns alternate).
- **Cycles in the state graph**:
- A *cyclic chain* (turns rotate among players until something happens): unroll. If `x_1 = a_1 + b_1 x_2, …, x_k = a_k + b_k x_1` then `x_1 = (a_1 + b_1 a_2 + b_1 b_2 a_3 + … ) / (1 − b_1 b_2 ⋯ b_k)`, then back-substitute. (Gunslingers: within a fixed set of survivors, misses rotate the turn; hits go to a smaller set, already solved.)
- General cycles: Gaussian elimination on the linear system (§7.4), O(n³) for n ≤ ~500.
- Infinite processes with decreasing probability: iterate until change < 1e-12 (value iteration) when the system is contractive.

```
// x[i] = a[i] + b[i] * x[(i+1) % k]  (cyclic chain), returns all x. Requires prod(b) != 1.
vector<double> solveCycle(const vector<double>& a, const vector<double>& b) {
    int k = sz(a); double num = 0, mul = 1;
    rep(i, 0, k) { num += mul * a[i]; mul *= b[i]; }
    vector<double> x(k);
    x[0] = num / (1 - mul);
    for (int i = k - 1; i >= 1; --i) x[i] = a[i] + b[i] * x[(i + 1) % k];
    return x;
}
```

Expected value modulo p

"Output E as $P \cdot Q^{-1} \bmod 998244353$": do the same DP with modular arithmetic; every division by an integer d becomes multiplication by `inv(d)` (§7.1). Probabilities like 1/n become `inv(n)`. `1/(1 - p)` becomes `inv((1 - p + MOD) % MOD)`.

Precision with doubles

Use `double` (or `long double`) and print 9+ decimals. Avoid computing tiny probabilities as products of thousands of terms (underflow → use logs). Subtracting nearly equal numbers loses precision — reformulate `(1 - (1 - p)^n` via `-expm1(n * log1p(-p))`).

4.5.3 Combinatorial games

Win/lose DP on a DAG of positions: a position is winning iff some move leads to a losing position; terminal positions are losing (normal play: player unable to move loses). Compute by memoized recursion.

Games with cycles (draws possible) — retrograde BFS: start from terminal positions; a position becomes LOSE when all its successors are WIN (count remaining successors), WIN when any successor is LOSE; positions never labelled are DRAW.

```
// g = moves (position -> next positions). res: 1 = win, -1 = lose, 0 = draw (for player to move)
vector<int> retrograde(const vector<vector<int>>& g) {
    int n = sz(g); vector<vector<int>> rg(n); vector<int> deg(n), res(n, 0);
    rep(v, 0, n) { deg[v] = sz(g[v]); for (int u : g[v]) rg[u].push_back(v); }
    queue<int> q;
    rep(v, 0, n) if (deg[v] == 0) { res[v] = -1; q.push(v); }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : rg[u]) if (res[v] == 0) {
            if (res[u] == -1) { res[v] = 1; q.push(v); }
            else if (--deg[v] == 0) { res[v] = -1; q.push(v); }
        }
    }
    return res;
}
```

Sprague–Grundy: for a sum of independent impartial games, `grundy(position) = mex{grundy(next)}`; the whole game is losing iff the xor of the components' Grundy values is 0.

- **Nim:** Grundy of a heap of size x is x → first player wins iff xor $\neq 0$.
- **Subtraction game** (remove 1..k): g(x) = x mod (k+1).
- **Splitting games** (a heap splits into two): g = mex over splits of g(a) xor g(b).
- **Staircase Nim:** only odd-indexed steps matter (xor of piles on odd steps).
- **Misère Nim:** same as Nim unless all heaps ≤ 1, then first player wins iff the number of heaps of size 1 is even.
- **Grundy values often become periodic**: compute the first few hundred by brute force and look for the period.

Two-player optimisation games (both maximise own score, take from ends of an array): `dp[l][r]` = best (my score − opponent's score) for the player to move on [l, r] = max(a_l − dp[l+1][r], a_r − dp[l][r−1]).

Strategy-stealing / symmetry: first player often wins by mirroring (central symmetry on a board, pairing moves). When stuck, brute force small n and print the win/lose table — a pattern (parity, mod k, powers of 2) usually appears.

5. Graphs and Trees

5.1 Shortest Paths and Graph Search

situation	algorithm	complexity
unweighted	BFS	O(n + m)
weights 0/1 (or 0/k)	0-1 BFS with deque	O(n + m)
non-negative weights	Dijkstra	O(m log n)
negative weights, detect negative cycle	Bellman–Ford	O(nm)
all pairs, n ≤ 500	Floyd–Warshall	$O(n^3)$
DAG (any weights)	DP in topological order	O(n + m)

5.1.1 Basics: bipartite check, cycle detection

```cpp
// [stress-tested] colour[v] in {0,1} if bipartite; returns false if an odd cycle exists
bool bipartite(const vector<vector<int>>& g, vector<int>& colour) {
    int n = sz(g); colour.assign(n, -1);
    rep(s, 0, n) if (colour[s] < 0) {
        colour[s] = 0; queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (colour[v] < 0) { colour[v] = colour[u] ^ 1; q.push(v); }
                else if (colour[v] == colour[u]) return false;
            }
        }
    }
    return true;
}
// [stress-tested] directed graph: returns the vertices of some cycle (in order), or {} if acyclic
vector<int> findCycle(const vector<vector<int>>& g) {
    int n = sz(g); vector<int> state(n, 0), par(n, -1), it(n, 0);
    rep(s, 0, n) if (!state[s]) {
        vector<int> st = {s}; state[s] = 1;
        while (!st.empty()) {
            int u = st.back();
            if (it[u] == sz(g[u])) { state[u] = 2; st.pop_back(); continue; }
            int v = g[u][it[u]++];
            if (state[v] == 0) { state[v] = 1; par[v] = u; st.push_back(v); }
            else if (state[v] == 1) {                // back edge u -> v closes a cycle
                vector<int> cyc = {v};
                for (int x = u; x != v; x = par[x]) cyc.push_back(x);
                reverse(cyc.begin() + 1, cyc.end());
                return cyc;
            }
        }
    }
    return {};
}
```

- Undirected cycle exists ⇔ some component has m_c ≥ n_c (or DSU `unite` fails).
- **Shortest cycle (girth)** in an unweighted undirected graph: BFS from every vertex; a non-tree edge (u, v) gives a cycle of length d[u] + d[v] + 1; O(nm).
- **k-th shortest walk** s → t (k small): Dijkstra where each vertex may be popped up to k times; the k-th pop of t is the answer. **Second shortest path** (strictly longer): keep `d1[v] < d2[v]` per vertex.

5.1.2 BFS on grids (template)

```cpp
// [stress-tested]
const int DR[] = {-1, 1, 0, 0}, DC[] = {0, 0, -1, 1};   // add diagonals for 8-neighbour
vector<vector<int>> gridBFS(const vector<string>& g, vector<pii> sources) {
    int R = sz(g), C = sz(g[0]);
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pii> q;
    for (auto [r, c] : sources) { d[r][c] = 0; q.push({r, c}); }   // multi-source
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        rep(k, 0, 4) {
            int nr = r + DR[k], nc = c + DC[k];
```

```
            if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1; q.push({nr, nc});
        }
    }
    return d;
}
```

- **State-space BFS:** node = (cell, extra state): direction, die orientation (ACPC 2016 *Dacey the Dice*), collected keys bitmask, remaining fuel. Encode as one integer `((r * C + c) * S + s)`.
- **Sliding moves** (ice: move until hitting an obstacle, ACPC 2024 *Slippery Floor*): neighbours of a cell are the 4 stopping cells; precompute or walk (each walk is O(R + C)).
- **Flood fill / count components**: BFS from every unvisited cell.

5.1.3 0-1 BFS

```
// sketch: adj[u] = {(v, w)} with w in {0, 1}
deque<int> dq; dist.assign(n, INF); dist[s] = 0; dq.push_back(s);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u]) if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        w ? dq.push_back(v) : dq.push_front(v);
    }
}
```

Typical: "minimum number of walls to break", "minimum number of direction changes" (state = (cell, direction), continuing straight costs 0, turning costs 1).

5.1.4 Dijkstra

```
// [stress-tested] adj[u] = {(v, w)}, w >= 0. par gives the shortest path tree.
vector<ll> dijkstra(const vector<vector<pair<int, ll>>>& adj, const vector<int>& srcs,
                    vector<int>* par = nullptr) {
    int n = sz(adj); vector<ll> d(n, INF);
    if (par) par->assign(n, -1);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    for (int s : srcs) { d[s] = 0; pq.push({0, s}); }
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;                       // stale entry
        for (auto [v, w] : adj[u]) if (du + w < d[v]) {
            d[v] = du + w; pq.push({d[v], v});
            if (par) (*par)[v] = u;
        }
    }
    return d;
}
```

- Dense graph ($n \le 5000$, $m \approx n^2$): $O(n^2)$ Dijkstra with a linear scan for the minimum is faster.
- **Count shortest paths:** when `du + w == d[v]` add `cnt[u]` to `cnt[v]` (process in pop order).
- **Edge (u,v,w) lies on some shortest s–t path** iff `ds[u] + w + dt[v] == ds[t]` (run from s and from t on the reversed graph).
- **Product graph Dijkstra** (ACPC 2023 *Horse Snatchers*: state = (town, still has horse)); (node, number of coupons used $\le$ k); (node, time mod k). Size n·S nodes.
- **Maximise the minimum along the path** (bottleneck/widest path): Dijkstra with `max-heap` on `min(cap so far, w)`, or Kruskal in decreasing order until s, t connected, or binary search + BFS.
- **Binary search + Dijkstra** (ACPC 2018 *Arachnophobia*): multi-source Dijkstra from all spiders gives `spiderDist[v]`; binary search D, run Dijkstra s→t using only vertices with `spiderDist ≥ D`, check `dist ≤ T`.
- Vertex weights: put the weight on incoming edges. Undirected: add both directions.
- Shortest path with "time-dependent" edges that are FIFO (arriving later never lets you leave earlier): Dijkstra still works with arrival-time relaxation.

Dijkstra on residues: which totals are representable (coin problem)

Coins c_1..c_k (unbounded), smallest coin `a`. `d[r]` = smallest representable total ≡ r (mod a). Then X is representable ⇔ `d[X mod a] ≤ X`. Largest non-representable = max(d[r]) − a (if gcd = 1). ACPC 2022 *Braaaains*.

```
// [stress-tested]
vector<ll> residueDist(const vector<ll>& coins) {
    ll a = *min_element(all(coins));
```

```
    vector<ll> d(a, INF); d[0] = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq; pq.push({0, 0});
    while (!pq.empty()) {
        auto [du, r] = pq.top(); pq.pop();
        if (du != d[r]) continue;
        for (ll c : coins) {
            int nr = (r + c) % a;
            if (du + c < d[nr]) { d[nr] = du + c; pq.push({d[nr], nr}); }
        }
    }
    return d;   // X representable iff d[X % a] <= X
}
```

5.1.5 Bellman–Ford and negative cycles

```
// [stress-tested] edges (u, v, w). Returns dist; dist = -INF for vertices reachable from a negative cycle.
vector<ll> bellmanFord(int n, const vector<array<ll, 3>>& edges, int s) {
    vector<ll> d(n, INF); d[s] = 0;
    rep(it, 0, n) for (auto [u, v, w] : edges)
        if (d[u] < INF && d[u] + w < d[v]) d[v] = (it == n - 1 ? -INF : d[u] + w);
    rep(it, 0, n) for (auto [u, v, w] : edges)          // propagate -INF
        if (d[u] == -INF && d[v] != -INF) d[v] = -INF;
    return d;
}
```

Marking only in the n-th round misses some affected vertices, so the second loop propagates `-INF` through everything reachable. **Difference constraints** `x_v - x_u ≤ w` → edge u → v with weight w; add a source with 0-edges to all; feasible iff no negative cycle; `x = dist` is a solution (maximal for each variable with x ≤ 0).

5.1.6 Floyd–Warshall

```
// [stress-tested] d[i][j] = edge weight or INF, d[i][i] = 0.
// Negative cycle through i iff d[i][i] < 0 afterwards.
void floyd(vector<vector<ll>>& d) {
    int n = sz(d);
    rep(k, 0, n) rep(i, 0, n) if (d[i][k] < INF) rep(j, 0, n)
        if (d[k][j] < INF) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

Variants (same triple loop): transitive closure (`reach[i][j] |= reach[i][k] && reach[k][j]`, or bitset rows), minimax path (`max` instead of `+`, `min` outer), counting paths in a DAG, shortest path when vertices are "unlocked" over time (process k in order of unlock time and answer queries in between).

5.2 MST, DAGs, Functional Graphs, Permutations

5.2.1 Minimum spanning tree

```
// [stress-tested] Kruskal: edges (w, u, v). Returns total weight (n-1 edges if connected). Needs DSU (§3.1).
struct DSUk { vector<int> p; DSUk(int n) : p(n, -1) {}
    int f(int x) { return p[x] < 0 ? x : p[x] = f(p[x]); }
    bool u(int a, int b) {
        a = f(a); b = f(b); if (a == b) return 0;
        if (p[a] > p[b]) swap(a, b);
        p[a] += p[b]; p[b] = a; return 1;
    }
};
ll kruskal(int n, vector<tuple<ll, int, int>> edges, vector<tuple<ll, int, int>>* used = nullptr) {
    sort(all(edges)); DSUk d(n); ll tot = 0;
    for (auto& [w, a, b] : edges) if (d.u(a, b)) { tot += w; if (used) used->push_back({w, a, b}); }
    return tot;
}
```

- **Dense graphs** (complete graph on points, n ≤ 5000): Prim $O(n^2)$ with an array, no heap.
- **Cycle property:** the heaviest edge on any cycle is not needed. **Cut property:** the lightest edge across any cut is in some MST.
- MST minimises the **maximum** edge on paths too: the minimax path between u and v is the path in the MST. Max spanning tree for maximin paths.
- Second-best MST: for each non-tree edge (u, v, w), replace the max tree edge on path(u, v) (binary lifting with max, §5.4).
- **Min ratio spanning tree** Σcost/Σgain (ACPC 2023 *Buildin' Roads*): binary search λ, MST on `cost - λ·gain`, feasible iff total ≤ 0 (§2.1). Doubles, 60–100 iterations.
- Steiner tree on k ≤ 10 terminals: dp[mask][v] + Dijkstra per mask ($O(3^k n + 2^k m \log n)$).

- Count spanning trees: Kirchhoff (determinant of Laplacian minor, §7.4).

5.2.2 Topological sort and DAG DP

```
// [stress-tested] Kahn. Returns empty vector if there is a cycle.
// Min-heap instead of queue => lexicographically smallest order.
vector<int> topo(const vector<vector<int>>& g) {
    int n = sz(g); vector<int> indeg(n), order;
    rep(u, 0, n) for (int v : g[u]) indeg[v]++;
    queue<int> q; rep(u, 0, n) if (!indeg[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return sz(order) == n ? order : vector<int>();
}
```

- Longest path / count paths / probability of reaching: DP in topo order. Number of paths can be huge → mod or doubles.
- Unique topological order ⇔ queue size is always 1 ⇔ Hamiltonian path exists in the DAG.
- **Min path cover** of a DAG (vertices) = n − max matching (§6.2); **with path overlap allowed** = same on the transitive closure. **Cover all edges** (ACPC 2022 *Escape Routes*): min flow with lower bounds (§6.1).
- **Dilworth:** minimum number of chains covering a poset = size of the largest antichain. For 2D dominance (pairs sorted by x, chains need y non-decreasing): min chains = longest strictly decreasing subsequence of y.
- Directed graph with cycles → condense SCCs first (§5.3), then DAG DP.

5.2.3 Functional graphs (each node has exactly one outgoing edge)

Structure: each component is one cycle with in-trees hanging off it.

```
// [stress-tested] cycId[v] = id of the cycle containing v (-1 if v is on a tail); cycLen[id] = its length
void functionalCycles(const vector<int>& f, vector<int>& cycId, vector<int>& cycLen) {
    int n = sz(f); vector<int> state(n, 0); cycId.assign(n, -1); cycLen.clear();
    rep(s, 0, n) if (!state[s]) {
        vector<int> path; int v = s;
        while (!state[v]) { state[v] = 1; path.push_back(v); v = f[v]; }   // 1 = on current path
        if (state[v] == 1) {                                               // found a new cycle
            int id = sz(cycLen), len = 0;
            for (int i = sz(path) - 1; ; --i) { cycId[path[i]] = id; ++len; if (path[i] == v) break; }
            cycLen.push_back(len);
        }
        for (int x : path) state[x] = 2;
    }
}
```

- **k-th successor** for huge k: binary lifting `up[j][v] = up[j-1][up[j-1][v]]`, O(n log k) build, O(log k) query. Or walk to the cycle and use k mod cycle length.
- Floyd's tortoise–hare finds a cycle in an implicit sequence x → f(x) with O(1) memory (e.g. pseudo-random generators, repeated states of a simulation: **detect the period, then skip ahead**).
- "Simulate 10^{18} steps of a deterministic process" ⇒ the state repeats; store first-seen time of each state in a map, jump by the period.

5.2.4 Permutations

- Decompose into cycles: `i → p[i]`. Minimum swaps (any two elements) to sort = n − #cycles. Adjacent swaps = #inversions (BIT, §3.2).
- p^k (apply k times): for each cycle of length L, shift by k mod L. Order of p = lcm of cycle lengths.
- Square root of a permutation, conjugacy: cycles of equal length pair up.
- Swaps restricted to graph edges (ACPC 2021 *Swapping Places*): within each connected component any permutation is achievable; construct by fixing leaves of a spanning tree first (route the right value to a leaf along the tree path, then delete the leaf) — $O(n^2)$ swaps.
- Rotations of a permutation maximising fixed points (ACPC 2022 *Maximum Fix*): element a_i is fixed by exactly one shift $k = (i - a_i) \bmod n$; count votes.

5.3 SCC, 2-SAT, Bridges, Articulation Points, Euler Paths

5.3.1 Strongly connected components (Tarjan)

```cpp
// [stress-tested] comp[v] in [0, ncomp). Components are numbered in REVERSE topological order
// (comp 0 is a sink): edge u->v between components implies comp[u] >= comp[v].
struct SCC {
    int n, timer = 0, ncomp = 0;
    vector<vector<int>> g; vector<int> tin, low, comp, st; vector<char> on;
    SCC(const vector<vector<int>>& g) : n(sz(g)), g(g), tin(n, -1), low(n), comp(n, -1), on(n) {
        rep(v, 0, n) if (tin[v] == -1) dfs(v);
    }
    void dfs(int v) {
        tin[v] = low[v] = timer++; st.push_back(v); on[v] = 1;
        for (int u : g[v]) {
            if (tin[u] == -1) { dfs(u); low[v] = min(low[v], low[u]); }
            else if (on[u]) low[v] = min(low[v], tin[u]);
        }
        if (low[v] == tin[v]) {
            while (true) {
                int u = st.back(); st.pop_back(); on[u] = 0; comp[u] = ncomp;
                if (u == v) break;
            }
            ++ncomp;
        }
    }
};
```

Condensation DAG: edge comp[u] → comp[v] for every edge with different comps. Typical follow-ups: min edges to add to make strongly connected = max(#sources, #sinks) (0 if one SCC); nodes reachable from all others = the unique sink SCC if exactly one sink.

Recursion depth = n; for n ≥ $5 \cdot 10^5$ on a path consider an iterative version or Kosaraju with explicit stacks.

5.3.2 2-SAT

```cpp
// [stress-tested] variables 0..n-1. lit(x, true) = x, lit(x, false) = not x.
struct TwoSat {
    int n; vector<vector<int>> g;
    TwoSat(int n) : n(n), g(2 * n) {}
    static int lit(int x, bool val) { return 2 * x + !val; }
    void implies(int a, int b) { g[a].push_back(b); g[b ^ 1].push_back(a ^ 1); }  // a -> b (and !b -> !a)
    void either(int a, int b) { implies(a ^ 1, b); }                              // a or b
    void setTrue(int a) { implies(a ^ 1, a); }
    void atMostOne(int a, int b) { implies(a, b ^ 1); }                           // not (a and b)
    void equal(int a, int b) { implies(a, b); implies(b, a); }
    bool solve(vector<bool>& val) {
        SCC s(g); val.assign(n, false);
        rep(x, 0, n) {
            if (s.comp[2 * x] == s.comp[2 * x + 1]) return false;
            val[x] = s.comp[2 * x] < s.comp[2 * x + 1];   // later in topo order = true
        }
        return true;
    }
};
```

Modelling: each boolean choice is a variable; constraints must be clauses of **two** literals. "Exactly one of a, b" = either(a, b) + atMostOne(a, b). "At most one among k literals" → prefix-variable encoding (O(k) extra variables). Often combined with binary search on a threshold (e.g. maximise min distance between chosen points: pairs closer than D can't both be chosen).

5.3.3 Bridges and articulation points

```cpp
// [stress-tested] undirected multigraph; edges given as (u, v). Handles parallel edges via edge ids.
struct BridgeCut {
    int n, timer = 0; vector<vector<pii>> g; vector<int> tin, low;
    vector<char> isBridge, isCut;
    BridgeCut(int n, const vector<pii>& edges)
        : n(n), g(n), tin(n, -1), low(n), isBridge(sz(edges)), isCut(n) {
        rep(i, 0, sz(edges)) { auto [u, v] = edges[i]; g[u].push_back({v, i}); g[v].push_back({u, i}); }
        rep(v, 0, n) if (tin[v] == -1) dfs(v, -1);
    }
    void dfs(int v, int pe) {
        tin[v] = low[v] = timer++; int children = 0;
        for (auto [u, id] : g[v]) {
            if (id == pe) continue;
```

```
            if (tin[u] != -1) { low[v] = min(low[v], tin[u]); continue; }
            dfs(u, id); low[v] = min(low[v], low[u]); ++children;
            if (low[u] > tin[v]) isBridge[id] = 1;
            if (low[u] >= tin[v] && pe != -1) isCut[v] = 1;
        }
        if (pe == -1 && children > 1) isCut[v] = 1;
    }
};
```

- **2-edge-connected components:** delete bridges, take components. Contracting them gives the **bridge tree** (a tree!) — then path questions become tree questions ("must every u–v path use edge e" ⇔ e is a bridge on the bridge-tree path).
- **Biconnected components / block-cut tree:** push edges on a stack in the DFS; when `low[u] >= tin[v]` pop edges up to (v, u) → one block. Block-cut tree: bipartite tree of blocks and cut vertices; "vertices that lie on every u–v path" = cut vertices on the block-cut tree path.
- Edge in a cycle ⇔ not a bridge. Graph stays connected after removing any one vertex ⇔ no articulation points.

5.3.4 Euler path / circuit (Hierholzer)

Existence: **undirected** connected (ignoring isolated vertices) with 0 odd-degree vertices (circuit) or exactly 2 (path, start at an odd one). **Directed**: every vertex in = out (circuit), or exactly one with out − in = 1 (start) and one with in − out = 1 (end).

```
// [stress-tested] returns vertex sequence of length m+1 using every edge once, or {} if impossible
// from s. directed=false: each edge usable in both directions.
vector<int> eulerPath(int n, const vector<pii>& edges, int s, bool directed) {
    int m = sz(edges); vector<vector<pii>> g(n);
    rep(i, 0, m) { auto [u, v] = edges[i]; g[u].push_back({v, i}); if (!directed) g[v].push_back({u, i}); }
    vector<int> ptr(n, 0), path, st = {s}; vector<char> used(m, 0);
    while (!st.empty()) {
        int v = st.back();
        while (ptr[v] < sz(g[v]) && used[g[v][ptr[v]].second]) ++ptr[v];
        if (ptr[v] == sz(g[v])) { path.push_back(v); st.pop_back(); }
        else { auto [u, id] = g[v][ptr[v]]; used[id] = 1; st.push_back(u); }
    }
    if (sz(path) != m + 1) return {};
    reverse(all(path));
    return path;
}
```

Check the degree conditions yourself first (and pick s accordingly): if they fail, the routine can still return a sequence of length m+1 that is not a valid trail. The length check only catches edges unreachable from s. Uses: **De Bruijn sequence** (nodes = (k−1)-length strings, edges = k-length strings, Euler circuit), reconstructing a string from all its k-mers, "draw without lifting the pen", domino chains.

5.4 Trees: LCA, Euler Tour, HLD, Centroid Decomposition

5.4.1 LCA with binary lifting (+ distances, k-th ancestor, path aggregates)

```
// [stress-tested] weighted tree, iterative DFS (safe for deep trees).
struct LCA {
    int n, LOG; vector<vector<int>> up; vector<int> dep, tin, tout; vector<ll> dist;
    LCA(const vector<vector<pair<int, ll>>>& g, int root = 0) : n(sz(g)) {
        LOG = 1; while ((1 << LOG) < n) ++LOG;
        up.assign(LOG, vector<int>(n, root)); dep.assign(n, 0); tin.assign(n, 0); tout.assign(n, 0);
        dist.assign(n, 0);
        int timer = 0; vector<pair<int, int>> st = {{root, 0}};   // (vertex, next child index)
        tin[root] = timer++;
        while (!st.empty()) {
            auto& [v, i] = st.back();
            if (i < sz(g[v])) {
                auto [u, w] = g[v][i++];
                if (u == up[0][v] && v != root) continue;
                up[0][u] = v; dep[u] = dep[v] + 1; dist[u] = dist[v] + w; tin[u] = timer++;
                st.push_back({u, 0});
            } else { tout[v] = timer++; st.pop_back(); }
        }
        rep(k, 1, LOG) rep(v, 0, n) up[k][v] = up[k - 1][up[k - 1][v]];
    }
    bool isAnc(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
    int lca(int u, int v) const {
        if (isAnc(u, v)) return u;
        if (isAnc(v, u)) return v;
        for (int k = LOG - 1; k >= 0; --k) if (!isAnc(up[k][u], v)) u = up[k][u];
        return up[0][u];
```

```
    }
    ll distance(int u, int v) const { return dist[u] + dist[v] - 2 * dist[lca(u, v)]; }
    int kthAncestor(int v, int k) const {   // k <= dep[v]
        rep(j, 0, LOG) if (k >> j & 1) v = up[j][v];
        return v;
    }
    // k-th vertex on the path u -> v (0-indexed), assumes k <= #edges on the path
    int kthOnPath(int u, int v, int k) const {
        int w = lca(u, v), du = dep[u] - dep[w], dv = dep[v] - dep[w];
        return k <= du ? kthAncestor(u, k) : kthAncestor(v, du + dv - k);
    }
};
```

- **Path aggregates (max edge on path, min, sum):** add a table `mx[k][v]` = max edge weight on the 2^k jump; combine while lifting both endpoints to the LCA.
- **Path sums with point updates:** Euler tour + BIT: value on vertex v contributes to the whole subtree, so add at `tin[v]`, subtract at `tout[v]`; root-to-v sum = prefix query at `tin[v]`; path(u, v) = s(u) + s(v) − 2 s(lca) + val(lca).
- **Subtree queries:** with pre-order indices `tin` (counting only entries), subtree of v = `[tin[v], tin[v] + size[v] - 1]` → BIT/segment tree.
- **Distance between nodes in unweighted tree, many queries:** depth[u] + depth[v] − 2·depth[lca].
- **Ancestor check** without LCA: tin/tout as above (O(1)).
- **Diameter endpoints:** farthest from any node is an endpoint a; farthest from a is b. Every vertex's eccentricity = max(dist(v, a), dist(v, b)). **Centre** = middle of the diameter path.
- **Virtual (auxiliary) tree** for queries on k marked nodes: sort by tin, add LCAs of adjacent pairs, build with a stack → O(k log n) nodes; then run the DP only on that tree.

5.4.2 Heavy-light decomposition

Any root-to-leaf path crosses O(log n) heavy chains; positions on a chain are contiguous, so path queries become O(log n) segment-tree range queries (O(log² n) total).

```
// [stress-tested] call path(u, v, f): f(l, r) on inclusive position ranges. Use with any segment tree
// indexed by pos[]. Edge values: store the edge weight at pos[child] and pass skipLca = true.
struct HLD {
    int n; vector<int> par, heavy, dep, head, pos, sub;
    HLD(const vector<vector<int>>& g, int root = 0)
        : n(sz(g)), par(n, -1), heavy(n, -1), dep(n), head(n), pos(n), sub(n, 1) {
        vector<int> order; order.reserve(n); order.push_back(root);
        rep(i, 0, n) {
            int v = order[i];
            for (int u : g[v]) if (u != par[v]) { par[u] = v; dep[u] = dep[v] + 1; order.push_back(u); }
        }
        for (int i = n - 1; i > 0; --i) { int v = order[i]; sub[par[v]] += sub[v]; }
        rep(i, 0, n) { int v = order[i]; int best = 0;
            for (int u : g[v]) if (u != par[v] && sub[u] > best) { best = sub[u]; heavy[v] = u; } }
        int cur = 0; vector<int> st = {root}; head[root] = root;
        while (!st.empty()) {                       // pre-order DFS, heavy child visited first
            int v = st.back(); st.pop_back(); pos[v] = cur++;
            for (int u : g[v]) if (u != par[v] && u != heavy[v]) { head[u] = u; st.push_back(u); }
            if (heavy[v] != -1) { head[heavy[v]] = head[v]; st.push_back(heavy[v]); }
        }
    }
    template <class F> void path(int u, int v, F f, bool skipLca = false) {
        for (; head[u] != head[v]; v = par[head[v]]) {
            if (dep[head[u]] > dep[head[v]]) swap(u, v);
            f(pos[head[v]], pos[v]);
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (pos[u] + skipLca <= pos[v]) f(pos[u] + skipLca, pos[v]);
    }
    pair<int, int> subtree(int v) { return {pos[v], pos[v] + sub[v] - 1}; }
};
```

Positions are a DFS pre-order with the heavy child first, so both chains and subtrees are contiguous: `subtree(v)` and `path(u, v)` can be mixed on one segment tree.

5.4.3 Centroid decomposition

Centroid: vertex whose removal leaves components of size ≤ n/2. Recursively decompose: depth O(log n); every path in the tree passes through the centroid of the smallest decomposition component containing both ends.

```cpp
// [stress-tested] cpar[v] = parent in the centroid tree (-1 for the root centroid)
struct Centroid {
    vector<vector<int>> g; vector<int> sub, cpar; vector<char> removed;
    Centroid(const vector<vector<int>>& g)
        : g(g), sub(sz(g)), cpar(sz(g), -1), removed(sz(g)) { build(0, -1); }
    int calcSize(int v, int p) {
        sub[v] = 1;
        for (int u : g[v]) if (u != p && !removed[u]) sub[v] += calcSize(u, v);
        return sub[v];
    }
    int find(int v, int p, int tot) {
        for (int u : g[v]) if (u != p && !removed[u] && sub[u] * 2 > tot) return find(u, v, tot);
        return v;
    }
    void build(int v, int p) {
        int c = find(v, -1, calcSize(v, -1));
        cpar[c] = p; removed[c] = 1;
        // ---- process all paths through c here (DFS from c over non-removed vertices) ----
        for (int u : g[c]) if (!removed[u]) build(u, c);
    }
};
```

Typical uses: - **Count paths with length ≤ K / exactly K:** at each centroid, collect depths from each child subtree; count pairs across different subtrees (sort + two pointers, subtract pairs within the same child). - **Nearest marked vertex queries:** `best[c]` = min distance from centroid c to a marked vertex in its component; update/query walks up `cpar` (O(log n) ancestors) using `dist(v, c)` (precomputed per level or via LCA). - Recursion depth of `calcSize` can be n on a path graph at the top level; fine for $n \le 2 \cdot 10^5$ on Kattis, otherwise make it iterative.

6. Flows, Matching, Assignment

6.1 Max Flow, Min Cut, and Modelling

Signals: "each X used at most once/at most c times", "assign", "disjoint paths", "minimum set of things to remove to disconnect", "choose a subset with dependencies maximising profit", "minimum number of paths/chains covering …". Small n (≤ 500 nodes, ≤ 10^4–10^5 edges) is a strong hint.

6.1.1 Dinic

```cpp
// [stress-tested] O(V^2 E) worst, much faster in practice; unit-capacity graphs O(E sqrt V).
struct Dinic {
    struct E { int to; ll cap; };
    vector<E> e; vector<vector<int>> g; vector<int> level, it;
    Dinic(int n) : g(n), level(n), it(n) {}
    int addEdge(int u, int v, ll cap, ll rcap = 0) {        // rcap = cap for undirected edges
        g[u].push_back(sz(e)); e.push_back({v, cap});
        g[v].push_back(sz(e)); e.push_back({u, rcap});
        return sz(e) - 2;                                    // id; flow on it = original cap - e[id].cap
    }
    bool bfs(int s, int t) {
        fill(all(level), -1); level[s] = 0; queue<int> q; q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : g[v]) if (e[id].cap > 0 && level[e[id].to] < 0) {
                level[e[id].to] = level[v] + 1; q.push(e[id].to);
            }
        }
        return level[t] >= 0;
    }
    ll dfs(int v, int t, ll f) {
        if (v == t) return f;
        ll pushed = 0;
        for (int& i = it[v]; i < sz(g[v]); ++i) {
            int id = g[v][i], to = e[id].to;
            if (e[id].cap <= 0 || level[to] != level[v] + 1) continue;
            ll d = dfs(to, t, min(f - pushed, e[id].cap));
            if (d > 0) {
                e[id].cap -= d; e[id ^ 1].cap += d; pushed += d;
                if (pushed == f) return pushed;
            }
        }
        return pushed;
    }
    ll maxflow(int s, int t, ll limit = INF) {
        ll flow = 0;
        while (flow < limit && bfs(s, t)) {
            fill(all(it), 0);
            flow += dfs(s, t, limit - flow);
        }
        return flow;
    }
    // after maxflow: S side of a minimum cut = vertices reachable from s in the residual graph
    bool inS(int v) const { return level[v] >= 0; }
};
```

- **Min cut edges:** edges u → v with `inS(u) && !inS(v)` (original edges only). Max flow = min cut capacity.
- Reuse: to rerun with modified capacities, rebuild the graph (or reset caps from a saved copy).
- Capacities as `ll`; use `INF` for "uncuttable" edges but make sure the answer can't involve them (otherwise "impossible").

6.1.2 Modelling cookbook

need	construction
several sources/sinks	super source S → each source (cap = supply), each sink → super sink T
vertex capacity c	split v into v_in → v_out with cap c; edges enter v_in, leave v_out
undirected edge cap c	one edge with cap c in both directions (`addEdge(u, v, c, c)`)
k edge-disjoint s–t paths	unit capacities, flow ≥ k (Menger); vertex-disjoint: split vertices with cap 1
bipartite matching	S → left (1), left → right (1), right → T (1)
each worker ≤ a_i tasks, task needs b_j workers	S → worker (a_i), worker → task (1), task → T (b_j); feasible iff flow = Σb_j

need	construction
minimise the maximum load	binary search the cap L on S → worker edges; feasible iff max flow = total
time steps (people moving over time)	time-expanded graph: node (v, t) for every t, edges (v,t) → (u,t+1)
pick at most one per row & column (grid)	bipartite rows ↔ columns, cell = edge
choose ≥/≤ constraints from both sides	lower bounds (below)

Project selection / maximum closure (min cut)

Projects with profit p_i (may be negative), dependencies "taking i requires taking j". Build: S → i with cap p_i for p_i > 0; i → T with cap −p_i for p_i < 0; i → j with cap INF for each dependency. **Max profit = Σ positive p_i − maxflow.** Chosen set = S side.

General "two-label" min cut (each item either A or B): cost a_i if item is in B (edge S → i, cap a_i), cost b_i if in A (edge i → T, cap b_i), penalty c for i in A and j in B (edge i → j, cap c). Works when penalties are non-negative and pairwise ("submodular"). Image segmentation, "assign each student to one of two buildings with friendship penalties".

Lower bounds (circulations with demands)

Edge u → v with flow in [l, c]: add edge with cap c − l, and record `ex[v] += l`, `ex[u] -= l`. Then S' → v (cap ex[v]) for ex[v] > 0, v → T' (cap −ex[v]) for ex[v] < 0. A feasible circulation exists iff maxflow(S', T') saturates all S' edges. For an s–t flow add edge t → s with cap INF first.

- **Max flow with lower bounds:** after feasibility, F = flow on the t → s edge; remove that edge (set both caps to 0); answer = F + maxflow(s, t) on the same residual graph (S', T' edges are saturated and can't be used).
- **Min flow with lower bounds:** feasible value F = flow on the t → s edge; remove that edge (set both caps to 0); answer = F − maxflow(t, s).

```cpp
// [stress-tested] Minimum number of s->t paths in a DAG covering every edge at least once
// (ACPC 2022 Escape Routes). Every edge: lower bound 1, capacity INF. Assumes each edge lies on
// some s->t path. Generalises: give each edge its own [l, c].
ll minFlowCoverEdges(int n, const vector<pii>& edges, int s, int t) {
    int S = n, T = n + 1; Dinic d(n + 2); vector<ll> ex(n, 0);
    const ll BIG = 1e12;
    for (auto [u, v] : edges) { d.addEdge(u, v, BIG - 1); ex[v] += 1; ex[u] -= 1; }
    int ts = d.addEdge(t, s, BIG);
    ll need = 0;
    rep(v, 0, n) {
        if (ex[v] > 0) { d.addEdge(S, v, ex[v]); need += ex[v]; }
        else if (ex[v] < 0) d.addEdge(v, T, -ex[v]);
    }
    if (d.maxflow(S, T) != need) return -1;              // infeasible
    ll F = BIG - d.e[ts].cap;                             // current s->t flow value
    d.e[ts].cap = d.e[ts ^ 1].cap = 0;
    return F - d.maxflow(t, s);
}
```

6.1.3 Flow theorems worth knowing

- **Max-flow = min-cut.** Integral capacities ⇒ integral max flow (so flows decompose into paths: each unit = one path).
- **Menger:** max #edge-disjoint s–t paths = min #edges to delete to disconnect s from t (same for vertices with splitting).
- **Hall:** a bipartite graph has a matching saturating the left side iff every left subset X has |N(X)| ≥ |X|. Deficiency version: max matching = |L| − max_X (|X| − |N(X)|).
- Max flow in a graph where all capacities are multiplied by k is k times — so "fractional" answers often come from binary search on a ratio, not from fractional flows.
- **Gomory–Hu tree** (all-pairs min cut, n − 1 max flows) exists if you need min cut between many pairs in an undirected graph.

6.2 Bipartite Matching and Covers

6.2.1 Hopcroft–Karp (O(E √V))

```cpp
// [stress-tested] left 0..n-1, right 0..m-1
struct HopcroftKarp {
    int n, m; vector<vector<int>> g; vector<int> mL, mR, dist;
    HopcroftKarp(int n, int m) : n(n), m(m), g(n), mL(n, -1), mR(m, -1), dist(n) {}
    void addEdge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q; bool found = false;
        rep(u, 0, n) { if (mL[u] < 0) { dist[u] = 0; q.push(u); } else dist[u] = -1; }
        while (!q.empty()) {
            int u = q.front(); q.pop();
```

```
            for (int v : g[u]) {
                int w = mR[v];
                if (w < 0) found = true;
                else if (dist[w] < 0) { dist[w] = dist[u] + 1; q.push(w); }
            }
        }
        return found;
    }
    bool dfs(int u) {
        for (int v : g[u]) {
            int w = mR[v];
            if (w < 0 || (dist[w] == dist[u] + 1 && dfs(w))) { mL[u] = v; mR[v] = u; return true; }
        }
        dist[u] = -1;
        return false;
    }
    int maxMatching() {
        int res = 0;
        while (bfs()) rep(u, 0, n) if (mL[u] < 0 && dfs(u)) ++res;
        return res;
    }
    // König: minimum vertex cover (call after maxMatching). Returns (left cover, right cover).
    pair<vector<int>, vector<int>> minVertexCover() {
        vector<char> visL(n, 0), visR(m, 0); queue<int> q;
        rep(u, 0, n) if (mL[u] < 0) { visL[u] = 1; q.push(u); }
        while (!q.empty()) {                                    // alternating BFS from free left
            int u = q.front(); q.pop();
            for (int v : g[u]) if (!visR[v] && mL[u] != v) {
                visR[v] = 1;
                if (mR[v] >= 0 && !visL[mR[v]]) { visL[mR[v]] = 1; q.push(mR[v]); }
            }
        }
        vector<int> L, R;
        rep(u, 0, n) if (!visL[u]) L.push_back(u);
        rep(v, 0, m) if (visR[v]) R.push_back(v);
        return {L, R};
    }
};
```

For tiny graphs, Kuhn's algorithm (DFS augment from each left vertex, O(VE)) is shorter; Hopcroft–Karp is safe up to $\sim 10^5$ vertices / 10^6 edges.

6.2.2 Facts (bipartite graphs, V = L ∪ R)

quantity	equals
min vertex cover	max matching (König)
max independent set	V − max matching (complement of a min vertex cover)
min edge cover (no isolated vertices)	V − max matching
min path cover of a DAG (vertex-disjoint)	n − max matching in split graph (u_out → v_in for each edge)
min path cover of a DAG (paths may share vertices)	same on the transitive closure
min chains covering a poset	max antichain (Dilworth) = min path cover on the closure
perfect matching exists	Hall's condition / max matching = n

- **Grid problems** (dominoes, placing non-attacking pieces, covering cells): colour cells like a chessboard → bipartite. Max non-attacking rooks on a board with holes: rows ↔ columns.
- **Max independent set in a bipartite conflict graph** = choose max items with no conflicting pair (e.g. points where conflicts only between "even" and "odd" sums).
- **Min vertex cover in a bipartite graph** = "minimum number of lines (rows/cols) to cover all marked cells".
- **Maximum weight** bipartite matching → Hungarian / min cost flow (§6.3).
- **General (non-bipartite) graphs:** max matching needs Edmonds' blossom ($O(V^3)$); for $n \le 20$–25, bitmask DP over matched vertices; for trees, greedy from leaves.
- **Path cover reconstruction**: matched edge u_out → v_in means "v follows u in its path"; paths start at vertices whose v_in is unmatched.

6.2.3 Stable marriage (Gale–Shapley)

```
// prefM[i] = men's preference lists (women ids, best first), prefW[j][i] = rank of man i for woman j
vector<int> galeShapley(const vector<vector<int>>& prefM, const vector<vector<int>>& rankW) {
    int n = sz(prefM); vector<int> wife(n, -1), husband(n, -1), nxt(n, 0); queue<int> freeMen;
```

```
    rep(i, 0, n) freeMen.push(i);
    while (!freeMen.empty()) {
        int m = freeMen.front(); freeMen.pop();
        int w = prefM[m][nxt[m]++];
        if (husband[w] < 0) { husband[w] = m; wife[m] = w; }
        else if (rankW[w][m] < rankW[w][husband[w]]) {
            wife[husband[w]] = -1; freeMen.push(husband[w]); husband[w] = m; wife[m] = w;
        } else freeMen.push(m);
    }
    return wife;   // man-optimal stable matching
}
```

6.3 Min-Cost Flow and the Assignment Problem

6.3.1 Hungarian algorithm (n ≤ m, O(n²m))

```
// [stress-tested] a = n x m cost matrix (n <= m). Minimum total cost assigning every row to a
// distinct column. Returns {cost, match} with match[row] = column. Maximise: negate costs.
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a) {
    int n = sz(a), m = sz(a[0]);
    const ll BIG = LLONG_MAX / 4;
    vector<ll> u(n + 1), v(m + 1); vector<int> p(m + 1), way(m + 1);
    rep(i, 1, n + 1) {
        p[0] = i; int j0 = 0;
        vector<ll> minv(m + 1, BIG); vector<char> used(m + 1, 0);
        do {
            used[j0] = 1; int i0 = p[j0], j1 = 0; ll delta = BIG;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
                if (minv[j] < delta) { delta = minv[j]; j1 = j; }
            }
            rep(j, 0, m + 1) if (used[j]) { u[p[j]] += delta; v[j] -= delta; } else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do { int j1 = way[j0]; p[j0] = p[j1]; j0 = j1; } while (j0);
    }
    vector<int> match(n);
    rep(j, 1, m + 1) if (p[j]) match[p[j] - 1] = j - 1;
    return {-v[0], match};
}
```

- Forbidden pairs: cost = large constant `C` (e.g. 1e12, but ≤ BIG/n); if the result ≥ C, no valid assignment exists.
- n > m: transpose. "Not everyone must be matched": add dummy columns with cost 0 (or the cost of staying unmatched).
- 500 × 500 runs in well under a second.

Worked pattern: enumerate the maximum, then assign (ACPC 2018 *Dog Trouble*)

Minimise `Σ (t − t_i)` where t = max assigned time ⇒ `N·t − Σ t_i`. Enumerate t over all distinct values (≤ 200); allow only pairs with t_ij ≤ t; maximise Σ t_ij with Hungarian (cost −t_ij, forbidden = C). Answer = min over t of `N·t − maxSum(t)`. Using a t larger than the true max only overestimates, so taking the min over all t is exact. 200 × O(50³) is tiny.

General lesson: **an objective mixing "max" and "sum" → fix the max by enumeration or binary search, then optimise the sum.**

6.3.2 Min-cost max-flow (successive shortest paths with SPFA)

```
// [stress-tested] handles negative edge costs (no negative cycles). O(F · V · E) worst case.
struct MCMF {
    struct E { int to; ll cap, cost; };
    vector<E> e; vector<vector<int>> g; vector<ll> dist; vector<int> pe; vector<char> inq;
    MCMF(int n) : g(n), dist(n), pe(n), inq(n) {}
    int addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back(sz(e)); e.push_back({v, cap, cost});
        g[v].push_back(sz(e)); e.push_back({u, 0, -cost});
        return sz(e) - 2;
    }
    // returns {flow, cost}. stopIfPositive: stop when the cheapest path costs >= 0
    // (gives min-cost *any-amount* flow, i.e. max profit when costs are negated profits)
    pair<ll, ll> run(int s, int t, ll maxf = INF, bool stopIfPositive = false) {
        ll flow = 0, cost = 0;
        while (flow < maxf) {
            fill(all(dist), INF); dist[s] = 0; queue<int> q; q.push(s); inq[s] = 1;
            while (!q.empty()) {
                int u = q.front(); q.pop(); inq[u] = 0;
```

```
                for (int id : g[u]) {
                    const E& ed = e[id];
                    if (ed.cap > 0 && dist[u] + ed.cost < dist[ed.to]) {
                        dist[ed.to] = dist[u] + ed.cost; pe[ed.to] = id;
                        if (!inq[ed.to]) { inq[ed.to] = 1; q.push(ed.to); }
                    }
                }
            }
            if (dist[t] == INF || (stopIfPositive && dist[t] >= 0)) break;
            ll f = maxf - flow;
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) f = min(f, e[pe[v]].cap);
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) { e[pe[v]].cap -= f; e[pe[v] ^ 1].cap += f; }
            flow += f; cost += f * dist[t];
        }
        return {flow, cost};
    }
};
```

Modelling with costs: - **Assignment with capacities** (worker i can do up to c_i jobs): S → worker (c_i, 0), worker → job (1, cost), job → T (1, 0). - **k disjoint paths of minimum total length**: unit capacities, run with `maxf = k`. - **Choose at most k items with conflicts along a line** (k non-overlapping interval families, max weight): chain i → i+1 with (k, 0), interval [l, r) as edge l → r with (1, −w). - **Max-weight matching where not all must be matched:** costs = −weight, `stopIfPositive = true`. - Convex costs (cost of the j-th unit is increasing): parallel edges with capacity 1 and costs c_1 ≤ c_2 ≤ …. - If it's too slow (large flow), use Dijkstra with Johnson potentials instead of SPFA, or capacity scaling — rarely needed at ACPC sizes.

7. Math

7.1 Modular Arithmetic, GCD, CRT, Binomials

7.1.1 Basics

```
// [stress-tested]
ll power(ll b, ll e, ll m) {                    // b^e mod m, m < 2^62 safe via __int128
    ll r = 1 % m; b %= m; if (b < 0) b += m;
    for (; e > 0; e >>= 1, b = (__int128)b * b % m) if (e & 1) r = (__int128)r * b % m;
    return r;
}
ll inv(ll a, ll p) { return power(a, p - 2, p); } // p prime, a not divisible by p
ll extgcd(ll a, ll b, ll& x, ll& y) {           // a*x + b*y = g = gcd(a, b)
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, g = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}
ll invGeneral(ll a, ll m) {                     // inverse mod any m, or -1 if gcd(a, m) != 1
    ll x, y; ll g = extgcd(((a % m) + m) % m, m, x, y);
    return g != 1 ? -1 : ((x % m) + m) % m;
}
```

- Always reduce after every `+`, `-` (`((a - b + MOD) % MOD)`), `*` (cast to `ll` first: `1LL * a * b % MOD`).
- Division by d ⇒ multiply by `inv(d)`; valid only if gcd(d, MOD) = 1. **Don't divide before taking mod in a formula unless d divides exactly** (e.g. n(n+1)/2 mod p: compute with the inverse of 2, or divide n or n+1 first).
- **Exponent reduction** (Fermat/Euler): a^e mod p = a ^ (e mod (p−1)) mod p when gcd(a, p) = 1. For towers `a^(b^c) mod m` use φ(m): $a^e \equiv$ a ^ (e mod φ(m) + φ(m)) for e ≥ log2 m (generalised Euler).
- Inverses of 1..n in O(n): `inv[1] = 1; inv[i] = (MOD - MOD/i) * inv[MOD % i] % MOD`.
- `mod 10^9 + 9`, `998244353`, `10^6` (not prime! no inverses) — read the statement carefully.

7.1.2 Factorials and binomials

```
// [stress-tested]
struct Comb {                                   // needs n < p (else n! = 0 mod p)
    vector<ll> f, fi; ll p;
    Comb(int n, ll p) : f(n + 1), fi(n + 1), p(p) {
        f[0] = 1; rep(i, 1, n + 1) f[i] = f[i - 1] * i % p;
        fi[n] = power(f[n], p - 2, p);
        for (int i = n; i > 0; --i) fi[i - 1] = fi[i] * i % p;
    }
    ll C(int n, int k) const { return k < 0 || k > n ? 0 : f[n] * fi[k] % p * fi[n - k] % p; }
};
// C(n, k) for huge n (up to 1e18) and small k: product formula, p prime > k
ll Cbig(ll n, int k, ll p) {
    if (k < 0 || k > n) return 0;
    ll num = 1, den = 1;
    rep(i, 0, k) { num = (__int128)num * ((n - i) % p) % p; den = den * ((i + 1) % p) % p; }
    return num * power(den, p - 2, p) % p;
}
// Lucas: C(n, k) mod small prime p, with c = Comb(p - 1, p)
ll lucas(ll n, ll k, const Comb& c) {
    if (k < 0 || k > n) return 0;
    ll res = 1;
    while (n || k) { res = res * c.C(n % c.p, k % c.p) % c.p; n /= c.p; k /= c.p; }
    return res;
}
```

- Without a prime modulus (or n ≤ 60 exact): Pascal's triangle `C[n][k] = C[n-1][k-1] + C[n-1][k]` (fits `ll` up to n = 66 for all k; `__int128` further).
- C(n, k) mod prime power / composite: factor m, Lucas–Granville or CRT; rarely needed.

7.1.3 Linear Diophantine equations and CRT

a·x + b·y = c solvable iff g = gcd(a, b) divides c. One solution: `x0 = x·(c/g)`, `y0 = y·(c/g)` from extgcd. All solutions: `x = x0 + k·(b/g)`, `y = y0 - k·(a/g)`. To find the smallest non-negative x: `x = ((x0 % (b/g)) + b/g) % (b/g)` (use `__int128` for the multiplication).

```cpp
// [stress-tested] x ≡ a1 (mod m1), x ≡ a2 (mod m2), moduli need not be coprime.
// Returns {x, lcm} with 0 <= x < lcm, or {-1, -1} if no solution. Keep lcm < 2^62.
pair<ll, ll> crt(ll a1, ll m1, ll a2, ll m2) {
    ll x, y, g = extgcd(m1, m2, x, y);
    if ((a2 - a1) % g != 0) return {-1, -1};
    ll l = m1 / g * m2;
    __int128 t = (__int128)(a2 - a1) / g * x % (m2 / g);
    __int128 res = (a1 + t * m1) % l;
    if (res < 0) res += l;
    return {(ll)res, l};
}
```

Fold over many congruences pairwise. Uses: combine answers mod several primes, "the clock shows x mod a and y mod b" problems, periodic events that must coincide (first time t where several cyclic processes are simultaneously in given states).

7.1.4 Discrete logarithm (baby-step giant-step)

```cpp
// [stress-tested] smallest x >= 0 with a^x ≡ b (mod m), requires gcd(a, m) = 1; -1 if none. O(√m).
ll discreteLog(ll a, ll b, ll m) {
    a %= m; b %= m;
    if (b == 1 % m) return 0;
    ll n = (ll)sqrtl((long double)m) + 1, an = 1;
    rep(i, 0, n) an = (__int128)an * a % m;
    unordered_map<ll, ll> baby;                        // b * a^j -> largest j
    ll cur = b;
    rep(j, 0, n) { baby[cur] = j; cur = (__int128)cur * a % m; }
    ll g = 1;
    for (ll i = 1; i <= n; ++i) {
        g = (__int128)g * an % m;                      // a^(i n)
        auto it = baby.find(g);
        if (it != baby.end()) return i * n - it->second;
    }
    return -1;
}
```

7.1.5 GCD / LCM facts

- gcd(a, b) = gcd(a, b − a) = gcd(a, b mod a); gcd of an array with one changing element → prefix/suffix gcds.
- gcd(a + k, b + k) as k varies: gcd(a + k, b − a) → only divisors of |b − a| matter.
- Prefix gcds of an array take ≤ log2(max) distinct values (each change at least halves) — "all subarray gcds" has O(n log V) distinct (value, start) pairs.
- lcm can overflow: `a / gcd(a, b) * b`, check with `__int128` or cap.
- Number of pairs (i, j) ≤ n with gcd = 1 → Möbius / φ sums (§7.2). Σ_{d|n} φ(d) = n.
- `__gcd`, `std::gcd`, `std::lcm` (C++17) work on `ll`.
- "**Find p given several values s_i² mod p**" (ACPC 2024 *Four Questions*): p divides `s_i² - r_i` for each query, so p divides `gcd` of these numbers. Choose queries so the gcd is p (use s > √p so s² actually wraps; several queries remove spurious factors).

7.1.6 Useful closed forms

- Σi = n(n+1)/2, Σi² = n(n+1)(2n+1)/6, Σi³ = (n(n+1)/2)², geometric Σ_{i<n} $r^i = (r^n - 1)/(r - 1)$.
- Σ_{i=1}^{n} ⌊n/i⌋ = O(n log n) to iterate directly; computing it fast uses the O(√n) block trick (§2.5).
- Harmonic numbers H_n ≈ ln n + 0.5772. Number of primes ≤ n ≈ n / ln n (78498 below 10^6, 664579 below 10^7).

7.2 Primes, Factorisation, Divisors, Multiplicative Functions

7.2.1 Sieves

```cpp
// [stress-tested] smallest prime factor for all n <= N (linear sieve) + primes. N = 1e7: ~0.1 s, 40 MB
struct Sieve {
    vector<int> spf, primes;
    Sieve(int N) : spf(N + 1, 0) {
        rep(i, 2, N + 1) {
            if (!spf[i]) { spf[i] = i; primes.push_back(i); }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > N) break;
                spf[i * p] = p;
            }
        }
    }
    vector<pair<int, int>> factor(int x) const {       // x <= N, O(log x)
```

```
        vector<pair<int, int>> f;
        while (x > 1) { int p = spf[x], e = 0; while (x % p == 0) x /= p, ++e; f.push_back({p, e}); }
        return f;
    }
};
```

- Plain Eratosthenes with `vector<bool>` handles $N = 10^8$ in ~1 s.
- **Segmented sieve** for primes in [L, R] with $R \le 10^{12}$, $R - L \le 10^6$: sieve primes up to $\sqrt{R}$, then cross out multiples in the window.
- The linear sieve also computes multiplicative functions: φ, μ, number of divisors — handle the `i % p == 0` case separately.

7.2.2 Trial division ($n \le 10^{12}$–10^{14}) and factoring big numbers

```
// [stress-tested]
vector<pair<ll, int>> factorTrial(ll n) {      // O(sqrt n); with a primes list, O(pi(sqrt n))
    vector<pair<ll, int>> f;
    for (ll p = 2; p * p <= n; ++p) if (n % p == 0) {
        int e = 0; while (n % p == 0) n /= p, ++e; f.push_back({p, e});
    }
    if (n > 1) f.push_back({n, 1});
    return f;
}
```

"All prime factors $< 10^6$" (ACPC 2022 *Tricky Factoring*) ⇒ divide out primes up to 10^6 from a sieve, even if n is 10^{16}.

```
// [stress-tested] deterministic Miller-Rabin for n < 2^64, Pollard rho. ~1 ms per 64-bit number.
ull mulmod(ull a, ull b, ull m) { return (unsigned __int128)a * b % m; }
ull powmod(ull b, ull e, ull m) {
    ull r = 1; b %= m;
    for (; e; e >>= 1, b = mulmod(b, b, m)) if (e & 1) r = mulmod(r, b, m);
    return r;
}
bool isPrime(ull n) {
    if (n < 2) return false;
    for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n == p;
    ull d = n - 1; int s = 0; while (!(d & 1)) d >>= 1, ++s;
    for (ull a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
        ull x = powmod(a % n, d, n);
        if (x == 0 || x == 1 || x == n - 1) continue;
        bool comp = true;
        rep(i, 1, s) { x = mulmod(x, x, n); if (x == n - 1) { comp = false; break; } }
        if (comp) return false;
    }
    return true;
}
ull pollard(ull n) {
    if (n % 2 == 0) return 2;
    static mt19937_64 rr(1234);
    while (true) {
        ull x = rr() % n, y = x, c = rr() % (n - 1) + 1, d = 1;
        auto f = [&](ull v) { return (mulmod(v, v, n) + c) % n; };
        while (d == 1) {
            ull q = 1, xs = x;
            rep(i, 0, 64) { x = f(x); y = f(f(y)); q = mulmod(q, x > y ? x - y : y - x, n); if (!q) break; }
            d = __gcd(q, n);
            if (d == 1) continue;
            if (d == n) {                       // batch overshot: redo step by step
                x = xs; d = 1;
                while (d == 1) { x = f(x); y = f(f(y)); d = __gcd(x > y ? x - y : y - x, n); }
            }
        }
        if (d != n) return d;
    }
}
void factorRho(ull n, map<ull, int>& f) {
    if (n == 1) return;
    if (isPrime(n)) { f[n]++; return; }
    ull d = pollard(n); factorRho(d, f); factorRho(n / d, f);
}
```

7.2.3 Divisors

- Enumerate all divisors from the factorisation (recursive product over prime powers). Count = $\Pi(e_i + 1)$, sum $\sigma = \Pi\ (p^{e+1} - 1)/(p - 1)$.

- Max divisor count: $\le 10^6 \to 240$; $\le 10^9 \to 1344$; $\le 10^{12} \to 6720$; $\le 10^{16} \to 41472$; $\le 10^{18} \to 103680$. So "iterate over all divisors" or even "all pairs of divisors" of a single number is often fine.
- **Harmonic enumeration:** for every $d \le N$, iterate multiples of d: $\Sigma\, N/d = O(N \log N)$. Use for "count pairs with gcd = d" (cnt[d] = #multiples choose 2, then subtract cnt of multiples of d from larger to smaller).
- Sum of proper divisors (aliquot, ACPC 2023 *Outlaw Numbers*) $= \sigma(n) - n$.
- Counting divisors of all numbers $\le N$: `for d: for m = d; m <= N; m += d: cnt[m]++`.

7.2.4 Euler's totient and Möbius

```
// [stress-tested]
ll phi(ll n) {                    // O(sqrt n)
    ll r = n;
    for (ll p = 2; p * p <= n; ++p) if (n % p == 0) { while (n % p == 0) n /= p; r -= r / p; }
    if (n > 1) r -= r / n;
    return r;
}
// sieve: phi[i] = i; for each prime p: for multiples m: phi[m] -= phi[m] / p
```

- $\varphi(n) = \#\{1 \le k \le n : \gcd(k, n) = 1\}$; $\Sigma_{d|n}\, \varphi(d) = n$; a^φ(m) ≡ 1 (mod m) when gcd(a, m) = 1.
- **Möbius** $\mu(n) = 0$ if a square divides n, else $(-1)^{\#\text{primes}}$. $\Sigma_{d|n}\, \mu(d) = [n = 1]$.
- **Möbius inversion:** if $g(n) = \Sigma_{d|n}\, f(d)$ then $f(n) = \Sigma_{d|n}\, \mu(d)\, g(n/d)$. Same for "multiples": if $G(d) = \Sigma_{d|m}\, F(m)$ then $F(d) = \Sigma_{d|m}\, \mu(m/d)\, G(m)$.
- **Count coprime pairs** ($i \le A$, $j \le B$): $\Sigma_d\, \mu(d) \cdot \lfloor A/d \rfloor \cdot \lfloor B/d \rfloor$ — with the √ block trick $O(\sqrt{n})$ per query.
- $\Sigma_{i=1}^{n} \gcd(i, n) = \Sigma_{d|n}\, d \cdot \varphi(n/d)$.

7.2.5 Other facts

- **Legendre:** exponent of prime p in n! $= \Sigma\, \lfloor n/p^k \rfloor$. Trailing zeros of n! = exponent of 5.
- **Kummer:** exponent of p in C(n, k) = number of carries when adding k and n − k in base p. C(n, k) odd ⇔ (k & n) == k.
- **Wilson:** $(p-1)! \equiv -1 \pmod p$. **Fermat:** $a^{p-1} \equiv 1$.
- Every integer > 1 is a product of primes; there are infinitely many primes in every coprime residue class; prime gaps below 2^{64} are ≤ 1550 (so "next prime after n" by testing successive numbers is fast).
- **Goldbach** (verified far beyond contest ranges): even n > 2 is a sum of two primes; odd n > 5 is a sum of three primes.
- **Quadratic residues:** for odd prime p, exactly (p−1)/2 non-zero squares; Euler's criterion a^((p−1)/2) ≡ ±1.
- Integer `a^b` exact comparison without overflow: compare `b·log(a)` with a margin, then verify exactly with `__int128`/capped multiplication (ACPC 2021 *Associative Exponents*).

7.3 Counting

7.3.1 The standard toolbox

objects	count
sequences of length k from n symbols	n^k
arrangements of k distinct out of n	n!/(n−k)!
subsets of size k	C(n, k)
multisets of size k from n types (stars & bars)	C(n + k − 1, k)
solutions of $x_1 + \dots + x_n = k$, $x_i \ge 0$	C(n + k − 1, n − 1)
same with $x_i \ge 1$	C(k − 1, n − 1)
same with $x_i \le c$	inclusion–exclusion: $\Sigma_j\, (-1)^j$ C(n, j) C(k − j(c+1) + n − 1, n − 1)
permutations of a multiset (counts c_i)	n! / Π c_i!
lattice paths (0,0) → (a,b) with steps R/U	C(a + b, a)
paths that never go below the diagonal	Catalan-type (reflection, below)
surjections n → k	$\Sigma_j\, (-1)^j$ C(k, j)$(k - j)^n = k! \cdot S(n, k)$
derangements D_n	$(n-1)(D_{n-1} + D_{n-2})$, $D_n \approx n!/e$
labelled trees on n vertices	n^{n-2} (Cayley); forests with k given roots: $k \cdot n^{n-k-1}$
binary strings of length n with no two adjacent 1s	Fibonacci F_{n+2}

Catalan C_n = C(2n, n)/(n+1) = C(2n, n) − C(2n, n+1): 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796. Counts balanced bracket sequences, full binary trees with n+1 leaves, triangulations of an (n+2)-gon, monotone lattice paths not crossing the diagonal, non-crossing matchings of 2n points, stack-sortable permutations. Recurrence C_{n+1} = Σ C_i C_{n−i}.

Reflection principle: a path that touches the forbidden line can be reflected (from its first touch) to a path starting at the mirrored start point; so #bad = #paths from the mirrored start. **Ballot:** sequences of n R-steps and m U-steps (m ≤ n) in which every prefix has #U ≤ #R: C(n + m, m) − C(n + m, m − 1) (n = m gives Catalan). Strictly more R than U in every non-empty prefix: (n − m)/(n + m)·C(n + m, m).

Stirling numbers of the 2nd kind S(n, k) (partitions of n labelled items into k non-empty unlabelled groups): S(n, k) = k·S(n−1, k) + S(n−1, k−1). **Bell numbers** B_n = Σ_k S(n, k): 1, 1, 2, 5, 15, 52, 203, 877, 4140. **Stirling 1st kind** (permutations with k cycles): c(n, k) = (n−1)c(n−1, k) + c(n−1, k−1).

Integer partitions p(n) (unordered sums): p(n) = 1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42 …, p(100) ≈ $1.9 \cdot 10^8$. DP: `for part in 1..n: for s = part..n: p[s] += p[s - part]` (coin-change counting).

7.3.2 Identities

- Pascal C(n, k) = C(n−1, k−1) + C(n−1, k); symmetry C(n, k) = C(n, n−k); Σ_k C(n, k) = 2^n; Σ_k $(-1)^k$ C(n, k) = 0.
- **Hockey stick:** Σ_{i=k}^n C(i, k) = C(n+1, k+1). **Vandermonde:** Σ_j C(a, j) C(b, k−j) = C(a + b, k).
- k·C(n, k) = n·C(n−1, k−1). Σ_k k·C(n, k) = $n \cdot 2^{n-1}$.
- C(n, k) is maximal at k = n/2; C(60, 30) ≈ $1.2 \cdot 10^{17}$ (fits in `ll`), C(67, 33) overflows `ll`.
- Number of monotone paths in a grid avoiding forbidden cells: sort cells, dp[i] = paths to cell i avoiding earlier forbidden ones = C(to i) − Σ_{j before i} dp[j]·C(j → i).

7.3.3 Inclusion–exclusion

|A_1 ∪ … ∪ A_n| = Σ over nonempty S of $(-1)^{|S|+1}$·|intersection of A_i for i ∈ S|. Typical shape: count objects violating **none** of n conditions = Σ over all S of $(-1)^{|S|}$·(#objects violating at least all conditions in S). With symmetric conditions this collapses to Σ_j $(-1)^j$ C(n, j)·f(j).

- Count integers ≤ N divisible by none of primes p_1..p_k: Σ over subsets $(-1)^{|S|}$ ⌊N / Π p⌋ (k ≤ 15 → 2^k terms; prune when the product exceeds N).
- Count arrangements where at least one of p properties holds (ACPC 2024 *A Little to the Right*): either inclusion–exclusion over subsets of properties, or observe structure (a property with all-distinct values defines exactly one order; answer = number of distinct such orders).
- Inclusion–exclusion over subsets of n ≤ 20 with SOS (§4.2) when the "intersection" depends on a mask.

7.3.4 Burnside / Pólya (counting up to symmetry)

Number of distinct objects = (1/|G|) Σ_{g ∈ G} fix(g). For colourings, fix(g) = $k^{c(g)}$ with c(g) = number of cycles of g on positions.

- **Necklaces** (rotations only), n beads, k colours: (1/n) Σ_{d|n} φ(d) $k^{n/d}$.
- **Bracelets** (rotations + reflections): necklaces/2 + (n odd: $k^{(n+1)/2}/2$; n even: $(k^{n/2} + k^{n/2+1})/4$).
- Cube faces with k colours (rotations): $(k^6 + 3k^4 + 12k^3 + 8k^2)/24$.
- Modulo p: divide by |G| using an inverse (p must not divide |G|).

7.3.5 Probability-flavoured counting

- Expected number of distinct values when drawing k times uniformly from n: $n(1 - (1 - 1/n)^k)$.
- Random permutation: expected cycles H_n; probability that 1 and 2 are in the same cycle 1/2.
- Birthday: collision among k random values in [0, N) likely once k ≈ √N (so 64-bit hashes of 10^6 strings are safe, 32-bit are not).

7.4 Matrices, Linear Recurrences, Gaussian Elimination, XOR Basis

7.4.1 Matrix exponentiation

```
// [stress-tested]
using Mat = vector<vector<ll>>;
Mat mul(const Mat& A, const Mat& B, ll mod) {
    int n = sz(A), m = sz(B[0]), k = sz(B);
    Mat C(n, vector<ll>(m, 0));
```

```
    rep(i, 0, n) rep(t, 0, k) if (A[i][t]) rep(j, 0, m) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % mod;
    return C;
}
Mat matpow(Mat A, ll e, ll mod) {
    int n = sz(A); Mat R(n, vector<ll>(n, 0)); rep(i, 0, n) R[i][i] = 1 % mod;
    for (; e > 0; e >>= 1, A = mul(A, A, mod)) if (e & 1) R = mul(R, A, mod);
    return R;
}
```

- **Linear recurrence** f(n) = c_1 f(n−1) + … + c_k f(n−k): companion matrix (first row = c, sub-diagonal ones), O(k³ log n). Add a constant/polynomial term by adding extra state rows (e.g. a row that stays 1).
- **Number of walks of length L** between all pairs = A^L (adjacency matrix). Shortest walk with exactly L edges: (min, +) matrix "multiplication" with the same `matpow` skeleton.
- **DP with a fixed linear transition repeated 10^{18} times** (e.g. tiling a 4 × n strip, states = profiles) → matrix power over the state vector.
- Cost is k³·log n: k = 100 → $10^6 \cdot 60$ = fine; k = 200 is the practical limit.
- Fibonacci: $[[1,1],[1,0]]^n = [[F_{n+1}, F_n],[F_n, F_{n-1}]]$. Identities: $F_{2n} = F_n(2F_{n+1} - F_n)$, $F_{2n+1} = F_n^2 + F_{n+1}^2$.

7.4.2 Berlekamp–Massey: guess the recurrence from terms

Compute the first ~2k terms of any sequence that satisfies a linear recurrence of order ≤ k (by brute force / DP), feed them to BM, get the recurrence; then jump to the n-th term in O(k² log n). Great for "count tilings/paths of size n ≤ 10^{18}" when the DP is linear but you don't want to derive the matrix.

```
// [stress-tested] mod must be prime. Returns tr with s[i] = sum_j tr[j] * s[i-1-j].
vector<ll> berlekampMassey(const vector<ll>& s, ll mod) {
    auto pw = [&](ll x, ll e) {
        ll r = 1; for (x %= mod; e; e >>= 1, x = x * x % mod) if (e & 1) r = r * x % mod; return r;
    };
    int n = sz(s), L = 0, m = 0; vector<ll> C(n, 0), B(n, 0), T; C[0] = B[0] = 1; ll b = 1;
    rep(i, 0, n) {
        ++m; ll d = s[i] % mod;
        rep(j, 1, L + 1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * pw(b, mod - 2) % mod;
        rep(j, m, n) C[j] = ((C[j] - coef * B[j - m]) % mod + mod) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }
    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
// k-th term (0-indexed) given first terms S (size >= sz(tr)) and recurrence tr
ll linearRec(const vector<ll>& S, const vector<ll>& tr, ll k, ll mod) {
    int n = sz(tr);
    if (n == 0) return 0;
    auto combine = [&](const vector<ll>& a, const vector<ll>& b) {
        vector<ll> res(2 * n + 1, 0);
        rep(i, 0, n + 1) rep(j, 0, n + 1) res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j, 0, n) res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1); return res;
    };
    vector<ll> pol(n + 1, 0), e(n + 1, 0); pol[0] = e[1] = 1;
    for (++k; k; k /= 2) { if (k % 2) pol = combine(pol, e); e = combine(e, e); }
    ll res = 0;
    rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

Give BM at least 2·(order) terms, preferably more (it returns the shortest recurrence consistent with what it saw).

7.4.3 Gaussian elimination (reals)

```
// [stress-tested] a = n rows of (m coefficients + rhs).
// Returns 0: no solution, 1: unique, 2: infinitely many.
// ans = one solution (free variables = 0).
int gauss(vector<vector<double>> a, vector<double>& ans) {
    const double EPS = 1e-9;
    int n = sz(a), m = sz(a[0]) - 1; vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
```

```
        rep(i, row, n) if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        if (fabs(a[sel][col]) < EPS) continue;
        swap(a[sel], a[row]); where[col] = row;
        rep(i, 0, n) if (i != row) {
            double c = a[i][col] / a[row][col];
            rep(j, col, m + 1) a[i][j] -= a[row][j] * c;
        }
        ++row;
    }
    ans.assign(m, 0);
    rep(i, 0, m) if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    rep(i, 0, n) {
        double s = 0; rep(j, 0, m) s += ans[j] * a[i][j];
        if (fabs(s - a[i][m]) > EPS) return 0;
    }
    rep(i, 0, m) if (where[i] == -1) return 2;
    return 1;
}
```

- **Expected values with cycles** (random walk on a graph, Markov chains): one equation per state `E_v - Σ p_vu E_u = c_v`, absorbing states `E = 0`. n ≤ 500.
- **Rank** = number of pivots.

```
// [stress-tested] determinant mod prime p (entries any ll). Same elimination solves systems mod p.
ll detMod(vector<vector<ll>> a, ll p) {
    auto pw = [&](ll b, ll e) {
        ll r = 1; for (b %= p; e; e >>= 1, b = b * b % p) if (e & 1) r = r * b % p; return r;
    };
    int n = sz(a); ll det = 1;
    for (auto& row : a) for (auto& x : row) x = ((x % p) + p) % p;
    rep(c, 0, n) {
        int piv = -1; rep(r, c, n) if (a[r][c]) { piv = r; break; }
        if (piv < 0) return 0;
        if (piv != c) { swap(a[piv], a[c]); det = (p - det) % p; }
        det = det * a[c][c] % p;
        ll iv = pw(a[c][c], p - 2);
        rep(r, c + 1, n) if (a[r][c]) {
            ll f = a[r][c] * iv % p;
            rep(k, c, n) a[r][k] = ((a[r][k] - f * a[c][k]) % p + p) % p;
        }
    }
    return det;
}
```

- **Kirchhoff (matrix-tree):** #spanning trees = any cofactor of the Laplacian L = D − A (delete one row and column, take determinant; mod p or doubles for small n).

7.4.4 Polynomial interpolation (Lagrange)

If the answer is a polynomial of degree k in n (e.g. $\Sigma_{i=1}^{n} i^k$ has degree k + 1; many counting DPs with a fixed number of nested sums), compute it for n = 0..k by brute force and evaluate at huge n in O(k).

```
// [stress-tested] y[i] = P(i) for i = 0..k (deg P <= k). Returns P(x) mod prime p (p > k).
ll lagrange(const vector<ll>& y, ll x, ll p) {
    int k = sz(y) - 1; x %= p; if (x < 0) x += p;
    if (x <= k) return y[x] % p;
    auto pw = [&](ll b, ll e) {
        ll r = 1; for (b %= p; e; e >>= 1, b = b * b % p) if (e & 1) r = r * b % p; return r;
    };
    vector<ll> pre(k + 2, 1), suf(k + 2, 1), fact(k + 1, 1);
    rep(i, 0, k + 1) pre[i + 1] = pre[i] * ((x - i) % p) % p;
    for (int i = k; i >= 0; --i) suf[i] = suf[i + 1] * ((x - i) % p) % p;
    rep(i, 1, k + 1) fact[i] = fact[i - 1] * i % p;
    ll res = 0;
    rep(i, 0, k + 1) {
        ll num = pre[i] * suf[i + 1] % p, den = fact[i] * fact[k - i] % p;
        ll term = y[i] % p * num % p * pw(den, p - 2) % p;
        res = ((k - i) & 1) ? (res - term + p) % p : (res + term) % p;
    }
    return res;
}
```

7.4.5 XOR basis (Gaussian elimination over GF(2))

```
// [stress-tested]
struct XorBasis {
```

```
    ll b[62] = {}; int cnt = 0;
    bool insert(ll x) {                              // returns false if x already representable
        for (int i = 61; i >= 0; --i) if (x >> i & 1) {
            if (!b[i]) { b[i] = x; ++cnt; return true; }
            x ^= b[i];
        }
        return false;
    }
    bool canMake(ll x) const { for (int i = 61; i >= 0; --i) if (x >> i & 1) x ^= b[i]; return x == 0; }
    ll maxXor(ll x = 0) const { for (int i = 61; i >= 0; --i) x = max(x, x ^ b[i]); return x; }
};
```

- Number of distinct subset-xors $= 2^{\text{cnt}}$; each achievable value is hit by $2^{n - \text{cnt}}$ subsets.
- **Max xor path** in an undirected graph between u and v: xor of any path + any subset of cycle xors → DFS xor-to-root `d`, insert `d[a] ^ d[b] ^ w` for every non-tree edge, answer `maxXor(d[u] ^ d[v])`.
- Systems of equations mod 2 with up to ~2000 variables: use `bitset<N>` rows, elimination $O(n^3/64)$.
- Lights-out style puzzles (press toggles neighbours): linear system over GF(2).

7.5 FFT / NTT (Convolution)

Convolution c_k = Σ_{i+j=k} a_i b_j in O(n log n). Recognise it: "**for every shift/offset/sum k, aggregate over pairs (i, j) with i + j = k (or i − j = k)**". Reverse one array to turn differences into sums.

7.5.1 FFT with doubles (exact for integer results up to ~10^{14})

```
// [stress-tested]
using cd = complex<double>;
void fft(vector<cd>& a, bool invert) {
    int n = sz(a);
    for (int i = 1, j = 0; i < n; ++i) {                // bit reversal
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    vector<cd> rt(max(1, n / 2));                        // accurate roots, computed in long double
    rep(k, 0, n / 2) { long double ang = 2 * acosl(-1) * k / n; rt[k] = cd(cosl(ang), sinl(ang)); }
    for (int len = 2; len <= n; len <<= 1) {
        int step = n / len;
        for (int i = 0; i < n; i += len) rep(j, 0, len / 2) {
            cd w = rt[j * step]; if (invert) w = conj(w);
            cd u = a[i + j], v = a[i + j + len / 2] * w;
            a[i + j] = u + v; a[i + j + len / 2] = u - v;
        }
    }
    if (invert) for (cd& x : a) x /= n;
}
vector<ll> multiply(const vector<ll>& a, const vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    int n = 1; while (n < sz(a) + sz(b)) n <<= 1;
    vector<cd> fa(all(a)), fb(all(b)); fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    rep(i, 0, n) fa[i] *= fb[i];
    fft(fa, true);
    vector<ll> res(sz(a) + sz(b) - 1);
    rep(i, 0, sz(res)) res[i] = llround(fa[i].real());
    return res;
}
```

7.5.2 NTT modulo 998244353 (exact)

```
// [stress-tested] mod = 998244353 = 119 * 2^23 + 1, primitive root 3. Max length 2^23.
const ll NMOD = 998244353;
ll npow(ll b, ll e) {
    ll r = 1; b %= NMOD;
    for (; e; e >>= 1, b = b * b % NMOD) if (e & 1) r = r * b % NMOD;
    return r;
}
void ntt(vector<ll>& a, bool invert) {
    int n = sz(a);
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
```

```
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll w = npow(3, (NMOD - 1) / len); if (invert) w = npow(w, NMOD - 2);
        for (int i = 0; i < n; i += len) {
            ll wn = 1;
            rep(j, 0, len / 2) {
                ll u = a[i + j], v = a[i + j + len / 2] * wn % NMOD;
                a[i + j] = u + v < NMOD ? u + v : u + v - NMOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + NMOD;
                wn = wn * w % NMOD;
            }
        }
    }
    if (invert) { ll ni = npow(n, NMOD - 2); for (ll& x : a) x = x * ni % NMOD; }
}
vector<ll> multiplyMod(vector<ll> a, vector<ll> b) {
    if (a.empty() || b.empty()) return {};
    int need = sz(a) + sz(b) - 1, n = 1; while (n < need) n <<= 1;
    a.resize(n); b.resize(n); ntt(a, false); ntt(b, false);
    rep(i, 0, n) a[i] = a[i] * b[i] % NMOD;
    ntt(a, true); a.resize(need); return a;
}
```

For mod 10^9+7 (not NTT-friendly): split each coefficient as `x = hi·2^15 + lo` and do 3–4 double FFTs, or three NTT primes + CRT. Usually the problem uses 998244353 if it intends NTT.

7.5.3 Applications

- **Count pairs by sum:** `cnt = multiply(freq, freq)`; pairs with i < j: subtract the diagonal (`freq[x]` at index 2x) and halve.
- **Pattern matching with mismatches / count matching positions** (for every alignment i, how many j have text[i+j] == pat[j]): for each letter c, `multiply(isC(text), reverse(isC(pat)))`; entry at index `i + m - 1` counts matches for alignment i. Sum over letters (or over the 2 colours of a binary image). O(σ·n log n).
- **Wildcards** (`?` matches anything): with p_j = 0 for `?`, alignment i matches iff Σ_j p_j t_{i+j}(p_j − t_{i+j})² = 0 → expands to 3 convolutions. Use random values for letters to avoid accidental zeros when working mod p.
- **2-D matching** (ACPC 2017 *Matchings*: count equal pixels for every placement of a small image inside a big one): flatten both images with the **big image's width W**: index r·W + c. Place the small image's pixel (r, c) at r·W + c too. Then one 1-D correlation gives all placements; only use top-left positions with c + w ≤ W and r + h ≤ H.
- **Big integer multiplication:** digits as coefficients, then carry.
- **Subset sums with multiplicities / knapsack counting** with many items of small weight: multiply generating polynomials (divide & conquer on the product to keep sizes balanced).
- **Polynomial with sums over all pairs of differences** |a_i − a_j| = d for every d: multiply freq by reversed freq.
- Sizes: n up to ~10^6 per multiplication is fine (a few hundred ms).

8. Strings

8.1 Hashing, Prefix Function (KMP), Z-Function

8.1.1 Polynomial hashing (mod $2^{61} - 1$, random base)

```
// [stress-tested] O(1) substring hash after O(n) preprocessing. Collision prob ~ n^2 / 2^61.
struct Hasher {
    static const ull M = (1ULL << 61) - 1;
    static ull mul(ull a, ull b) {
        __uint128_t c = (__uint128_t)a * b;
        ull r = (ull)(c & M) + (ull)(c >> 61);
        return r >= M ? r - M : r;
    }
    static ull base() {
        static ull B = mt19937_64(chrono::steady_clock::now().time_since_epoch().count())() % (M - 1000) + 500;
        return B;
    }
    vector<ull> h, p;
    Hasher(const string& s) : h(sz(s) + 1, 0), p(sz(s) + 1, 1) {
        ull B = base();
        rep(i, 0, sz(s)) { h[i + 1] = (mul(h[i], B) + (unsigned char)s[i]) % M; p[i + 1] = mul(p[i], B); }
    }
    ull get(int l, int r) const {                  // hash of s[l, r)
        ull v = h[r] + M - mul(h[l], p[r - l]);
        return v >= M ? v - M : v;
    }
    // hash of concatenation A + B given hash(A), hash(B), len(B): mul(hA, p^lenB) + hB  (mod M)
};
```

Uses: - **Compare substrings** in O(1); **LCP of two suffixes** in O(log n) by binary search on length → suffix comparison → sort suffixes/rotations with a custom comparator in O(n log² n). - **Longest repeated substring / longest common substring of k strings**: binary search the length, put all hashes of that length in a hash set. - **Palindrome check** of s[l, r): hash of s vs hash of reverse(s) on the mirrored range. - **Canonical forms** for sets of strings: hash of the minimal rotation (§8.2) for "equal up to rotation" (ACPC 2020 *Password Rotation*: insert min-rotation of s and of reverse(s)). - Hashing sequences of ints: use values + 1 (avoid 0) and the same formula; for multisets use sum/xor of random 64-bit values per element (§2.5). - Anti-hack is irrelevant on Kattis, but **never use base 31/mod 2^{64} with natural overflow** on adversarial tests (Thue–Morse strings break it).

8.1.2 Prefix function and KMP

`pi[i]` = length of the longest proper border (prefix = suffix) of s[0..i].

```
// [stress-tested]
vector<int> prefixFunction(const string& s) {
    int n = sz(s); vector<int> pi(n, 0);
    rep(i, 1, n) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
vector<int> kmpFind(const string& text, const string& pat) {    // start indices of occurrences
    vector<int> pi = prefixFunction(pat), res; int j = 0, m = sz(pat);
    if (m == 0) return res;
    rep(i, 0, sz(text)) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == m) { res.push_back(i - m + 1); j = pi[j - 1]; }
    }
    return res;
}
```

- **Smallest period** of s: p = n − pi[n−1]; s is a repetition of its prefix of length p iff n % p == 0.
- All borders: n, pi[n−1], pi[pi[n−1]−1], … (chain).
- **Count occurrences of each prefix** in s: `cnt[pi[i]]++`, then propagate `cnt[pi[i-1]] += cnt[i]` from long to short, plus 1 each.

- **KMP automaton**: `go[state][c]` for building DP over strings that must avoid/contain a pattern ("count strings of length n not containing P": dp over (length, automaton state) — matrix power for huge n).
- Rotation check: t is a rotation of s ⇔ $|s| = |t|$ and t occurs in s + s.

8.1.3 Z-function

`z[i]` = length of the longest common prefix of s and s[i..].

```
// [stress-tested]
vector<int> zFunction(const string& s) {
    int n = sz(s); vector<int> z(n, 0);
    if (n) z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
```

- Occurrences of P in T: z-function of `P + '#' + T`, positions with $z \geq |P|$.
- For each suffix of T, the longest prefix of P that starts there: same concatenation trick.
- Number of distinct substrings in $O(n^2)$ by appending characters and using z on the reversed prefix.
- z and pi are interconvertible; use whichever is easier to reason about.

8.2 Palindromes, Rotations, Tries, Aho–Corasick, Suffix Structures

8.2.1 Manacher (all maximal palindromes in O(n))

```
// [stress-tested] t = "#s0#s1#...#"; p[i] = radius in t (including centre).
// The palindrome centred at t[i] has length p[i] - 1 in s and starts at s index (i - p[i] + 1) / 2.
vector<int> manacher(const string& s) {
    string t = "#"; for (char c : s) { t += c; t += '#'; }
    int n = sz(t); vector<int> p(n, 0);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 1 : min(p[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && t[i - k] == t[i + k]) ++k;
        p[i] = k--;
        if (i + k > r) { l = i - k; r = i + k; }
    }
    return p;
}
```

- Longest palindromic substring: max over i of `p[i] - 1`.
- Number of palindromic substrings: $\Sigma_i \lfloor p[i] / 2 \rfloor$.
- Is s[l..r] a palindrome: centre i = l + r + 1 in t; check `p[i] - 1 ≥ r - l + 1`.
- Palindromic **subsequences** are a different problem: interval DP (§4.1).
- Minimum insertions to make a palindrome = n − LPS (longest palindromic subsequence = LCS(s, reverse s)).

8.2.2 Minimal rotation / canonical cyclic form

```
// [stress-tested] index of the lexicographically smallest rotation, O(n)
int minRotation(const string& s) {
    int n = sz(s), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) { ++k; continue; }
        if (a > b) i += k + 1; else j += k + 1;
        if (i == j) ++j;
        k = 0;
    }
    return min(i, j);
}
// canonical form up to rotation:      rotate(s, minRotation(s))
// up to rotation AND reflection:       min(canon(s), canon(reverse(s)))
```

Works for any sequence (vector of ints) — useful for necklaces, cyclic polygons (canonical edge/angle sequence), cycle comparison in graphs.

8.2.3 Trie

```
// [stress-tested]
struct Trie {
    vector<array<int, 26>> nxt; vector<int> endCnt, passCnt;
    Trie() { newNode(); }
    int newNode() {
        nxt.push_back({}); nxt.back().fill(-1); endCnt.push_back(0); passCnt.push_back(0);
        return sz(nxt) - 1;
    }
    void insert(const string& s) {
        int v = 0; passCnt[0]++;
        for (char ch : s) {
            int c = ch - 'a';
            if (nxt[v][c] < 0) { int u = newNode(); nxt[v][c] = u; }
            v = nxt[v][c]; passCnt[v]++;
        }
        endCnt[v]++;
    }
    int countPrefix(const string& s) {             // # inserted strings with prefix s
        int v = 0; for (char ch : s) { v = nxt[v][ch - 'a']; if (v < 0) return 0; } return passCnt[v];
    }
};
```

Binary trie over bits (high to low) for max xor with a set: at each level go to the child with the opposite bit if it exists. Store counts to support deletion and "count values with x ^ v < k".

8.2.4 Aho–Corasick (many patterns in one text)

```
// [stress-tested] lowercase. After build(): cnt[state] = #patterns that end at state (incl. via links).
struct AhoCorasick {
    vector<array<int, 26>> go; vector<int> link, cnt, order;
    AhoCorasick() { add(); }
    int add() {
        go.push_back({}); go.back().fill(-1); link.push_back(0); cnt.push_back(0);
        return sz(go) - 1;
    }
    int insert(const string& s) {                  // returns terminal state of s
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (go[v][c] < 0) { int u = add(); go[v][c] = u; }
            v = go[v][c];
        }
        cnt[v]++; return v;
    }
    void build() {
        queue<int> q; order.clear();
        rep(c, 0, 26) { if (go[0][c] < 0) go[0][c] = 0; else { link[go[0][c]] = 0; q.push(go[0][c]); } }
        while (!q.empty()) {
            int v = q.front(); q.pop(); order.push_back(v);
            cnt[v] += cnt[link[v]];
            rep(c, 0, 26) {
                int u = go[v][c];
                if (u < 0) go[v][c] = go[link[v]][c];
                else { link[u] = go[link[v]][c]; q.push(u); }
            }
        }
    }
    ll countMatches(const string& text) {          // total pattern occurrences in text
        ll res = 0; int v = 0;
        for (char ch : text) { v = go[v][ch - 'a']; res += cnt[v]; }
        return res;
    }
};
```

- Occurrences **per pattern**: count visits per state while scanning, then push counts up the suffix-link tree in reverse BFS `order` (`visits[link[v]] += visits[v]`), read at each pattern's terminal state.
- DP over the automaton: "count strings of length n containing none of the patterns" = dp over (length, state) avoiding states with cnt > 0.

8.2.5 Suffix array + LCP

```
// [stress-tested] sa[0] = n (empty suffix), sa[1..n] = suffixes in sorted order.
// lcp[i] = LCP(suffix sa[i-1], suffix sa[i]) for i >= 1. Characters must be > 0.
struct SuffixArray {
```

```
    vector<int> sa, lcp;
    SuffixArray(string s, int lim = 256) {
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vector<int> x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y; iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j; iota(all(y), n - j);
            rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y); p = 1; x[sa[0]] = 0;
            rep(i, 1, n) {
                a = sa[i - 1]; b = sa[i];
                x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
            }
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1]; s[i + k] == s[j + k]; k++);
    }
};
```

- **Distinct substrings** = $n(n+1)/2 - \Sigma$ lcp.
- **LCP of any two suffixes** = range-min of lcp between their ranks (sparse table, §3.4).
- **Longest repeated substring** = max lcp. **Longest common substring of A and B**: SA of `A + '#' + B` (separator smaller than letters but > 0, e.g. '#'), max lcp over adjacent pairs from different sides.
- **k-th distinct substring**: walk sa in order, each suffix contributes (n − sa[i] − lcp[i]) new substrings.
- Pattern search: binary search over sa comparing with the pattern, O(m log n).

8.2.6 Suffix automaton (when you need counts of occurrences of every substring)

```
// [stress-tested] states <= 2n. Distinct substrings = sum over v != 0 of len[v] - len[link[v]].
struct SAM {
    struct St { int len, link; map<char, int> next; };
    vector<St> st; int last = 0;
    SAM() { st.push_back({0, -1, {}}); }
    void extend(char c) {
        int cur = sz(st); st.push_back({st[last].len + 1, 0, {}});
        int p = last;
        while (p != -1 && !st[p].next.count(c)) { st[p].next[c] = cur; p = st[p].link; }
        if (p != -1) {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                int clone = sz(st); St cl = st[q]; cl.len = st[p].len + 1; st.push_back(cl);
                while (p != -1 && st[p].next.count(c) && st[p].next[c] == q) {
                    st[p].next[c] = clone; p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
};
```

Occurrences of each state's substrings: `occ[cur] = 1` for non-clone states, then add occ[v] to occ[link[v]] in decreasing order of len. A string P occurs in s iff walking `next` from state 0 along P never fails.

9. Geometry

9.1 Geometry Primitives

Rule 1: stay in integers as long as possible. Inputs are usually integers; cross/dot products of integer points are exact in `ll` ($|\text{coords}| \le 10^9 \Rightarrow$ products $\le 4 \cdot 10^{18}$: borderline, use `__int128` for cross products of differences if coordinates reach 10^9). Only divide/sqrt at the very end. **Rule 2:** with doubles, compare with `EPS` (1e-9 relative to the scale of the input).

9.1.1 Point / vector

```
// [stress-tested]
template <class T> struct P {
    T x, y;
    P(T x = 0, T y = 0) : x(x), y(y) {}
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    P operator*(T k) const { return {x * k, y * k}; }
    P operator/(T k) const { return {x / k, y / k}; }
    bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(P o) const { return x == o.x && y == o.y; }
    T dot(P o) const { return x * o.x + y * o.y; }
    T cross(P o) const { return x * o.y - y * o.x; }
    T cross(P a, P b) const { return (a - *this).cross(b - *this); }   // >0: a->b turns left around *this
    T norm2() const { return x * x + y * y; }
    double len() const { return sqrt((double)norm2()); }
    double angle() const { return atan2((double)y, (double)x); }        // (-pi, pi]
    P perp() const { return {-y, x}; }                                  // rotate +90°
    P<double> rotate(double a) const { return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)}; }
};
using Pt = P<ll>; using Pd = P<double>;
template <class T> int sgn(T v) { return (v > 0) - (v < 0); }
// orientation of c relative to line a->b: +1 left (CCW), -1 right (CW), 0 collinear
template <class T> int orient(P<T> a, P<T> b, P<T> c) { return sgn(a.cross(b, c)); }
```

9.1.2 Segments and lines

```
// [stress-tested] is p on segment [a, b] (integers, exact)
bool onSegment(Pt a, Pt b, Pt p) {
    return a.cross(b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
                              && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
// do closed segments [a,b] and [c,d] intersect (touching counts)
bool segIntersect(Pt a, Pt b, Pt c, Pt d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d), o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 * o2 < 0 && o3 * o4 < 0) return true;                        // proper crossing
    return onSegment(a, b, c) || onSegment(a, b, d) || onSegment(c, d, a) || onSegment(c, d, b);
}
// intersection point of infinite lines (a,b) and (c,d); requires non-parallel
Pd lineIntersection(Pd a, Pd b, Pd c, Pd d) {
    double t = (c - a).cross(d - c) / (b - a).cross(d - c);
    return a + (b - a) * t;
}
// distance from p to segment [a, b]
double distPointSegment(Pd a, Pd b, Pd p) {
    Pd ab = b - a; double L = ab.norm2();
    if (L == 0) return (p - a).len();
    double t = max(0.0, min(1.0, (p - a).dot(ab) / L));
    return (p - (a + ab * t)).len();
}
// projection of p onto the line (a, b); reflection = 2 * proj - p
Pd projectLine(Pd a, Pd b, Pd p) { Pd ab = b - a; return a + ab * ((p - a).dot(ab) / ab.norm2()); }
```

- Distance from p to the infinite line (a, b): `|cross(b - a, p - a)| / |b - a|`.
- Segment–segment distance: 0 if they intersect, else min of the 4 point–segment distances.
- **Lattice points** on segment (a, b) inclusive: `gcd(|dx|, |dy|) + 1`.
- Line through two points as `A x + B y = C`: A = y2 − y1, B = x1 − x2, C = A·x1 + B·y1 (integers).
- Parallel ⇔ cross(b − a, d − c) == 0; perpendicular ⇔ dot == 0.

9.1.3 Angles and sorting by angle

```
// [stress-tested] sort vectors by polar angle in [0, 2pi), exact for integers; ties by distance not handled
bool half(Pt p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
void polarSort(vector<Pt>& v) {
    sort(all(v), [](Pt a, Pt b) {
        if (half(a) != half(b)) return half(a) < half(b);
        return a.cross(b) > 0;
    });
}
```

- Angle between vectors: `atan2(cross, dot)` (signed, robust) rather than `acos(dot/(|a||b|))` (inaccurate near 0, π).
- Rotating sweep around a point (max points visible in an angle window): polar sort, double the array with angles + 2π, two pointers.

9.1.4 Polygons

```
// [stress-tested] twice the signed area (positive if counter-clockwise). Exact in integers.
ll area2(const vector<Pt>& poly) {
    ll a = 0; int n = sz(poly);
    rep(i, 0, n) a += poly[i].cross(poly[(i + 1) % n]);
    return a;
}
// point in simple polygon: 1 inside, 0 outside, -1 on boundary. O(n), exact.
int inPolygon(const vector<Pt>& poly, Pt p) {
    int n = sz(poly); bool in = false;
    rep(i, 0, n) {
        Pt a = poly[i], b = poly[(i + 1) % n];
        if (onSegment(a, b, p)) return -1;
        if ((a.y > p.y) != (b.y > p.y)) {               // edge crosses the horizontal line y = p.y
            __int128 c = (__int128)(b - a).cross(p - a);
            if ((c > 0) == (b.y > a.y)) in = !in;       // p is strictly left of an upward edge, etc.
        }
    }
    return in ? 1 : 0;
}
```

- **Pick's theorem** (lattice polygon): Area = I + B/2 − 1 ⇒ interior points I = (area2 − B + 2) / 2 with B = Σ gcd(|dx|, |dy|) over edges.
- Centroid of polygon: C = Σ (p_i + p_{i+1})·cross(p_i, p_{i+1}) / (3·area2).
- **Point in convex polygon in O(log n):** binary search the fan triangle from vertex 0 (`orient` with poly[0], poly[mid]).
- Polygon is convex ⇔ all consecutive turns have the same sign (ignore zero for collinear).
- Area of a polygon cut by a half-plane: walk edges, keep inside vertices + intersection points (Sutherland–Hodgman), O(n).
- **Polar area** (curve r(θ)): area = $\int r(\theta)^2/2\, d\theta$ (ACPC 2021 *Wonky Pizza*: r linear in θ → closed-form cubic; then binary search the cut angle).

```
// [stress-tested] strictly convex CCW polygon (n >= 3): is p inside or on the boundary? O(log n)
bool inConvex(const vector<Pt>& poly, Pt p) {
    int n = sz(poly);
    if (poly[0].cross(poly[1], p) < 0 || poly[0].cross(poly[n - 1], p) > 0) return false;
    int lo = 1, hi = n - 1;
    while (hi - lo > 1) { int mid = (lo + hi) / 2; (poly[0].cross(poly[mid], p) >= 0 ? lo : hi) = mid; }
    return poly[lo].cross(poly[lo + 1], p) >= 0;
}
// [stress-tested] part of a polygon to the LEFT of the directed line s -> e (Sutherland–Hodgman).
// Works for convex polygons (for non-convex ones the result may have degenerate bridges).
vector<Pd> polygonCut(const vector<Pd>& poly, Pd s, Pd e) {
    vector<Pd> res; int n = sz(poly);
    rep(i, 0, n) {
        Pd cur = poly[i], prev = poly[(i + n - 1) % n];
        double a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a > 0) != (b > 0)) res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a > 0) res.push_back(cur);
    }
    return res;
}
```

Intersection of convex polygons / half-planes (small n): start from a big box and `polygonCut` by every half-plane, $O(n^2)$.

9.1.5 Precision checklist

- Use `long double` if values $\sim 10^9$ and you need 1e-6 after subtractive operations.

- Never test `==` on doubles; `fabs(a - b) < EPS`. Choose EPS by scale: coordinates 10^4 → EPS 1e-9 fine; 10^9 → use relative error.
- `sqrt` of a slightly negative number → NaN: `sqrt(max(0.0, x))`. Same for `acos(min(1.0, max(-1.0, x)))`.
- Print `-0.000` as `0.000`: add `0.0` or check `fabs(x) < 5e-10`.
- Integer geometry: test collinearity / orientation exactly; compare distances by squared lengths.

9.2 Convex Hull, Calipers, Closest Pair, Circles, Integration

Needs `P`, `Pt`, `Pd` from §9.1.

9.2.1 Convex hull (monotone chain)

```
// [stress-tested] counter-clockwise, no collinear points, no duplicate first/last. Handles n <= 2.
vector<Pt> convexHull(vector<Pt> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<Pt> h(sz(pts) + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (Pt p : pts) {
            while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0) t--;   // < 0 keeps collinear points
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

- Hull perimeter/area: sum edge lengths / `area2` of the hull.
- Keeping collinear boundary points (`< 0`) breaks when *all* points are collinear (returns duplicates) — handle that case separately.
- **Online upper hull** for CHT-like problems → §4.4. **Dynamic hull** is rarely needed.
- Fractal/self-similar shapes (ACPC 2020 *Wrapping Trees*): the hull of the limit set is determined by extreme points of the generator; reason about which corners survive.

9.2.2 Rotating calipers

```
// [stress-tested] squared diameter (farthest pair) of a convex polygon given CCW (output of convexHull)
ll diameter2(const vector<Pt>& h) {
    int n = sz(h);
    if (n == 1) return 0;
    if (n == 2) return (h[0] - h[1]).norm2();
    ll best = 0;
    for (int i = 0, j = 1; i < n; ++i) {
        while ((h[(i + 1) % n] - h[i]).cross(h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
        best = max({best, (h[i] - h[j]).norm2(), (h[(i + 1) % n] - h[j]).norm2()});
    }
    return best;
}
```

Same two-pointer skeleton gives: minimum width (min over edges of max distance to the antipodal vertex), smallest enclosing rectangle, max distance between two convex polygons.

9.2.3 Closest pair of points

```
// [stress-tested] squared distance of the closest pair, O(n log n)
ll closestPair2(vector<Pt> p) {
    sort(all(p), [](Pt a, Pt b) { return a.x < b.x; });
    set<pair<ll, ll>> S; ll best = LLONG_MAX; int j = 0;
    rep(i, 0, sz(p)) {
        ll d = (ll)ceil(sqrt((long double)best));
        while (p[i].x - p[j].x > d) { S.erase({p[j].y, p[j].x}); ++j; }
        auto lo = S.lower_bound({p[i].y - d, LLONG_MIN}), hi = S.upper_bound({p[i].y + d, LLONG_MAX});
        for (auto it = lo; it != hi; ++it) {
            ll dx = p[i].x - it->second, dy = p[i].y - it->first;
            best = min(best, dx * dx + dy * dy);
        }
        S.insert({p[i].y, p[i].x});
    }
    return best;
}
```

9.2.4 Circles

```
// [stress-tested] circumcentre of triangle ABC (non-collinear)
Pd ccCenter(Pd A, Pd B, Pd C) {
    Pd b = C - A, c = B - A;
    return A + (b * c.norm2() - c * b.norm2()).perp() / b.cross(c) / 2;
}
// [stress-tested] minimum enclosing circle, expected O(n) (Welzl, randomised)
pair<Pd, double> minEnclosingCircle(vector<Pd> ps) {
    shuffle(all(ps), mt19937(1));
    Pd o = ps[0]; double r = 0, EPS = 1 + 1e-9;
    rep(i, 0, sz(ps)) if ((o - ps[i]).len() > r * EPS) {
        o = ps[i]; r = 0;
        rep(j, 0, i) if ((o - ps[j]).len() > r * EPS) {
            o = (ps[i] + ps[j]) / 2; r = (o - ps[i]).len();
            rep(k, 0, j) if ((o - ps[k]).len() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]); r = (o - ps[i]).len();
            }
        }
    }
    return {o, r};
}
// intersections of circle (c, r) with the infinite line (a, b)
vector<Pd> circleLine(Pd c, double r, Pd a, Pd b) {
    Pd ab = b - a, p = a + ab * ((c - a).dot(ab) / ab.norm2());
    double s = a.cross(b, c), h2 = r * r - s * s / ab.norm2();
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    Pd h = ab / ab.len() * sqrt(h2);
    return {p - h, p + h};
}
// intersections of two circles (0, 1 or 2 points); identical circles -> {}
vector<Pd> circleCircle(Pd a, double r1, Pd b, double r2) {
    Pd v = b - a; double d2 = v.norm2();
    if (d2 == 0) return {};
    double sum = r1 + r2, dif = r1 - r2;
    if (sum * sum < d2 || dif * dif > d2) return {};
    double p = (d2 + r1 * r1 - r2 * r2) / (2 * d2), h2 = r1 * r1 - p * p * d2;
    Pd mid = a + v * p, per = v.perp() * sqrt(max(0.0, h2) / d2);
    if (h2 <= 0) return {mid};
    return {mid + per, mid - per};
}
```

- Tangent length from external point P: $\sqrt{(|PC|^2 - r^2)}$. Angle subtended: $2\cdot\text{asin}(r / |PC|)$.
- Area of intersection of two circles with distance d: `r1²·acos((d² + r1² − r2²)/(2 d r1)) + r2²·acos((d² + r2² − r1²)/(2 d r2)) − ½√((−d+r1+r2)(d+r1−r2)(d−r1+r2)(d+r1+r2))` (handle containment / disjoint first).
- Circle through 3 points: `ccCenter`. Point inside circle: compare squared distances.
- Moving circle / point vs shrinking zone (ACPC 2018 *PUBNite*): write distance as a function of time, binary/ternary search the time, or solve the quadratic directly.

9.2.5 Sphere / 3-D quick facts

- Great-circle distance between points with unit vectors u, v on a sphere radius R: `R·acos(u·v)` (clamp the dot). Chord length `2R·sin(θ/2)`.
- Lat/long → xyz: $(\cos\varphi\cos\lambda, \cos\varphi\sin\lambda, \sin\varphi)$.
- Moving along a tangent line a distance s from the surface point: distance to centre $\sqrt{(R^2 + s^2)}$ (ACPC 2023 *Flying Straight* compares arc vs tangent).
- 3-D cross product `(a.y b.z − a.z b.y, a.z b.x − a.x b.z, a.x b.y − a.y b.x)`; volume of tetrahedron $= |a\cdot(b\times c)|/6$.

9.2.6 Numerical integration and root finding

```
// composite Simpson on [a, b] with n (even) intervals; error O(h^4)
template <class F> double simpson(F f, double a, double b, int n = 2000) {
    double h = (b - a) / n, s = f(a) + f(b);
    rep(i, 1, n) s += f(a + i * h) * (i & 1 ? 4 : 2);
    return s * h / 3;
}
```

- Area of a union of shapes: integrate the covered length of the vertical line $x = t$ (sweep events at all intersection x-coordinates, Simpson within each slab).
- Roots of a monotone function: binary search (§2.1). Polynomial root isolation: roots of the derivative split the axis into monotone pieces.

10. Implementation, Interactive, Misc

10.1 Interactive Problems

ACPC 2024 had two (*Four Questions*, *Snow Way Out*). They punish protocol mistakes more than algorithmic ones.

10.1.1 Protocol checklist

- **Flush after every query**: `cout << "? " << x << endl;` (`endl` flushes) or `cout.flush()`. With `printf` use `fflush(stdout)`.
- Read the judge's reply **immediately** after each query; never read ahead.
- Keep `cin.tie(nullptr)` *off* or flush manually: with `sync_with_stdio(false)` + `endl` it's fine.
- Exactly the output format: "q 43", "a 71", "3 4 ?" … including trailing tokens like `?` or `.`.
- After the final answer: **exit immediately** (`return 0`). Extra output = WA, waiting for input = TLE.
- If the judge can reply with an error (e.g. `-1`), exit at once.
- Query limits are hard: count your queries in the worst case *before* coding. "Adaptive" judges choose the hidden value to make you fail — your strategy must work for every consistent answer.

```cpp
// sketch: typical skeleton
ll ask(ll s) {
    cout << "q " << s << endl;            // endl flushes
    ll r; cin >> r;
    if (r < 0) exit(0);                   // judge error -> quit to get WA, not TLE
    return r;
}
int main() {
    // ... decide queries ...
    cout << "a " << answer << endl;
    return 0;
}
```

10.1.2 Strategy patterns

- **Binary search**: hidden x in [1, N] with "higher/lower" answers ⇒ ⌈log2 N⌉ queries. Ternary answers (<, =, >) save a little.
- **Information counting**: with q queries each having k possible replies, you can distinguish at most k^q cases. If $k^q <$ #candidates, the intended solution exploits structure (e.g. replies carry more information than they seem).
- **Number theory replies** (s^2 mod p for unknown prime p, *Four Questions*): the reply $r = s^2 - kp$ means `p | s² - r`. Take gcd of several `s_i² - r_i` (choose s_i with $s_i^2 > p$, e.g. near the maximum allowed); strip small factors / test primality to isolate p. Use `__int128` if products exceed 64 bits.
- **Comparison-based (closer/farther/same)** on a small grid (*Snow Way Out*: 100 cells, 6 replies of 3 types): plan offline. Maintain the set of candidate cells consistent with all replies; choose the next query point that minimises the worst-case remaining set size (minimax). If greedy isn't enough, do a full game-tree search with memoisation on the candidate set (bitset of ≤ 100 cells → hash) — the search space is small. Precompute in your head / at runtime: 100 candidate cells × 100 query points × 3 replies per level.
- **Find a hidden permutation/array** with sum/xor queries: query pairs (1,2), (2,3), (1,3) to solve 3 unknowns, then chain.
- **Guess with random queries** when the judge is non-adaptive: random pivots resist bad cases.
- **Graph/tree discovery**: ask distances from one vertex, then reconstruct layer by layer.

10.1.3 Testing locally

Write a tiny interactor in Python that plays the judge, and connect with a pipe:

```python
# judge.py: python3 judge.py SECRET  (talks to solution via stdin/stdout)
import sys, subprocess
secret = int(sys.argv[1])
p = subprocess.Popen(["./sol"], stdin=subprocess.PIPE, stdout=subprocess.PIPE, text=True)
queries = 0
while True:
    line = p.stdout.readline().split()
    if not line: print("solution exited without answer"); break
    if line[0] == "q":
        queries += 1
        s = int(line[1]); p.stdin.write(f"{s * s % secret}\n"); p.stdin.flush()
    else:
        print("answer", line[1], "correct" if int(line[1]) == secret else "WRONG", "queries", queries); break
```

Loop it over many secrets (including worst cases: smallest/largest values, primes near the bound).

10.2 Parsing, Simulation, Dates, Big Numbers

10.2.1 Reading input safely

```
// frag
string line;
getline(cin, line);                       // whole line (after cin >> x, first call cin.ignore())
if (!line.empty() && line.back() == '\r') line.pop_back();   // Windows line endings
stringstream ss(line); vector<string> tok; string w;
while (ss >> w) tok.push_back(w);         // split on whitespace
// split on a custom delimiter
stringstream s2(line); string part;
while (getline(s2, part, ',')) { /* use part */ }
```

- Unknown number of test cases: `while (cin >> n)`. Lines until a sentinel: check before processing.
- Numbers with decimals that must be exact (money, "two decimal places"): read as string, convert to integer cents.
- Characters of a grid: read rows as strings; `vector<string> g(R)`; pad with a border of walls to avoid bounds checks.

10.2.2 Expression parsing (recursive descent)

Grammar with precedence: `expr = term (('+'|'-') term)*`, `term = factor (('*'|'/') factor)*`, `factor = number | '(' expr ')' | '-' factor`. Add a level per precedence class; right-associative operators (`^`) recurse on the right.

```
// [stress-tested] integers, + - * / ^ (right assoc), unary minus, parentheses, spaces allowed
struct Parser {
    string s; size_t i = 0;
    Parser(string str) { for (char c : str) if (c != ' ') s += c; }
    char peek() { return i < s.size() ? s[i] : 0; }
    ll expr() {
        ll v = term();
        while (peek() == '+' || peek() == '-') {
            char op = s[i++]; ll r = term(); v = op == '+' ? v + r : v - r;
        }
        return v;
    }
    ll term() {
        ll v = unary();
        while (peek() == '*' || peek() == '/') {
            char op = s[i++]; ll r = unary(); v = op == '*' ? v * r : v / r;
        }
        return v;
    }
    ll unary() {
        if (peek() == '-') { ++i; return -unary(); }
        if (peek() == '+') { ++i; return unary(); }
        return power();
    }
    ll power() {
        ll b = atom();
        if (peek() == '^') { ++i; ll e = unary(); ll r = 1; while (e-- > 0) r *= b; return r; }
        return b;
    }
    ll atom() {
        if (peek() == '(') { ++i; ll v = expr(); ++i; return v; }   // skip ')'
        ll v = 0; while (isdigit(peek())) v = v * 10 + (s[i++] - '0');
        return v;
    }
};
// usage: Parser p("2*(3+4)-5"); ll v = p.expr();
```

- **Mini-languages** (ACPC 2016 *Modelling Problems*: assignments, conditionals, assertions with nested blocks): same idea — one function per syntactic construct, each consumes its characters and returns a node/value. Build an AST if the program must be run many times (e.g. once per assignment of unknown bits: brute force 2^k executions).
- Evaluate the parse for *all* parenthesisations → that's interval DP (§4.1), not a parser.
- Shunting-yard (two stacks: values, operators) is an alternative when the grammar is flat.

10.2.3 Simulation tips

- Write the simulation exactly as the statement describes first (it's the brute force), then optimise only the hot part.
- **Event simulation:** priority queue of (time, event); process in time order; ties broken exactly as the statement says.

- **Skip ahead:** if the state repeats (cycle detection, §5.2) or many steps do nothing (jump to the next interesting time), skip them in O(1).
- Grids: `dr/dc` arrays; direction turning `d = (d + 1) % 4` (clockwise if order is N, E, S, W).
- Circular lists with removals (players leaving a game, Josephus): `std::list` or arrays `nxt/prv`; or a BIT/order-statistics structure to find the k-th alive player in O(log n).
- Josephus (every k-th removed, 0-indexed survivor): `J(1) = 0; J(n) = (J(n−1) + k) % n`.

10.2.4 Dates and times

```cpp
// [stress-tested]
bool leap(int y) { return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; }
int daysInMonth(int y, int m) {
    static const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return d[m - 1] + (m == 2 && leap(y));
}
// days since a fixed epoch (proleptic Gregorian); difference of two dates = difference of these
ll dayNumber(ll y, ll m, ll d) {
    if (m < 3) { y--; m += 12; }
    return 365 * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d - 1;
}
// day of week: (dayNumber(y, m, d) + 2) % 7 gives 0 = Monday ... 6 = Sunday
```

- Clock problems: work in seconds (or the LCM of the hand periods) to keep integers; hand angles: hour 0.5°/min, minute 6°/min, second 6°/s. Hands align at multiples of 12/11 hours (hour–minute) and 60/59 minutes (minute–second) — use exact fractions.
- Time zones / 24h wraparound: normalise with `((t % M) + M) % M`.

10.2.5 Big numbers without Python

- Use strings of digits; add/subtract/multiply-by-small/divide-by-small in O(n). Multiplying two big numbers: O(n·m) schoolbook or FFT (§7.5).
- Division by 10^k is just moving the decimal point (ACPC 2017 *Divide by 100*); strip trailing zeros and a trailing '.'.
- Comparing huge numbers: compare lengths first, then lexicographically.
- `__int128` covers up to $\sim 1.7 \cdot 10^{38}$.

10.2.6 Base conversion and misc

- To base b: repeated `% b`, `/ b`, reverse. Negative base -2: same loop but fix negative remainders (`r += 2; n += 1`).
- Roman numerals: greedy with the table {1000 M, 900 CM, 500 D, 400 CD, 100 C, 90 XC, 50 L, 40 XL, 10 X, 9 IX, 5 V, 4 IV, 1 I}.
- Rotate grid 90° clockwise: `res[j][R - 1 - i] = g[i][j]`. Transpose: `res[j][i] = g[i][j]`.
- Rounding to nearest integer: `llround(x)`; to k decimals: print with `setprecision(k)` (banker's rounding is not used by printf; halfway cases depend on binary representation — add 1e-9 if the statement demands "round half up").

10.3 Python as a Backup Language

Use Python for: arbitrary-precision integers, exact fractions, quick string manipulation, one-off math. Avoid it for heavy loops ($\geq 10^7$ operations) unless PyPy is used.

```python
import sys, math
from fractions import Fraction
from decimal import Decimal, getcontext
from functools import lru_cache
from itertools import permutations, combinations, product, accumulate
from collections import deque, defaultdict, Counter
import heapq, bisect

input = sys.stdin.readline                      # fast line input
data = sys.stdin.buffer.read().split()          # fastest: all tokens at once (bytes)
sys.setrecursionlimit(1 << 20)                  # deep recursion (may also need threading stack size)

# exact big integers
x = 10**100 + 7; y = pow(3, 10**18, x)         # modular power
inv = pow(5, -1, 10**9 + 7)                     # modular inverse (Python 3.8+)
r = math.isqrt(10**30)                          # exact floor sqrt
c = math.comb(100, 50); p = math.perm(10, 3)    # binomial / permutations
g = math.gcd(12, 18); l = math.lcm(4, 6)        # lcm: Python 3.9+

# exact rationals / high precision decimals
```

```
f = Fraction(1, 3) + Fraction(1, 6)             # 1/2
getcontext().prec = 100; d = Decimal(2).sqrt()

# memoised recursion
@lru_cache(maxsize=None)
def ways(n):
    return 1 if n <= 1 else ways(n - 1) + ways(n - 2)

# output many lines quickly
out = []
for i in range(5): out.append(str(i))
sys.stdout.write("\n".join(out) + "\n")

# formatting
print(f"{3.14159265:.6f}")                      # fixed 6 decimals
print(int("ff", 16), bin(10)[2:], format(255, "x"))
```

- `heapq` is a min-heap; push `(-priority, item)` for a max-heap.
- `bisect.bisect_left(a, x)` = `lower_bound`; `bisect_right` = `upper_bound`.
- Deep recursion in CPython crashes even with a high limit; run the solver in a `threading.Thread` with `threading.stack_size(1 << 27)`, or rewrite iteratively.
- Strings are immutable: build with a list and `"".join(...)`.
- Integer division floors toward $-\infty$ (`-7 // 2 == -4`), unlike C++ (truncates toward 0). `%` result has the sign of the divisor.